Databases project 2025-2026
Phase 1

Olympicaos

Member list (name and email): Stanisław Gancarczyk

stanislaw.gancarczyk@students.salle.edu.pl

Dataset: Olympic Games Dataset

Delivery date: 23.04.2026

Index

1 INTRODUCTION (1-2 PAGES) 2

1.1 PROJECT TEAM 2
1.2 DATASET DESCRIPTION 4
1.3 VALIDATION QUESTIONS 5

2 ENTITY-RELATIONSHIP MODEL 6

2.1 DIAGRAM 6
2.2 JUSTIFICATION (4-6 PAGES) 8

3 RELATIONAL MODEL 12

3.1 DIAGRAM 12
3.2 JUSTIFICATION (1-3 PAGES) 14

4 CONCLUSIONS 16

4.1 UTILIZATION OF RESOURCES (1 PAGE) 16
4.2 AI USAGE (IF NEEDED, 1-2 PAGES) 17
4.3 LESSONS LEARNED AND CONCLUSIONS (1 PAGE) 19

1 Introduction (1-2 pages)

The Olympic Games are one of the most complex and data-intensive sporting events globally, comprising thousands of athletes, coaches, technical officials, and events across many disciplines. My main reason for undertaking this project is to develop a fully functional and comprehensive database based on the Paris 2024 Olympic Games. Unlike storing data as information or knowledge, my aim is to develop a highly flexible database that is open to modifications, updates, and structural expansions in the future. Working with raw data in flat files poses a challenge for me as a developer, and I am driven by the challenge of converting these raw data files into a clean, efficient, and logical data structure.

In terms of my education, my main aim for undertaking this project is to bridge the gap between theory and practice in terms of database engineering. I aim to gain a deep and thorough understanding of both conceptual and relational data models as I work on my project. The analytical skills required in analyzing the data provided and determining relationships between data entities are important skills I aim to master. Moreover, I aim to gain a thorough understanding of the strict rules of database normalization, specifically the achievement of Third Normal Form (3NF), in order to maintain data integrity and prevent data redundancy and anomalies in my complex Olympic Games dataset.
Another major goal for myself, which I aim to accomplish with this project, is to attain a very high level of fluency and comfort with SQL. Through creating my own database design and creating complex validation questions, I aim to push myself to create very advanced queries to derive useful and even interesting information from the data set.

This project is also an exciting experience for me as it represents a very significant academic milestone. As an academic student at university in Poland, I had my first major experience with databases. My experience with databases was very heavy with PL/SQL, which is an extension of SQL for the Oracle relational database system. Although this experience has provided me with an excellent, hands-on experience with procedural database programming, creating functions, and even creating triggers, it has mostly been limited to working within an existing, pre-designed database structure. This project fortunately forces me to become a database architect. It allows me to utilize my foundational knowledge of SQL from Poland, but it also allows me to greatly expand my skill set with regard to the very initial phases of database design. The end goal for myself is to prove to myself that I can take raw, unorganized data and engineer it into a very professional relational database system.

1.1 Project team

My name is Stanisław Gancarczyk, and I am a Computer Science student with an age of 19. I hail from Poland. At the moment, I am pursuing my Computer Science degree at the Faculty of Electronics and Information Technology (EITI) at the Warsaw University of Technology. For this specific database project, I will be working independently and thus forming my own single-member team. This means I willl be responsible for all aspects and roles involved throughout the project lifecycle, starting from the initial stages of data exploration and Entity-Relationship modeling to the end stages of database design and query development.

In terms of my academic qualifications and knowledge about database systems, I have a solid foundation and understanding about database systems from my previous university days. In my previous academic career, I have been heavily exposed to SQL and, more specifically, Oracle PL/SQL procedural language. This project gives me an amazing opportunity to work independently and thus transition from procedural programming to the role and responsibility of a database architect.

Besides my academic and professional life, I make sure to have a good work-life balance by engaging in various hobbies and activities. Music is an integral part of my life. I have been playing the piano since my childhood. Besides this, I am a sports enthusiast and make sure to stay healthy and active by regularly going to the gym and playing football.

1.2 Dataset description

The primary dataset chosen for this project revolves around the Paris 2024 Olympic Summer Games. It consists of multiple interconnected CSV files that capture the logistical, demographic, and competitive aspects of the event. To fulfill the specific academic requirements of this project, I also expanded this domain by generating an additional, custom dataset using the Mockaroo data generation tool.

The core dataset includes files such as athletes.csv, coaches.csv, technical_officials.csv, teams.csv, events.csv, schedules.csv, venues.csv, nocs.csv, medalists.csv and medals.csv. These files contain a wealth of information, ranging from the biographical details of over 11,000 athletes to the precise scheduling of events across various Parisian venues.

Structural Characteristics and Normalization Challenges

From a database engineering perspective, the most prominent characteristic of this raw dataset is its highly denormalized, "flat" structure. During my initial data exploration, I identified several structural challenges that make it an excellent candidate for rigorous conceptual modeling and normalization up to the Third Normal Form (3NF):

- **Severe Data Redundancy:** Geographic identifiers (such as country_code, country, and country_long) are heavily repeated across almost every file, including athletes and coaches. This creates transitive dependencies that must be resolved.

- **1NF Violations:** Several columns store non-atomic values. For example, in the teams.csv file, the lists of assigned athletes and their corresponding codes are stored as text arrays

- **Derived Attributes:** The dataset includes pre-calculated columns, such as num_athletes and num_coaches , which pose a risk to data integrity in a relational model and should be calculated dynamically instead.

External Mockaroo Dataset (Sponsors)
To successfully meet the project requirement of integrating an independently created CSV file, I generated a realistic sponsorship dataset (sponsors.csv). This dataset introduces the concept of corporate financial backing into the Olympic domain. It contains dynamically generated records with attributes including sponsor_id, company_name, industry, and sponsorship_budget. I intentionally designed this dataset to include a country_code field , allowing it to be seamlessly linked to the NOC(Country entity) in my normalized ER model.

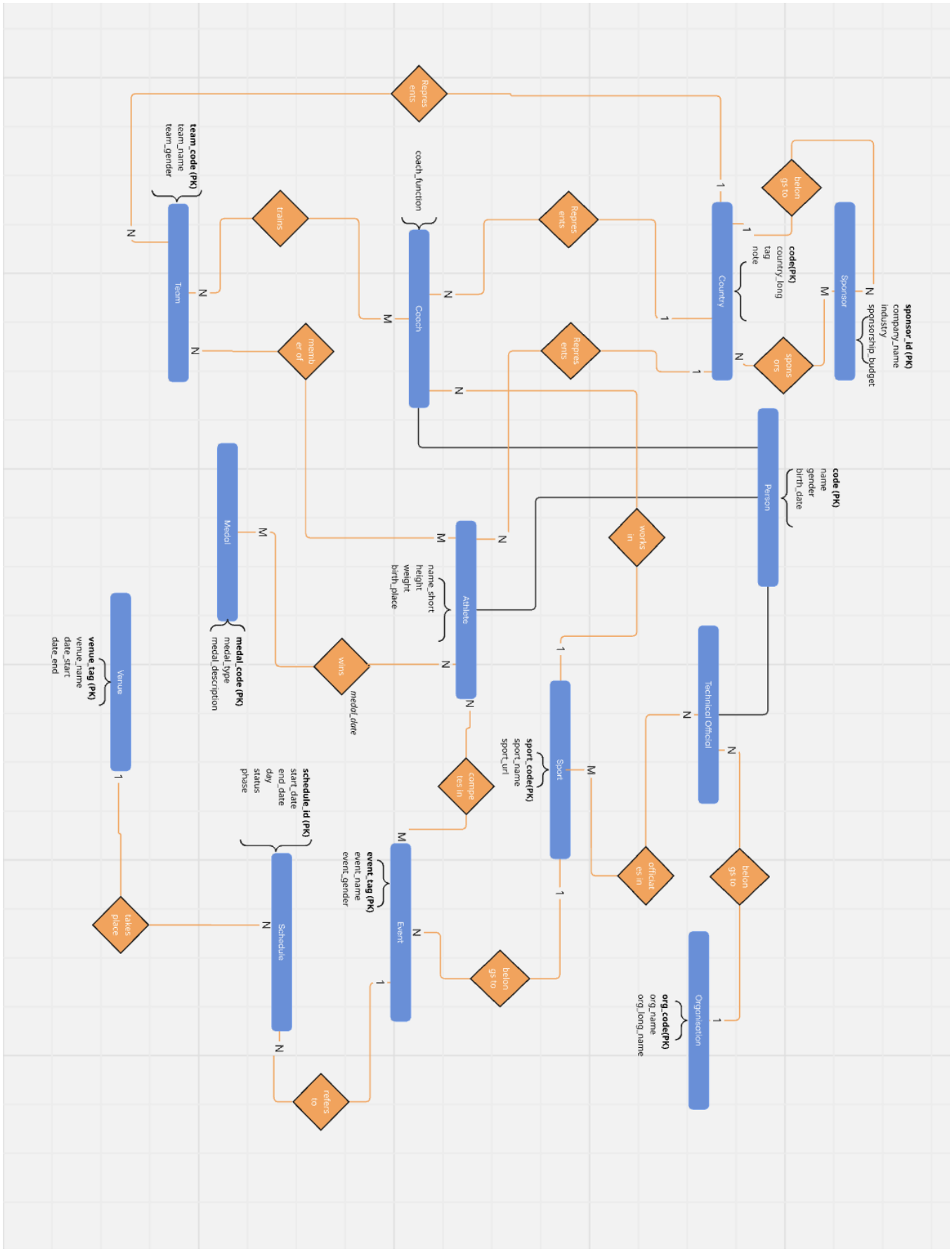
1.3 Validation questions

1. Which athletes have won more than two gold medals in a single sport?
2. What is the total sponsorship budget for countries that won at least one medal?
3. Which coaches train both an individual athlete and a national team at the same time?
4. What is the average age of female athletes competing in artistic gymnastics?
5. Which venues hosted the highest number of events during the Olympic games?
6. How many technical officials belong to the international gymnastics federation (FIG)?
7. Which country has the highest ratio of coaches to athletes?
8. List all the athletes who were born in a different country than the country they represent?
9. Which companies from the "Finance" industry provide sponsorship to the top 5 countries in the overall medal standings?
10. What is the most common function among technical officials who officiate in Water Polo (count)?
11. Which national team won the most medals?
12. What are the names of their assigned coaches?
13. Whats top 3 sports with the youngest athletes on average?
14. Select athletes that are registered to compete in more than one distinct sport?
15. What are the companies from industry Healthcare that sponsor countries with over 100 participating athletes?
16. What coaches who work in sport where their represented country failed to win any medals?
17. Which athletes won a medal in every event that took place exacly on their birthday?
18. What is the total number of athletes participating only in team events compared to those who play only in individual ones?
19. Which venues only hosted events for a single specific sport discipline?
20. Are there any countries that have female coaches assigned to train male teams?
21. How many are they?
22. List all technical officials who officiate in sports where France won a gold medal
23. Which athletes won medals in both a team event and a individual event?
24. How many events had finished before 25th of July?
25. What is the avg height and weight of male athletes who won a medal in basketball compared to those who did not?
26. Which country received most money from sponsors but didn't win anything?
27. How many athletes have same coach but are from different countries?
28. What is the sport that has most number of different venues for it events?
29. Who is the oldest athlete?
30. Who is the oldest technical official?
31. Who they represent?(organization)
32. Which country has more than 5 coaches in total?

2 Entity-relationship model

2.1 Diagram

https://miro.com/welcomeonboard/TldoOUxjd1NpeUtnejVmWnpNNGoxOWkxbFJDa0E0dmVJL2Q3QmU1eEJldEZNZ2xMejcyZURMZStBSFYyTm1GbDVZbThnOXpmb1NjeGV4UUZjempOdnpzczhwV2I3aUtWdWVCa0gzZzhyQ3luYkRCL3BKNEZZNHdZQ3d5bXV6ZGxBd044SHFHaVlWYWk0d3NxeHNmeG9BPT0hdjE=?share_link_id=138974504989


Sponsor
sponsor_id (PK)
company_name
industry
sponsorship_budget
belongs to
Country
code(PK)
country_long
tag
note
sponsors
Represents
Person
code (PK)
name
gender
birth_date
Coach
coach_function
Team
team_code (PK)
team_name
team_gender
trains
member of
Athlete
name_short
height
weight
birth_place
works in
Technical Official
officiates in
belongs to
Organisation
org_code(PK)
org_name
org_long_name
Sport
sport_code(PK)
sport_name
sport_url
competes in
Event
event_tag (PK)
event_name
event_gender
belongs to
Schedule
schedule_id (PK)
start_date
end_date
day
status
phase
refers to
takes place
Venue
venue_tag (PK)
venue_name
date_start
date_end
Medal
medal_code (PK)
medal_type
medal_description
wins
medal_date

2.2 Justification (4-6 pages)

Conceptual Model Design Decisions

The following section provides a detailed justification of all design decisions made during the development of the Entity-Relationship (ER) conceptual model for the Paris 2024 Olympic Games database. Each decision is grounded in the analysis of the provided CSV datasets, the principles of database normalization (1NF, 2NF, 3NF), and the goal of producing a model that is both semantically correct and practically useful for answering complex queries.

Generalization: Person Entity

One of the most significant structural decisions in the model was the introduction of a generalized PERSON entity, from which three specialized sub-entities — ATHLETE, COACH, and TECHNICAL OFFICIAL — are derived. This decision was motivated by a direct analysis of the source CSV files: athletes.csv, coaches.csv, and technical_officials.csv.

All three files share a common set of attributes: a unique code (PK), name, gender, and birth_date. Storing these attributes redundantly in three separate, independent entities would violate the principle of data economy and introduce unnecessary duplication. In the context of normalization, while this is not a direct violation of 1NF, 2NF, or 3NF within a single table, it represents a semantic redundancy at the conceptual level — a problem that generalization is specifically designed to address.

The PERSON entity therefore captures all shared attributes:

- code (PK) — unique identifier shared across all three CSV files
- name — full name of the individual
- gender — present in all three source files
- birth_date — present in all three source files

Each sub-entity then contains only the attributes that are unique to its role. ATHLETE holds name_short, height, weight, and birth_place — all of which are exclusive to athletes.csv and have no equivalent in the other two files. COACH retains only function, which describes the coaching role. TECHNICAL OFFICIAL similarly retains only function. This clean separation avoids storing empty or null values and ensures that each entity remains cohesive and meaningful.

It is also worth noting that the codes in athletes.csv, coaches.csv, and technical_officials.csv do not overlap (with negligible exceptions confirmed by data analysis), which confirms that these three roles represent truly distinct groups of individuals. This further validates the generalization pattern: one shared identity structure, three specialized behavioral extensions.

In the ER diagram, the inheritance relationship is represented by plain lines connecting PERSON to each sub-entity, without relationship diamonds — consistent with the generalization notation used in this course.

The NOC(country) entity and its central role

The country entity is derived directly from nocs.csv and serves as one of the most connected nodes in the entire model. Its attributes — noc_code (PK), country_long, tag, and note — appear in virtually every other CSV file in the dataset, which immediately justified its promotion to a standalone entity.

From the perspective of Third Normal Form (3NF), storing country_long, tag, and note inside every entity that references a country (athletes, coaches, teams, sponsors) would create transitive dependencies. For example, in the ATHLETE entity: code → noc_code → country_long. The attribute country_long does not depend on the athlete's code — it depends on the noc_code. This is a textbook 3NF violation. By extracting NOC into its own entity and using noc_code as a foreign key, all transitive dependencies are eliminated.

It is important to note that NOC is not simply a list of countries in the conventional geographic sense. The nocs.csv file contains entries such as AIN (Athletes of Individual Neutrality — stateless or neutral athletes) and historical entities like AHO (Netherlands Antilles, which no longer exists as a country). The note field contains Olympic membership status codes (P for Participant, H for Historical, NP for Non-Participating). This confirms that NOC is a distinct Olympic administrative concept, not a generic geographic entity, and fully deserves its own dedicated entity in the model.

The Country entity maintains 1:N relationships with ATHLETE, COACH, TEAM, and SPONSOR (each individual or team belongs to exactly one country), and an N:M relationship with SPONSOR (a sponsor may support multiple countries, and one country may have multiple sponsors).

Organisation entity for technical officials

A separate ORGANISATION entity was introduced specifically to handle the affiliation of Technical Officials. This decision was not immediately obvious but became necessary upon closer inspection of the technical_officials.csv data.

The field organisation_code in technical_officials.csv does not exclusively contain country codes. While many technical officials are affiliated with a national committee (e.g., ITA for Italy, CRO for Croatia), a significant portion are affiliated with international sports federations whose codes are not present in nocs.csv at all — for example: FIBA (International Basketball Federation), FIG (International Gymnastics Federation), UCI (Union Cycliste Internationale), WT (World Taekwondo), and WAQU (World Aquatics).

Data analysis confirmed that out of 107 unique organisation_code values found in technical_officials.csv, 14 are not present in nocs.csv. Linking TECHNICAL OFFICIAL directly to COUNTRY would therefore be semantically incorrect for a significant portion of records — a referee appointed by FIBA does not belong to a national committee but to an international federation.

The ORGANISATION entity resolves this by providing a generalised affiliation anchor with attributes organisation_code (PK), organisation, and organisation_long. This allows both country-affiliated and federation-affiliated officials to be represented correctly without forcing a false equivalence between national committees and international bodies.

Sport and Event: Separating discipline from Competition

The model distinguishes between SPORT and EVENT as two separate entities, a decision that reflects the actual hierarchical structure of the Olympic Games and prevents a significant normalization problem.

SPORT represents a discipline (e.g., Athletics, Swimming, Archery), while EVENT represents a specific competition within that discipline (e.g., Men's 100m, Women's 200m Freestyle, Men's Individual). The events.csv file explicitly maps each event to its parent sport, confirming this hierarchy. Data analysis shows that individual sports contain multiple events: for example, Artistic Gymnastics has 14 events, and Archery has 5.

Without this separation, the sport name would need to be repeated in every EVENT record — a violation of 3NF, since sport_name would depend on sport_code rather than on event_tag. By making EVENT reference SPORT via a foreign key (sport_code FK), this transitive dependency is eliminated and the model correctly reflects that one SPORT contains many EVENTs (1:N relationship).

It is also worth noting that the concept of sport and discipline in the source data are effectively synonymous — the disciplines column in athletes.csv and coaches.csv contains the same values as the sport column in events.csv. This was an explicit design decision: we merged these into a single SPORT entity rather than creating two near-identical entities that would duplicate the same information.

Team entity and its relationships

The TEAM entity is sourced from teams.csv and represents a named group of athletes competing together in a specific discipline. Its core attributes are team_code (PK), team_name, and team_gender.

The relationship between ATHLETE and TEAM was determined to be N:M based on direct data analysis: 650 athletes in the dataset are members of more than one team simultaneously. This occurs because, for example, a swimmer may be part of both a relay team and an individual classification team, or because athletes participating in mixed-gender events are counted in multiple team rosters. Representing this as a 1:N relationship would have been factually incorrect and would have lost data.

Similarly, the COACH–TEAM relationship is N:M: data analysis confirmed that 58 coaches are responsible for more than one team (e.g., a coach may manage both the men's and women's teams in the same discipline). While intuition might suggest a 1:1 relationship, the data clearly indicates otherwise.

TEAM also has a N:1 relationship with COUNTRY — every team represents exactly one national committee, and this was confirmed by data analysis showing zero teams with more than one country code. Storing the country name inside TEAM would violate 3NF (team_code → noc_code → country_long), so COUNTRY is referenced via foreign key.

The medal entity and the Wins relationship

The MEDAL entity represents the three types of Olympic medals: Gold, Silver, and Bronze. Although this entity contains only a small number of records, its existence as a standalone entity is justified by the nature of the relationship between athletes and medals.

The relationship ATHLETE — WINS — MEDAL is N:M for two distinct reasons. First, a single athlete can win multiple medals across different events (200 athletes in the dataset hold more than one medal). Second, a single medal type (e.g., Gold) is awarded to many athletes across many events. If MEDAL were simply an attribute of ATHLETE, it would be impossible to track which medal was won in which event, or to query all gold medalists across the entire Games.

The WINS relationship carries its own attribute — medal_date — which records when the medal was awarded. This is a relationship attribute: it does not belong to ATHLETE (the athlete does not have a single fixed medal date) nor to MEDAL (the medal type has no inherent date). It belongs to the specific act of winning, which is exactly what relationship attributes are designed to capture.

Schedule and Venue entities: Modelling the event timetable

The SCHEDULE entity is sourced from schedules.csv and represents individual time slots in the Olympic timetable. Its attributes include schedule_id (PK, a surrogate key not present in the original CSV but added for relational model purposes), start_date, end_date, day, status, and phase.

SCHEDULE is linked to EVENT in a N:1 relationship — many schedule entries can refer to the same event across different phases (qualifications, semifinals, finals). Data analysis confirmed that 245 distinct event names appear in more than one schedule record.

VENUE is sourced from venues.csv and holds venue_tag (PK), venue_name, date_start, and date_end.

Storing venue_name or venue location details inside SCHEDULE would have produced a 3NF violation: schedule_id → venue_tag → venue_name. Extracting VENUE as a separate entity and referencing it via foreign key cleanly eliminates this dependency and centralises venue information.

Sponsor entity: custom generated Dataset

The SPONSOR entity represents the custom CSV file created using the Mockaroo data generation tool, fulfilling the project requirement to extend the original dataset with a new, linked file. The decision to model sponsorship was motivated by real-world relevance: the Olympic Games are one of the most heavily sponsored events in the world, and sponsor data would be a natural complement to any Olympic database.

The entity contains sponsor_id (PK), company_name, industry (sourced from a realistic list of industries), and sponsorship_budget (a numeric value representing the financial commitment). The country_code attribute links each sponsor to a Country, establishing that a sponsor is headquartered in a specific country.

Two relationships were defined for SPONSOR. First, a N:1 relationship — belongs to — with Country, indicating the sponsor's country of origin. Second, a N:M relationship — sponsors — with

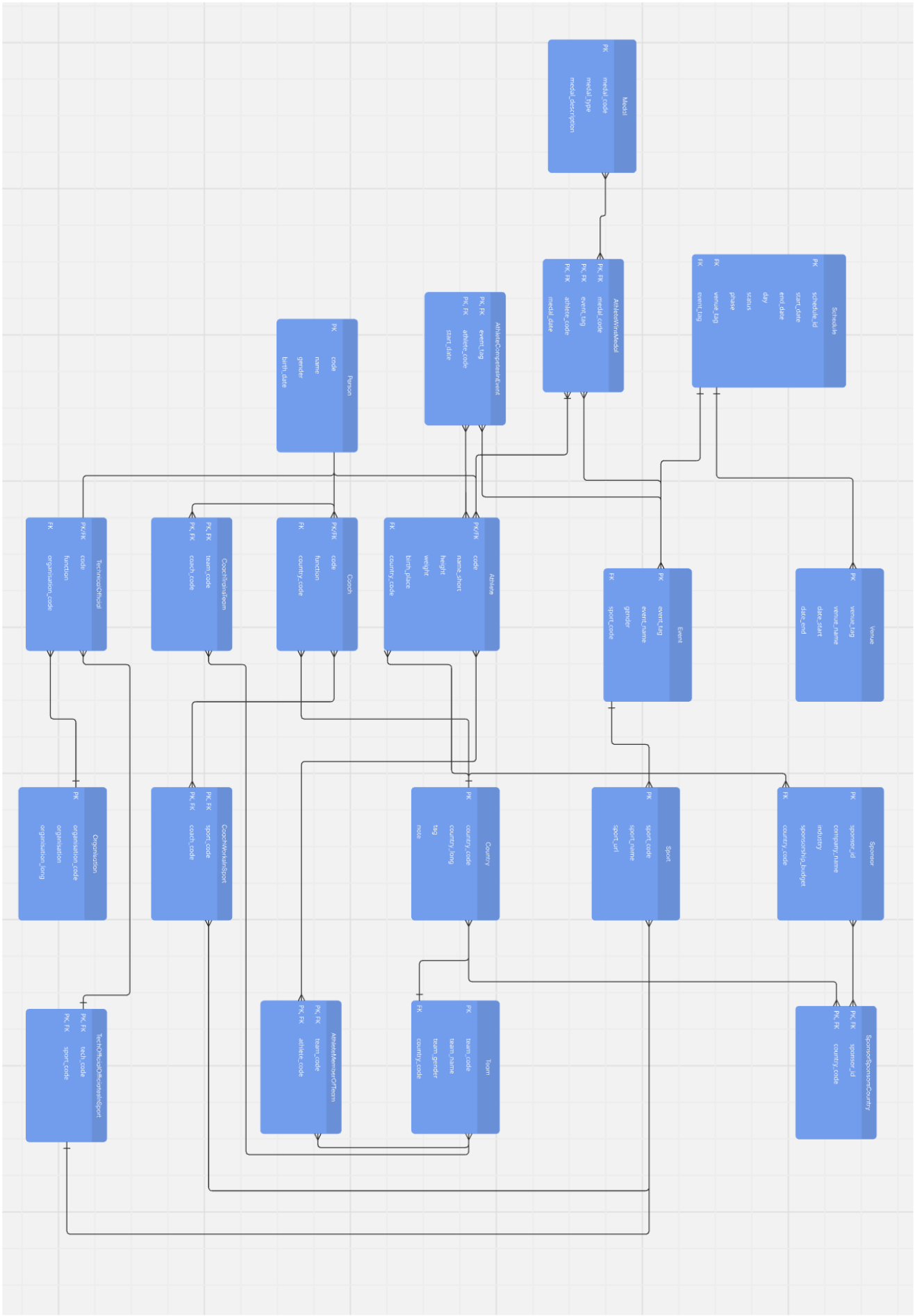
Country, representing which national teams or committees the sponsor financially supports. This N:M relationship is realistic: a multinational corporation such as a major technology firm may sponsor multiple national committees, and each Country may have multiple sponsors.

The country_code values in the generated CSV were restricted to a predefined list of Country codes sourced directly from nocs.csv, ensuring referential integrity between the new file and the existing dataset.

3 Relational model

3.1 Diagram

https://miro.com/welcomeonboard/TldoOUxjd1NpeUtnejVmWnpNNGoxOWkxbFJDa0E0dmVJL2Q3QmU1eEJldEZNZ2xMejcyZURMZStBSFYyTm1GbDVZbThnOXpmb1NjeGV4UUZjempOdnpzczhwV2I3aUtWdWVCa0gzZzhyQ3luYkRCL3BKNEZZNHdZQ3d5bXV6ZGxBd044SHFHaVlWYWk0d3NxeHNmeG9BPT0hdjE=?share_link_id=138974504989


MedalType
medal_code
medal_type
medal_description
AthleteWithMedal
medal_code
event_tag
athlete_code
medal_date
ScheduleIn
schedule_id
start_date
end_date
day
status
phase
venue_tag
event_tag
AthleteCompetesInEvent
event_tag
athlete_code
start_date
Person
code
name
gender
birth_date
TechnicalOfficial
code
function
organisation_code
CoachTrainsTeam
team_code
coach_code
Coach
code
function
country_code
Athlete
code
name_short
height
weight
birth_place
country_code
Event
event_tag
event_name
gender
sport_code
Venue
venue_tag
venue_name
date_start
date_end
Organisation
organisation_code
organisation
organisation_long
CoachWorksInSport
sport_code
coach_code
Country
country_code
country_long
flag
note
Sport
sport_code
sport_name
sport_url
Sponsor
sponsor_id
company_name
industry
sponsorship_budget
country_code
TechOfficialOfficiatesInSport
tech_code
sport_code
AthleteMemberOfTeam
team_code
athlete_code
Team
team_code
team_name
team_gender
country_code
SponsorSponsorsCountry
sponsor_id
country_code

3.2 Justification (1-3 pages)

The relational database model was developed from the conceptual database model called the Entity-Relationship Model. Every decision that was made during the conceptual stage had a tangible effect on the resulting relational model, and the paragraphs that follow will illustrate why these decisions were made.

From Generalization to Inheritance tables

The generalization PERSON within the ER model corresponds to a parent table named “Person” having the common attributes (code, name, gender, birth_date) and three child tables (Athlete, Coach, and TechnicalOfficial) which have specific attributes only. Each of these child tables uses the same code attribute both as a primary key and as a foreign key for referring to the table “Person.” This design strategy was selected over the strategy of table merging where all attributes were stored in a single table, since table merging would have resulted in many NULL values in which the attributes “height,” “weight,” and “function” of the table “Coach” and “Athlete,” respectively, would be NULL.

N:M relationships solution

Each N:M association within the ER model was converted into a junction table where the PK of the table is the combination of the two foreign keys. This is the correct way of representing N:M associations in a relational schema design. For instance, the association between ATHLETE and EVENT as “competes in” is now a table called AthleteCompetesInEvent where the combination of athlete_code and event_tag acts as the PK of the table, and start_date, a relationship attribute, is also included without any modification from the ER diagram. Likewise, the table AthleteWinsMedal has an additional attribute medal_date, which is not a part of the PK, as it represents the winning of a medal, not the athlete or the medal itself.

Foreign key placement - 1:N relationships

The foreign key for all 1:N associations was made at the end of the many side according to convention in relation database design. As an example, the sport_code is the foreign key of the table Event which refers to the table Sport since one sport consists of many events but each event can only belong to one sport. This is applicable also for Schedule which refers to both Event and Venue, for Team referring to Country, and for TechnicalOfficial referring to Organisation.

Organisation Table and Referential Integrity

The Organisation table was a straight conversion from the Organisation entity in the ER model, but it had to be created on its own due to the need to enforce referential integrity when it comes to records for TechnicalOfficials. Due to the fact that organisation_code contains both the NOC codes and international federation codes such as FIBA, FIG and UCI, that do not have entries in the Country table, there was no other choice but to create an Organisation table on its own.

Surrogate Keys and Naming Conventions

To facilitate the creation of a surrogate primary key (schedule_id), a surrogate key was generated for the Schedule table since there is no natural key within the source CSV file that uniquely identifies each record within the Schedule table. While creating a composite key based on the attributes start_date and event would have been possible, doing so may prove to be ineffective due to the fact that there could be several phases of the same event that take place on the same date.

Normalisation Compliance

The relational schema resulting from this process conforms to all three levels of normalisation. All entities in this database are composed only of atomic data (1NF). Any entity that has a composite key, all attributes that are not part of the key depend entirely on the whole composite key and not a part thereof (2NF) — this applies to all junction entities in the schema. No attribute that is not part of the key depends transitively on another attribute that is not the key (3NF). Attributes such as countries belong in the Country entity; sports belong in the Sport entity, and so forth.

4 Conclusions

4.1 Utilization of resources (1 page)

Phase	Stanislaw	Total
Data exploration	10	10
Entity-Relationship model	35	35
Relational model	15	15
Total:	60	60

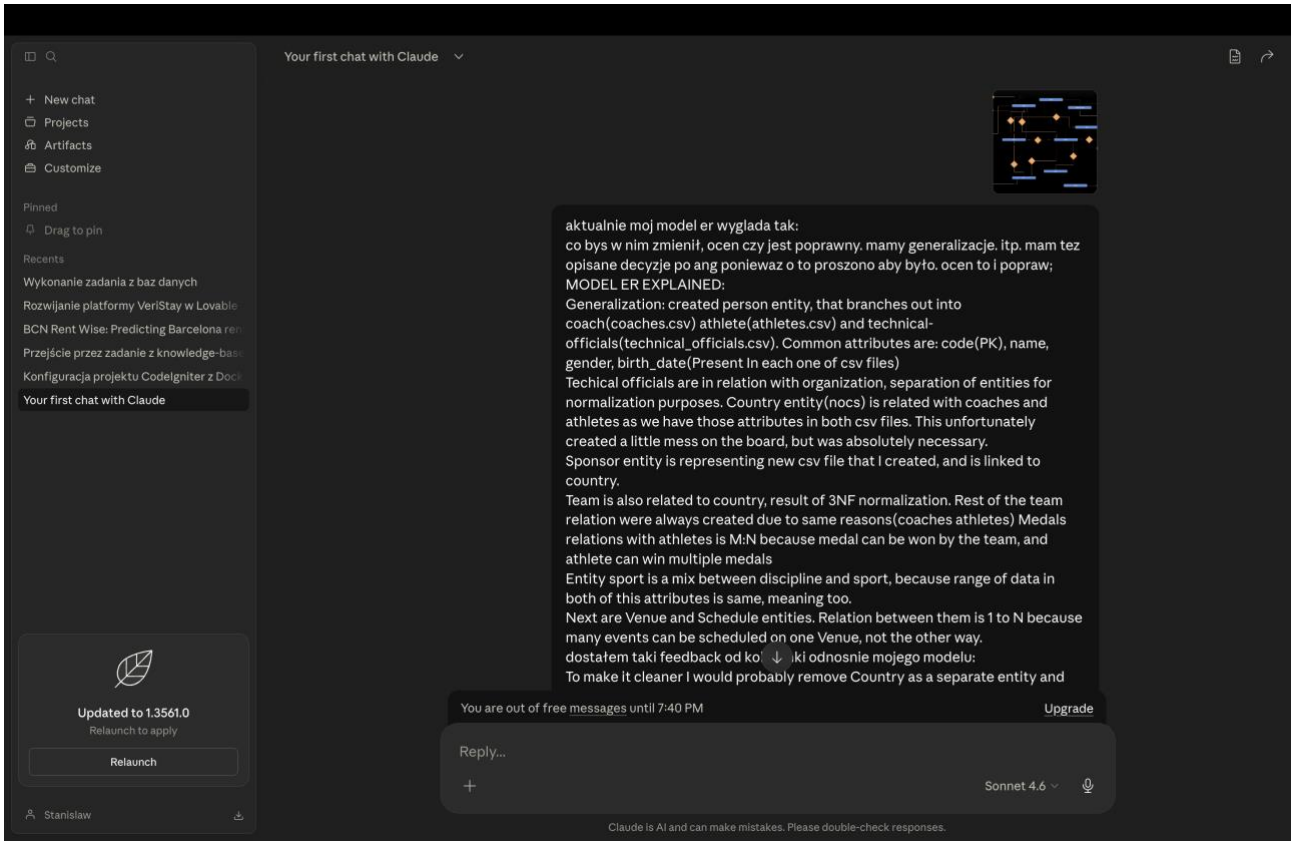
Entity-Relationship model was the hardest for me, and the most time consuming. I couldn't decide which entities to create, to be useful. Relationships between them were also hard for me to establish. I hadn't much experience with it in Poland, even though I worked with databases before.

I didn't know where to look for help. Ai wasn't giving me satisfying solutions, and to be honest was not useful at all with this assignment.

4.2 AI usage (if needed, 1-2 pages)

Artificial intelligence was helpful with creating this file. I have difficulties with constructing professional sentences in English, because that's not the language in which I study in Poland. The way I worked with ai here was just sentencing my thoughts and asking Google Gemini to rephrase them in English so they sound more professional, and they better convey the meaning of my thoughts. As I mentioned before, Ai wasn't much help when it came to creating ER model. Answers were not satisfying me, were chaotic and changed all the time due to the questions I asked. I learned that possibly it would take me much less time to figure this conceptual model myself from beginning without asking chat for any clues. So the prompts I used were: "Expand this justification, make it look more professional, include all key elements from justification below: [polish text]"

Ai was helpful when it came to using miro. It was my first time using this site, so tips given by ai were invaluable. The same applies to using Mockaroo. Nocs.csv file analysis would be very time consuming on my own, thanks to Ai I could create csv file fully integrated with my datasets. (extracting every unique country code from nocs.csv)



Translation of this claude query:

“Currently my er model looks like this:
what would you change in it, assess whether it is correct. we have generalizations. etc. I also have the decisions described in English because that's what they asked for. evaluate it and improve it;
MODEL ER EXPLAINED:
Generalization: created person entity, that branches out into coach(coaches.csv) athlete(athletes.csv) and technical-officials(technical_officials.csv). Common attributes are: code(PK), name, gender, birth_date(Present In each one of csv files)
Technical officials are in relation with organization, separation of entities for normalization purposes. Country entity(nocs) is related to coaches and athletes as we have those attributes in both csv files. This unfortunately created a little mess on the board, but was absolutely necessary.
Sponsor entity is representing new csv file that I created, and is linked to country.
Team is also related to country, result of 3NF normalization. Rest of the team relation were always created due to same reasons(coaches athletes) Medals relations with athletes is M:N because medal can be won by the team, and athlete can win multiple medals
Entity sport is a mix between discipline and sport, because range of data in both of this attributes is same, meaning too.
Next are Venue and Schedule entities. Relation between them is 1 to N because many events can be scheduled on one Venue, not the other way.
I received this feedback from a friend regarding my model:
To make it cleaner I would probably remove Country as a separate entity and just keep it as an attribute, because it doesn't really add much on its own. Technical Office also feels a bit forced, so I would merge it with whatever entity is actually organizing things. Gymnasium and Venue look redundant, so I’d combine them into a single entity. For Medal, if it’s only used to represent results, I’d turn it into an attribute instead of keeping it as a full entity. Sponsor is only fine if it plays an important role across multiple parts of the system; otherwise, it might be unnecessary.

I don't know if I can remove country because it's a rather important entity, and these fields (columns) are in multiple CSV files. I don't understand the idea of combining technical official, gymnasium, and venue. Please tell me if I should do this. The graph is currently a bit unreadable. Please analyze it carefully.”

4.3 Lessons learned and conclusions (1 page)

Working on this project made me better understand how databeses are being created from scratch. I faced many difficulties during the process of creating this file and models, but now I can say I finally know how It works.

The biggest lesson I understood is to always analyze the data before you start designing anything. In the beginning, some assumptions seemed totally reasonable — like thinking every coach trains just one team, or that every technical official is tied to a country. It made sense on paper, but once I dove into the actual CSV files, i found the truth. Some coaches managed multiple teams, and a lot of technical officials were part of international federations instead of national committees. That proved the point that a conceptual model isn't about how things should fit — it's about how things actually work in the real world.

Another thing I learned was how normalization affects complexity. At first, splitting off entities like Country, Sport, Venue, and Organisation into their own tables seemed like overkill. Why not just stick the country name right into every athlete's record? But as the model grew, it turned out that each separation solved a real problem — no more update headaches, no extra copies, and referential links stayed intact. Normalization stopped feeling like a dry theory and started making everything more manageable.

Figuring out how to generalize was tricky, especially creating a PERSON entity that covered Athlete, Coach, and TechnicalOfficial. I realized that those csv files were very similar and copying myself all over again would be unnecessary. Choosing when to generalize and when to keep things separate is something you only really learn by doing, and this project made me see how important that decision actually is.

Translating the ER model into a relational schema was less straightforward than I thought. Deciding on surrogate keys, figuring out where foreign keys should go, and naming junction tables all called for judgment. I hoped it would take me much less time but it came out to be really time-consuming.

Working with actual data from the Paris 2024 Olympic Games was very interesting for me. The dataset wasn't just some simple classroom example; it had all sorts of quirks — overlapping IDs, messy multi-value fields, and domain-specific details like NOC(country entity) membership that I had to dig into before I could model them right. Dealing with that complexity was the most valuable part.

If I had to do this project all over again, I would focus more on data analysis, it would save me lot time.

5 Datatype selection (1-2 pages)

The choice of data type for each column was made based on three aspects, the data as perceived in the raw CSV files, the relation schema derived in Stage 1 and the constraint of Oracle database. Each decision was made collectively after examining the maximum values for length and the range of the values in raw dataset.

The data type VARCHAR2 has been selected for all character type attributes. It unlike CHAR type does not have padding of spaces with it which helps to keep a minimal storage space and will not cause any comparison problem in cases where the trailing spaces will be an issue. The length of these fields were adjusted according to the maximum length found in the CSV file, giving 1.5 times as buffer. Countrycode (VARCHAR2(3)) since all the nocs in nocs.csv are 3 chars in length (FRA, USA, AIN). Sportcode (VARCHAR2(3)) due to same reasons. Organisationcode (VARCHAR2(10)) to store the largest federation code. Eventtag (VARCHAR2(80)) and teamcode (VARCHAR2(30)) because these can be slightly longer due to generation with an import process while simple codes will have less length. Venuetag (VARCHAR2(60)) to accommodate the largest observed URL slug which is 38 characters long (saint-quentin-en-yvelines-bmx-stadium). All the name and description related columns were given a length of either VARCHAR2(100) or VARCHAR2(200) as these contain information which could include international names, accent marks, characters of other alphabets.

The NUMBER data type has been used for all numeric identifiers and measurements. This data type, with its properties, can be used for financial data (sponsorshipbudget) without any floating point precision errors. The attribute Person.code has been used as NUMBER(10) because the largest code value is 9,460,001, and it will have enough capacity to store future values. Medal.medalcode as NUMBER(1) since the data contains only 3 values (1, 2, 3 representing gold, silver and bronze respectively). Attribute Athlete.height as NUMBER(5,2) for storing value up to 999.99 cm and for the same reason Athlete.weight and Sponsor.sponsorship_budget were assigned NUMBER(5,2) and NUMBER(15,2) respectively.

The DATE data type has been used where only year, month and day was required as an attribute and there is no necessity of time and time zone. Oracle's DATE data type in its internal storage always stores date and time of midnight for such cases. The attributes Person.birthdate, Venue.datestart, Venue.dateend, Schedule.day, AthleteWinsMedal.medaldate are date fields. These are recorded in YYYY-MM-DD format in source CSVs but while importing to oracle it gets changed into a TIMESTAMP data format in the specified NLS session format (DD-MON-RR HH.MI.SS.FF9 AM). Because of that all these fields were loaded using CAST(TOTIMESTAMP(value, 'DD-MON-RR HH.MI.SS.FF9 AM') AS DATE) and not just TODATE(). The format mask specification for fraction seconds is 'FF9' that suggests that the precision is of 9 decimal places of a second.

The TIMESTAMP WITH TIMEZONE data type was used only for schedule.startdate and schedule.enddate because the values for these attributes are given in ISO-8601 with offset information and this data type can capture the same efficiently without the loss of information. A simple DATE field would have been inappropriate to calculate how many events ended on or before a specific date in a particular time zone as stated in validation query 24, as the value may be, for instance, 2024-07-24T15:00:00+02:00, and this timezone information would be lost if used in the DATE data type. For that purpose it was used as TIMESTAMP WITH TIMEZONE and loaded using 'YYYY-MM-DD"T"HH24:MI:SSTZH:TZM' format mask.

6 Physical model codification (1-2 pages)

This physical model was put into the script file G_physicalmodel.sql. One of the most important design criteria required of this script by the project specification was that the script be "idempotent"; i.e., that it could be run on the same database multiple times with no side effects or errors, without requiring any manual cleanup in between runs. This was achieved as follows.

The script begins by issuing DROP TABLE... CASCADE CONSTRAINTS PURGE statements on all tables and sequences in a specific reverse order:child tables first, then their parent table. CASCADE CONSTRAINTS ensures that the dropped tables have no other foreign key constraints pointing at them from tables outside the set of tables currently being dropped; the PURGE clause drops them immediately, removing them from Oracle's Recycle Bin as it would otherwise keep a backup copy in a system-generated name like BIN$.. And charge for the storage. On a run against a blank, new database, all of the DROP statements result in an ORA-00942 error since the table to be dropped does not exist yet; Oracle handles ORA-00942 as a non-fatal error and execution continues. On a subsequent run, Oracle is smarter and all of the DROP statements will execute without errors; this behavior is expected from an idempotent script.

After the DROPs, all tables were recreated in their dependency order. Initially parent tables without external FKs were recreated (Country, Organisation, Sport, Medal and Venue). Following these were the Person generalization hierarchy: The Person table was created before Athlete, Coach and TechOfficial, as both of these are child tables and both refer to the primary key Person.code. All other base entity tables were then recreated: Team, Sponsor and Event, followed by Schedule (which has an FK to both Event and Venue). Lastly, all 7 junction tables were recreated: AthleteCompetesInEvent, AthleteWinsMedal, AthleteMemberOfTeam, CoachTrainsTeam, CoachWorksInSport, TechOfficialOfficiatesInSport, SponsorSponsorsCountry. This process ensured that whenever a table was recreated referencing an existing table via FK, the referenced table had already been created.

All primary key and foreign key constraints were given explicit names; specifically pktablename for a primary key on tablename and fkchildtableparenttable for a foreign key from childtable to parenttable. So the primary key for Athlete is named pkathlete, and the foreign key from Athlete to Person is named fkathleteperson. While it makes the script longer, named constraints are enormously easier to work with in a production environment as they allow individual queries anddisabling/enabling without interference with other constraints, appear by name in Oracle errors, and are identifiable in user views (e.g. Userconstraints). Anonymous constraints were given a random system-generated name, such as SYSC009123, making them extremely difficult to identify without querying the data dictionary.

The Schedule table did not have a natural composite key that was guaranteed to be unique. The schedules.csv file has no naturally unique key identifying a specific row in that file. Furthermore, it is expected that a particular event could appear more than once in the schedules (i.e.,qualification, semifinal, final phases all on the same date). So a synthetic key, scheduleid, was used. Oracle does not support MySQL's AUTOINCREMENT or PostgreSQL's SERIAL commands for tables; it supports sequences. So an Oracle sequence named schedule_seq was created with start with 1, increment by 1, and nocache. The sequence was explicitly given nocache as opposed to the default

cache value of 20; during testing the gaps between consecutive ID values during the loading were found undesirable, and when running the script in a fully blank state in a production setting we wanted to ensure the sequence always starts back at 1 for ease of use when debugging.

Every table has comments written inline at the point of creation and at every column declaration. This text describes what the column represents, the original source file for the column's data, and any necessary handling of it such as type conversion and how it is mapped. The script is thus self-documenting without need of accompanying documentation to understand the data schema.

7 Data import (2-3 pages per file)

7.1 athletes.csv

The athletes.csv is the largest and most complex source file of the whole data-set, with 11,113 rows and 36 columns, and a file size of 6.89 MB. It is used as a source for 2 physical model tables,PersonandAthlete. The file contains biographical and physical details about every athlete participating at the Paris 2024 Olympic Games; from basic information, such as name, date of birth, to highly personal details, such as heros, philosophers, rituals and hobbies. For a number of those personal columns, there is a large amount of free text with entries in multiple languages.

One initial problem with importing the athletes.csv file was a connection timeout at Oracle Cloud; the file is too large, and the Data Load wizard at Oracle times out the connection during loading the whole athletes.csv to TEMPATHLETES. The work-around was splitting the athletes.csv file into three, separate parts of approx 3,800 rows each, using a Python script, retaining the column headers in all three files. Then, loading each part to TEMPATHLETES separately, and selecting the "Append" option, instead of "Replace", to load to an existing table rather than emptying it. A query confirmed that TEMP_ATHLETES now holds the full 11,113 records.

All 36 columns of TEMPATHLETES are declared as VARCHAR2. However, for many columns, a larger than seemingly obvious length is required.Hobbies,reason,philosophy,familyandritualcolumnsmay contain several paragraphs of free text, so those have been declared as VARCHAR2(4000), the maximal value for the VARCHAR2 type in Oracle, not using CLOB type. Thecolumnsaare namednameandnametvand contain sometimes very long names from non-Latin scripts, hence declared as VARCHAR2(400).The birthplaceandresidenceplacecolumns contains, at times, a city and a region, hence declared as VARCHAR2(500). The columnscurrent,as present in the source file, had to be renamed tocurrentflag,because CURRENT is a reserved keyword in Oracle, asfunctionis, and therefore it had to be renamed functioncol.

The INSERT-SELECT-sentences used to populate the physical model tables are separated into 2 sentences. First, we are filling the Person table with the four shared attributes (code,name, gender, birthdate), using UNION with the corresponding sentence of the coach- and technical official-files, to combine the three source-files in one statement. Union is used instead of union all, as we want to eliminate potential double codes if some people appear in more than one file; we make use of the countrycode,code to identify the same person. TONUMBER(TRIM(code)) is used and guarded by REGEXPLIKE(TRIM(code), ‘^d+$’) which will reject rows where the code is not purely numeric (avoiding ORA-01858). We need CAST(TOTIMESTAMP(TRIM(birthdate), ‘DD-MON-RR HH.MI.SS.FF9 AM’) AS DATE) to format Oracle Data Load's NLS session format (which is a conversion of YYYY-MM-DD, the format given by ISO 8601), adding the FF9 to manage Oracle's automatic append of nanoseconds.

Second, we are filling the Athlete table with the five Athlete-specific attributes (nameshort,height, weight, birthplace, and countrycode). Theheightandweightcolumns are cast using TONUMBER(NULLIF(TRIM(value),’’))to transform empty cells into NULL and avoid any ORA-01843 errors. The country_code column acts as a foreign key from the National Olympic Committee table; we ensure that it only contains codes existing in that table, and guard the column by EXISTS and the code column by checking if it exist in the Person table to remove athletes who do not pass any integrity constraint. All 11,113 athletes were successfully added.

Most of the columns in athletes.csv were not taken into account, due to their non-relational and too personal nature (hobbies, hero, philosophy, influence, sporting relatives, rituals, other sports, occupation, education, family, lang, coach, reason, nicknames, nationality, residenceplace, birthcountry,nametv). Those remain in TEMPATHLETES and could, if ever required, be incorporated at a later stage.

7.2 schedules.csv

The schedules.csv file has 3885 rows and 16 columns, which represents every single time slot from the 2024 Paris Olympic schedule. It was the source table to the Schedule physical model table which is one of the hardest tables to populate from due to three specific challenges. They are the format of the timestamps, the surrogate key requirement and the mismatch in venue identifier codes.

All 16 columns in the TEMPSCHEDULES table were created as VARCHAR2. The startdate and enddate columns held complete ISO-8601 timestamps with a time zone offset in the format 2024-07-24T15:00:00+02:00. The format included a date, time and a UTC offset string, e.g., the date followed by a T and then time followed by the offset, e.g. The day column was just a date in YYYY-MM-DD format. The venuecode column held three character short codes such as AQC, BCY, CDM etc representing the venue of the particular scheduled slot. The discipline_code column held the code representing the sport, e.g. ARC represents Archery, SWM represents Swimming and the event column held the name of the event as text, e.g. Archery individual. The phase column contained the competition phase name, e.g. Quarterfinal, Semifinal, Final, Preliminary Round, with a maximum value length of 60 characters, it was declared as VARCHAR2(200) to hold future longer values for phase descriptions.

The first problem when populating the Schedule physical model table was the timestamp format. The format 2024-07-24T15:00:00+02:00 is not an ISO-8601 UTC timestamp, as it was initially expected (e.g., 2024-07-27T09:00:00Z). The T is a literal character and the +02:00 represents the hours and minutes offset rather than Z. In Oracle this means that a format mask of 'YYYY-MM-DD"T"HH24:MI:SSTZH:TZM' must be used where TZH is the hour offset and TZM is the minute offset. The double quotation marks around T mean that T will be treated as a literal character. Without the double quotation marks, a ORA-01861 error will occur.

The second problem was that the Schedule physical model table required a surrogate key called scheduleid because the source CSV did not have a natural unique identifier. A composite primary key could theoretically have been created from startdate and eventtag, but this would fail because the same event occurs numerous times on the same date within different phases of the competition. The surrogate key has been generated by a SEQUENCE object called scheduleseq which is selected into from schedule_seq.NEXTVAL within the INSERT-SELECT statement to produce the required values 1, 2, 3...automatically per row inserted. At the start of each run, the SEQUENCE object is dropped and re-created so that it always begins with a value of 1 and no gap in sequences occurs.

The third and biggest challenge was that there was a data quality problem between schedules.csv and venues.csv. The venuecode column of schedules.csv holds short three character codes such as AQC for Aquatics Centre, BCY for Bercy Arena and CDM for Champ de Mars Arena, whereas the venuetag column of venues.csv holds long URL-style slugs such as aquatics-centre, bercy-arena, champ-de-mars-arena. They can not be joined without some transformation. An Oracle DECODE()

function has been created that contains 40 entries mapping each of the unique venuecodes in the schedules.csv to the corresponding venuestag. The values of the mappings have been confirmed against the text in venues.csv of the venue name. Multiple venue_codes can exist that represent the same venue as one physical venue can host an event in different locations, e.g. LC1, LC2, LC3, LC4 all map to la-concorde, and SP1, SP4, SP6 all map to south-paris-arena.

The foreign key eventtag in Schedule has been rebuilt by combining disciplinecode and event values using the same REGEXPREPLACE expression that was used to create eventtag for insertion into the Event table Rows from schedules.csv where the rebuilt eventtag does not appear in the event_tag column in the Event table have been excluded using an EXISTS subquery. The exclusions in this instance include the opening ceremony events and other time slots in the schedule that do not belong to an event entity. All entries in schedules.csv that relate to competitive events were successfully inserted into the Schedule table.

7.3 Medallists.csv

The medallists.csv file is the main source for two physical model tables: AthleteWinsMedal and AthleteCompetesInEvent. This file consists of 2,315 rows and 21 columns, and contains all awarded medals to date in Paris 2024 Olympics. The file is organized such that an award of a team gold medal will generate many lines-one for each athlete on the team. This makes the file suitable as a source for records about individual athletes winning medals.

The TEMPMEDALLISTS table was created with all 21 columns set as VARCHAR2. The columns primarily used during import are: medaldate, medaltype, medalcode, discipline, event, codeathlete and codeteam. The 'discipline' column actually stores the name of the sport (e.g. Swimming, Artistic Gymnastics, Athletics) instead of the code used in the physical model (sportcode). The 'event' columnstores the full name of the sport (e.g. Women's 100m Breaststroke, Men's Individual Time Trial). The 'codeathlete' column stores the number of the athlete which maps to the code column as the primary key in the Person and Athlete tables. The 'code_team' columnstores the code of the team involved (this field is null for single athletes' events).

The first technical difficulty encountered during import is with the column 'medalcode'. The CSV stores this as a decimal number: 1.0 for Gold, 2.0 for Silver, 3.0 for Bronze. Oracle Data Load converts these values to NUMBER in the database, which is compatible with the NUMBER(1) field in the physical model. However, direct TONUMBER() conversion would result in numbers such as 1.0, 2.0, 3.0, which are invalid for the foreign key constraint to the Medal table, which stores this information as an integer value (1, 2 or 3). The ROUND(TONUMBER(TRIM(medalcode))) conversion handles this nicely, turning 1.0 into 1, 2.0 into 2, and 3.0 into 3 consistently.

The second technical challenge arises with the 'medaldate' column. Similar to 'birthdate', this field is stored in the ISO-8601 format in the original file (YYYY-MM-DD, e.g., 2024-07-29). Oracle Data Load will automatically parse this into the default NLS session format (e.g., DD-MON-RR HH.MI.SS.FF9 AM). So, we use CAST(TOTIMESTAMP(TRIM(medaldate), 'DD-MON-RR HH.MI.SS.FF9 AM') AS DATE) for this field.

The third technical hurdle is the reconstruction of the eventtag foreign key. The medallists.csv file doesn't have a sportcode field but only has a 'discipline' field where the name of the sport is stored as plain text. Thus, a join operation to the Sport table will be necessary in order to reconstruct eventtag in the same format it's found in the physical model's Event table (which is sportcode || ' - ' || REGEXPREPLACE-cleaned sport name). Thisjoin is accomplished using JOIN Sport s ON UPPER(TRIM(s.sportname)) = UPPER(TRIM(tm.discipline)). This matches the sport code to the correct sport name by first using UPPER() on both fields to ensure a case-insensitive match, then TRIM-ing any whitespace and then matching to the 'discipline' field of the temporary medalists table. Finally, the identical formula used in other parts of the script to build event_tag will be used.

The INSERT-SELECT statements used to populate AthleteWinsMedal and AthleteCompetesInEvent both employ DISTINCT in order to de-duplicate entries in the source file, as multiple entries may exist for a single medal if the source file lists relay athletes more than once. The EXISTS clauses before the INSERT statement are used to ensure that the athlete code, sport code, event tag, sport code, sport name, team code, team name and medalcode already exist in the target table before insertion. Only medals that evaluate to 1, 2, or 3 after conversion are inserted. The logic for AthleteCompetesInEvent simply maps medaldate to start_date because it's the only date that can be used to indicate when an athlete started competing on a team event.

7.4 teams.csv

The file 'teams.csv' has 1,698 rows and 16 columns representing every national team participating in the Paris 2024 Olympic Games in every team-based sport. This is the source table for three physical model tables; 'Team', 'AthleteMemberOfTeam', 'CoachTrainsTeam'. Besides the basic identification of the teams it also contains the entire list of assigned Athletes and Coaches per team. The list of athletes and coaches has been stored in a format that requires a non-standard import.

A TEMPTEAMS table was created where all the columns in the original table were defined as VARCHAR2. The most important columns are code (unique identifier for the team e.g. BKBMTEAM5---AUS01), team (human-readable team name e.g. Australia Women's Basketball Team), teamgender (category of gender for the team where M = Men's, W = Women's, X = Mixed and O = Open), countrycode (3-letter NOC for the country which is a foreign key to Country table), disciplinescode (sport code e.g. BKB for Basketball), athletescodes and coachescodes. Since athletescodes and coachescodes might contain many codes per list of athletes/coaches (for sports such as football, handball) the columns were declared as VARCHAR2(4000) (max standard VARCHR2 length for Oracle). The 'current' column from the original table has been renamed to 'current_flag' in the TEMP table as CURRENT is a reserved keyword in Oracle.

The INSERT-SELECT from TEMPTEAMS to the physical model table Team is straightforward. The code from the source table is mapped to teamcode (primary key), team maps to teamname, teamgender directly maps, countrycode to countrycode (foreign key for the Country table). An EXISTS subquery is used to ensure that only rows for which the country_code exists in the Country table are inserted.

The difficulty of this file is related to athletescodes and coachescodes. They contain the complete list of athlete/coach codes per team as a Python list-like string such as ['1913366', '1913367', '1913369']. This is how Python represents lists when exporting them to CSV, and thus contains multiple values for a single field in a cell. Standard INSERT-SELECT statements do not handle multi-value fields and pure SQL can't parse such multi-value string columns. So, PL/SQL anonymous blocks were written to import these two tables.

PL/SQL block for AthleteMemberOfTeam uses a cursor FOR loop over all the rows in TEMPTEAMS. For each team row, REGEXPCOUNT() is used to find how many numbers exist in the athletescodes list, and then a numeric FOR loop from 1 to this count is used to parse individual codes from the list using REGEXPSUBSTR() with pattern '\d+' and an occurrence count. For each parsed code, a query using COUNT(*) is used to check if the teamcode exists in the Team table and if the athlete code exists in the Athlete table, and if both conditions are true (count > 0), an insert statement to the AthleteMemberOfTeam table is executed. DUPVALONINDEX exception is

handled to skip double insertions due to repeated athlete codes in the same cell, which sometimes occur when data are duplicated. A similar block is used to import CoachTrainsTeam.

Also, many rows have NULL values for coachescodes column, meaning they are for team's without any assigned coach in the data. Oracle Data Load imports Python's None to database's NULL, so the block works by the WHERE coachescodes IS NOT NULL and the REGEXPLIKE(TRIM(coachescodes), '\d') check is added to avoid parsing if the column value is an empty string, and the PL/SQL block iterates correctly. After importing, there were 6990 rows for AthleteMemberOfTeam and 513 rows for CoachTrainsTeam. Both of these figures were verified against source data.

7.5 technical_officials.csv

Technical_officials.csv (1021 rows, 12 columns) This table stores the complete details of all technical officials who officiated in the Paris 2024 Olympics (Referee, Judge, Line Judge, Challenge Referee and Medical Delegate). This file becomes the source for 3 physical model tables: Person (via UNION with athletes and coaches), TechnicalOfficial, and Organisation. A decision regarding structural analysis on this file led to one of the most crucial design choices in the whole relational model, the separate creation of an Organisation table apart from a Country table.

TEMPTECHOFFICIALS table was created by placing all 12 columns in VARCHAR2 format. The columns include code (unique number ID of an official), name, gender, birthdate, function (the job role of the official), organisationcode (unique number ID of the appointing organisation), organisation (short name of the appointing organisation), organisationlong (official name of the appointing organisation) and disciplines (the sport/sports an official is skilled in). In the TEMP table the 'function' column was renamed functioncol, because 'FUNCTION' is a reserved keyword in Oracle. Similarly 'current' was renamed to current_flag as 'CURRENT' is a reserved keyword. The disciplines column was made VARCHAR2(500) because it contains a Python-style list like ['Hockey'] or ['Hockey', 'Field Hockey'] if the official is proficient in more than one sport.

The very first INSERT-SELECT statement from this table inserts into the Organisation table. After applying a SELECT DISTINCT to get all the organisation codes,organisation names and organisation long names, there are 107 distinct organizations recorded. Examining these 107 codes showed that it contains two distinct types of entity: first three-letter codes that directly map to existing nocs.csv entries (representing national olympic committees), and second, 14 codes representing individual international sports federations that are entirely unique to this file (e.g., FIBA, FIG, UCI, WAQU, WT, IJF). TechnicalOfficial couldn't directly reference the Country table because if that were the case, the 14 types of officials would be incorrectly categorized without a way to map them to an entity other than Country. By separating them out, an Organisation entity can hold both NOC codes and other types of governing bodies, streamlining the foreign key relationship for TechnicalOfficial.

The Organisation table is populated first, as TechnicalOfficial will hold a foreign key constraint pointing to Organisation.organisationcode. All the attributes in the Person physical model table are inserted via a UNION query with athletes and coaches, where the REGEXPLIKE and CAST casting of column data is performed exactly as demonstrated in section 7.1. The TechnicalOfficial-specific attributes (the primary and foreign key column code, the renamed functioncol that will map to the physical model function attribute, and the organisationcode foreign key column) are inserted in a second INSERT-SELECT statement and include EXISTS subqueries to ensure that both the person and organisation foreign keys exist.

One noticeable data quality issue lies within the birthdate column. Of all 1,021 technical officials, only 19 contain a non-null value in the birthdate column. This data quality issue caused query Q30 and Q31, which are asking for the oldest technical official, to return only the 19 officials with a birth_date value that exist in the database, and does not seem to be caused by an error in the data load, but instead by the inherent nature of the source data.

A challenge regarding parsing was presented by the disciplines column. As is seen in section 7.1, disciplines holds values in Python list strings such as ['Hockey'] or ['Hockey', 'Field Hockey']. Unlike the other coded value foreign keys already discussed, this column contains text sport names that will need to be cross-referenced against sportname in the Sport table. REGEXPREPLACE was used with the pattern "[[]"]", removing the square brackets and quotation marks to leave comma separated sport names that are easily parsable. INSTR then searches within this comma separated string for any occurrences of each sport_name. This enables TechOfficialOfficiatesInSport to be fully populated in one SQL statement, regardless of the number of disciplines an official can officiate. There were 1,238 rows inserted into the TechOfficialOfficiatesInSport table.

7.6 events.csv

File: events.csv This file has 329 records and 5 attributes (event, tag, sport, sportcode, sporturl). Although it is one of the smallest files, it is probably the most important in the set. This file populates the two physical model tables Sport and Event and also serves as the source of the synthetic primary key eventtag that is used as a foreign key in 4 other tables (Schedule, AthleteWinsMedal, AthleteCompetesInEvent and Schedule's eventtag column). If this formula generating the event_tag is in anyway inaccurate, then all foreign key constraints in the entire database will be violated.

This table was created with 5 VARCHAR2 column types: event (Name of event - Men's Individual), tag (The URL of the sport, for instance 'archery' in the case of the five archery events - this is the sport tag and not the event tag), sport (Name of the sport - Archery), sportcode (e.g. ARC), sporturl (URL of the sport on the Olympics web-site, longest field observed 63 chars long, declared as VARCHAR2(300) to avoid any potential truncation.

The first INSERT-SELECT populates the table SPORT with distinct values from the CSV. There are 5 events associated with Archery sport in the CSV so the SELECT DISTINCT on sportcode, sport, and sporturl had to be performed to derive the 45 unique sport codes without duplication. The 'sport' attribute from the CSV maps to sportname in the physical model while the 'sporturl' attribute maps to sport_url.

The second INSERT-SELECT populates the table EVENT and, for this process, this step is the most complex one; generating the eventtag. The CSV does not contain any field which could serve as a primary key. Since a single sport would generally have more than one event associated with it (the tag is the same for all 5 archery events for example, sport tag = archery), it is not possible to use this attribute for primary key. So a synthetic key eventtag was generated by combining the sportcode, a hyphen, and the event tag stripped of non-alphanumeric characters (by using the expression REGEXPREPLACE). The expression "[^a-zA-Z0-9]" finds all characters that are not letters or digits, and replaces them with a hyphen. For example Men's Individual becomes Men-s-Individual, which means a combination of sport code, a hyphen, and the cleaned up event tag like ARC-Men-s-Individual.

A major data quality issue arose during the initial attempt. The two JUDO events, Women +78 kg and Women -78 kg, after undergoing the REGEXPREPLACE procedure, resulted in the same tag JUD-Women--78-kg, because both + and – signs were replaced by hyphens. This prevented the PRIMARY KEY constraint on the table Event from being fulfilled, resulting in ORA-00001. The solution was to apply two REPLACE() statements before the REGEXPREPLACE() call. First all plus signs were replaced by the word "plus" and all minus signs by the word "minus". This resulted in the tagsJUD-Women-plus78-kg and JUD-Women-minus78-kg respectively.

The same transformation expression (REPLACE(REPLACE(event, '+', 'plus'), '-', 'minus') before the REGEXPREPLACE) was applied to all subsequent INSERT-SELECT statements involving eventtag namely those for Schedule, AthleteWinsMedal and AthleteCompetesInEvent. A different formula at this step in any other insert-select would lead to foreign key violations as the event_tag generated here would not match that in the EVENT table.

There was no attribute for gender of the event in the CSV file events.csv. So this was generated using the case when statements as eventgender is determined by if the word 'WOMEN' is present in the event attribute (case insensitive) returns 'Women', if the word MEN but not WOMEN is present then returns 'Men', if MIXED present then returns 'Mixed', else returns 'Open'. These cases were applied for all 329 events. Post this operation a count of 0 shows no NULL gender entries for the event table. The Event table post this process consists of 329 records for 45 distinct sports and there were no duplicate eventtag values.

7.7 nocs.csv

The file nocs.csv, containing 224 rows and 5 columns: code, country, countrylong, tag and note. It is used to create the table Country in the physical model. Country is the most used table in the entire database, with almost all other entities possessing a foreign key pointing to it. For this reason, Country is the first one to be populated, prior to any other INSERT-SELECT. The TEMPNOCS table has been created with five VARCHAR2 columns. The code column contains three-letter NOCs, the code having been declared VARCHAR2(10). The country column contains short country names such as France or AIN, having been declared VARCHAR2(100). The countrylong column contains full official names such as French Republic and has been declared VARCHAR2(200) to accommodate the largest of entries observed. The tag column contains URL-safe country names such as france or great-britain, declared VARCHAR2(100). Finally, the note column, containing Olympic committee status, is VARCHAR2(10). The INSERT-SELECT from TEMPNOCS into Country is the simplest one in the entire project. All values are in the correct format and type, so they just need to be TRIMmed before being inserted in the target table, as any other VARCHAR2 column needs to be TRIMmed by design and because the source table code needs to become the country_code column of the target table.

A key design consideration that arises is the true nature of the entity Country. It is, despite the name, not intended to represent geographic countries but Olympic National Olympic Committees (NOCs). There are entries such as AIN which are not sovereign nations: these are special cases such as Athletes of Individual Neutrality at the Paris 2024 Olympics. Other entries are historical (AHO for the Netherlands Antilles), but are still part of the Olympic record. The note column describes this: P (Participating), NP (Non-participating) or H (Historical). This was verified by finding the three distinct values in the note column. The reason for naming the primary key country_code rather than code is to avoid potential ambiguity with a more general geographic country identifier.

All 224 NOC entries are necessary, as subsequent INSERT-SELECT operations on Athlete, Coach, Team and Sponsor rely on EXISTS subqueries against Country. Any row from any of those tables that uses a country_code that doesn't exist in Country is implicitly excluded by the EXISTS subquery. Post-import checks show that all 224 rows were correctly inserted.

7.8 coaches.csv

The coaches.csv file is one of 974 lines and 12 fields, containing the entire database record for every coach competing in the Paris 2024 Olympics. From this table, the two physical tables person and coach are populated. This table's structure mirrors technicalofficials.csv. It contains the same columns with the exception of coachid, instead containing code. The columns available are: code, name, gender, birthdate, function, category, countrycode, country, countrylong, disciplines and events. As stated for the technical officials table, in this one all the countrycode values are valid country codes as given in nocs.csv.

TEMPCOACHES: As before 12 varchar2s were chosen to load the data into TEMPCOACHES. As in the athletes and technical officials files, the column 'function' in source was renamed 'functioncol' because 'FUNCTION' is a reserved word in OracleSQL. If this column were imported with the name 'function' the SQL interpreter would encounter a syntax error when the CREATE TABLE command was parsed; Oracle would interpret the 'function' as the SQL function instead of the actual name of the column. Similarly, the current column was renamed to 'currentflag'. The disciplines column has a maximum length of 500 varchar2 to accommodate varying sport names such as 'Artistic Gymnastics'.

INSERT-SELECT 1: This file contributes its records to the person table through a UNION combined with records for athletes and technical officials. The records added to the person table are code, name, gender, birthdate. These were imported using the same method as the dates and the following formula to convert the string format into date format: CAST(TOTIMESTAMP(TRIM(birthdate), 'DD-MON-RR HH.MI.SS.FF9 AM') AS DATE). In order to deal with issues relating to 'code' entries which can contain a mixture of letters and numbers, REGEXPLIKE(TRIM(code), '^\d+$') is used to filter these out, before the TO_NUMBER() function is used on these digits to convert to integer type, thus ensuring there will be no errors with the data types. The UNION is used in place of UNION ALL in order to ensure that duplicate records in the person table are not inserted if code entries already exist from previous files (athletes or technical officials).

INSERT-SELECT 2: All 974 coaching records were transferred into the physical coach table. This table has the columns code (PK, FK), function, and countrycode (FK). Before these records were loaded into the coach table, two EXISTS queries were implemented to verify that the coach record and the relevant countrycode are valid (i.e. Exists in the person and country tables respectively). As no errors arose for any record when they were checked against these query, it means all records had a valid country code assigned to them.

The disciplines column in the coach file differs from those in technical officials and teams. Rather than storing a Python list, it stores a plain string such as 'Hockey' or 'Artistic Gymnastics'. Thus for loading into CoachWorksInSport, instead of splitting a string like this is has simply joined. JOIN Sport s ON UPPER(TRIM(s.sport_name)) = UPPER(TRIM(tc.disciplines)) used in order to ensure the strings were case insensitive and stripped of white space. This correctly matches 974 entries from the coaching data with thesport table entries. 974 lines were inserted, which is exactly the number of coaching records, since each coach in the dataset has one and only one associated discipline.

The category column in the coach file was not transferred to the physical model, as it was not necessary for the queries that are intended for the database, according to stage one relational design of the physical model. Similarly, country, countrylong and event entries were omitted due to the same reason, where all countries are accounted for via the countrycode column.

7.9 venues.csv

The venues.csv file has 35 rows and 6 columns. The columns are venue, sports, datestart, dateend, tag, url. This is the source for the Venue physical model table which holds the 35 competition venues for the Paris 2024 Olympics. This is one of the smallest files in the dataset but is structrually significant because every record in the schedule table holds a foreign key to Venue (the venue_tag) so the Venue physical model is referenced in every entry in Schedule.

The TEMPVENUES table was created as a copy of venues.csv with 6 VARCHAR2 columns. The venue column contains the human readable venue name, eg. Aquatics Centre or Stade de France and was declared as VARCHAR2(200). The sports column contains a Python-style list of sports held at that venue eg. ['Artistic Swimming', 'Water Polo', 'Swimming'] and was declared as VARCHAR2(500). The datestart and date_end columns contain ISO-8601 timestamps and was declared as VARCHAR2(50). The tag column contains the URL slug identifier and was declared as VARCHAR2(100). The url column contains the full URL to the venue and was declared as VARCHAR2(300).

The INSERT-SELECT statement from TEMPVENUES to the Venue physical model table mapped 4 of the 6 source columns. The tag column mapped to venuetag (primary key) and venue column to venuename. For datestart and dateend a custom conversion was needed as the full ISO-8601 timestamp includes timezone and time but the Venue table only needs to store the calendar date. The query TODATE(SUBSTR(TRIM(datestart), 1, 10), 'YYYY-MM-DD') extracts the first 10 characters of the timestamp string (the YYYY-MM-DD part) and converts that to an Oracle date format. This bypasses the Oracle Data Load timestamp conversion problem by not even passing the date and time to an Oracle date function until the string part is extracted. Sports and url columns were inserted to TEMPVENUES but were not loaded to the physical model as the sport-venue relationship is not stored directly on Venue but implied by the Schedule table.

The Venue table is important in the venuecode to venuetag mapping problem described in 7.3 (schedules.csv) below because the 35 venuetag values stored in Venue, taken directly from the tag column of venues.csv, are the target values which all 40 DECODE() calls on Schedule must provide. The venuecode to venuetag mapping in schedules.csv was built by visually matching venue names from venue column of venues.csv with venue column in schedules.csv and making a judgement if they were the same. There were several cases to consider where one or more venue codes map to a single venuetag. Examples are LC1, LC2, LC3 and LC4 in schedules.csv which all map to la-concorde in venues.csv, SP1, SP4 and SP6 all map to south-paris-arena and VN1 and VN2 map to vaires-sur-marne-nautical-stadium (at two different water courses).

7.10 medals.csv

The medals.csv contains considerably more rows than we would anticipate for a medal reference table-it holds the entire history of all medal awards (one row per athlete per medal). It is organized similarly to medallists.csv. However for the physical model only, this file was used simply to populate the Medal reference table, the only three rows it possesses; one for Gold Medal (medalcode 1), one for Silver Medal (medalcode 2), and one for Bronze Medal (medal_code 3).

The TEMPMEDALS was created with 13 VARCHAR2 columns matching the source file's column structure. They were: medaltype, medalcode, medaldate, name, gender, discipline, event, eventtype, urlevent, code, countrycode, country, countrylong. They were loaded in their source order and as they appear in the CSV data; none was changed as specified in the project requirements.

The INSERT-SELECT from TEMPMEDALS into the Medal physical model table is not a typical SELECT statement. The Medal table in the physical model contains a medaldescription column that has no counterpart in any of the source files. Since there is no source data, it was not possible to fill this column through a simple SELECT from TEMPMEDALS; therefore, the three Medal records are entered via individual INSERT statements with hard coded values: medalcode 1 and medaltype 'Gold Medal' along with descriptive text, and similar entries for medalcode 2 and 3. The medal_description value was written by the author, intended as human-readable description.

This manual data entry is explained in the import script comment where it was implemented, noting that medaldescription is missing from the source. All of the other columns in TEMPMEDALS-including individual medal award records along withathlete and country codes, event name, date, etc.-were unused from this source file, and were not included since the complete and structured medallists.csv file was the primary source for AthleteWinsMedal and AthleteCompetesInEvent. The medals.csv data was used merely as verification that the three medal code types and descriptions were correct.

7.11 sponsors.csv

The sponsors.csv file is the custom generated file, generated by Mockaroo (as required for an extension), using an independently generated CSV file linked to an existing file. It is comprised of 150 rows and 5 columns: sponsorid, companyname, industry, sponsorshipbudget, and countrycode. This represents corporate financial sponsorship in the context of the Olympic movement and acts as a commercial counterpart to the other (athletic and logistical) source files.

The TEMPSPONSORS table was defined as having 5 columns, all of VARCHAR2. The sponsorid, generated as sequential numbers, is of VARCHAR2(20). The companyname, simulated real world data, is of VARCHAR2(200). The industry data, i.e. Finance, Healthcare or Food and Beverage is of VARCHAR2(100). The sponsorshipbudget figures are decimal numbers which have been read as VARCHAR2(30). Country codes were read in as VARCHAR2(10).

In Mockaroo, when creating sponsors.csv, the countrycode field was set as Custom List, populated with all available, unique countrycodes from the nocs.csv file, so that no referential integrity issues could occur at import, as each Sponsor record was guaranteed to reference an existing countrycode in the Country table. This was necessary as the foreign key relationship is from Sponsor to Country, on countrycode, and without this constraint the row in Sponsor would silently be ignored due to the EXISTS clause on insert.

The INSERT-SELECT from TEMPSPONSORS to the Sponsor physical table involves a few conversions; sponsorid into a numerical primary key, companyname and industry into plain text, and sponsorshipbudget into a numerical value, from their respective VARCHAR2 definitions, via a TRIMM and TO_NUMBER function. An EXISTS subquery ensures that each row successfully maps to the Country table, prior to insert; 150 rows were successfully inserted.

The second INSERT-SELECT from sponsors.csv populates the SponsorSponsorsCountry junction table, which is required to manage the N:M relationship between sponsors and countries sponsored. Since the sponsors.csv file specifies only a single country code, there is a 1:1 relationship between the Sponsor record and the SponsorSponsorsCountry record, which links the sponsoring company to the country it is headquartered in. The N:M junction table is necessary because in reality, a single sponsor might sponsor several countries; it maps exactly the 150 sponsor records and their associated 150 country codes (and only those 150), into the SponsorSponsorsCountry table. All 150 rows were successfully inserted into SponsorSponsorsCountry.

8 Query implementation

8.1 Question: Which athletes have won more than two gold medals in a single sport?

```
SELECT p.name AS athlete_name, s.sport_name AS sport,
    COUNT(awm.medal_code) AS gold_medals
      FROM AthleteWinsMedal awm
      JOIN Medal  m ON awm.medal_code = m.medal_code AND m.medal_type = 'Gold Medal'
      JOIN Event  e ON awm.event_tag  = e.event_tag
      JOIN Sport  s ON e.sport_code   = s.sport_code
      JOIN Person p ON awm.athlete_code = p.code
GROUP BY p.name, s.sport_name
HAVING COUNT(awm.medal_code) > 2
ORDER BY gold_medals DESC, athlete_name;
```

ATHLETE_NAME	SPORT	GOLD_MEDALS
MARCHAND Leon	Swimming	4
BILES Simone	Artistic Gymnastics	3
CARRINGTON Lisa	Canoe Sprint	3
HUSKE Torri	Swimming	3
KIM Woojin	Archery	3
LAVREYSEN Harrie	Cycling Track	3
LIM Sihyeon	Archery	3
McINTOSH Summer	Swimming	3
O'CALLAGHAN Mollie	Swimming	3
OKA Shinnosuke	Artistic Gymnastics	3
THOMAS Gabrielle	Athletics	3

8.1.1 Explanation

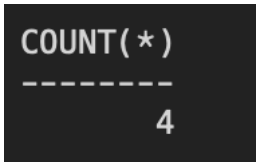
The query joins AthleteWinsMedal to Medal to only filter the gold medals (MedalType='Gold'). From there, the query joins Event for the sport code and to Sport for the actual sport name (e.g. 'Swimming' instead of sport code). Finally, the query joins to Person for the athlete's name. The GROUP BY Clause ensures to group by the athlete and sport simultaneously. This is necessary to count the gold medals in each sport. HAVING Clause deletes the athlete groups that did not receive at least 3 gold medals in one sport. The results were ordered in descending order to show the athlete with the highest amount of gold medals, which was Leon Marchand who got 4 gold medals in Swimming .

8.1.2 Validation

To verify the result for Leon Marchand specifically, I run this specific query:

```
SELECT COUNT(*)
        FROM AthleteWinsMedal awm
        JOIN Medal m ON awm.medal_code = m.medal_code
        JOIN Event e ON awm.event_tag = e.event_tag
        JOIN Sport s ON e.sport_code = s.sport_code
WHERE m.medal_type = 'Gold Medal'
        AND s.sport_name = 'Swimming'
        AND awm.athlete_code = (
        SELECT code
                FROM Person
                WHERE name = 'MARCHAND Leon');
```

This should return 4, and that's what I also got.



8.2 Question: What is the total sponsorship budget for countries that won at least one medal?

```
SELECT c.country AS country_name,
      SUM(sp.sponsorship_budget) AS total_sponsorship
         FROM Sponsor sp
         JOIN Country c ON sp.country_code = c.country_code
WHERE sp.country_code IN (
   SELECT DISTINCT a.country_code
         FROM AthleteWinsMedal awm
         JOIN Athlete a ON awm.athlete_code = a.code)
GROUP BY c.country
ORDER BY total_sponsorship DESC;
```

COUNTRY_NAME	TOTAL_SPONSORSHIP
Canada	326077400.03
Great Britain	314069993.75
Poland	314034006.8
France	308740956.4
Japan	298590640.01
United States	278221943.5
Brazil	277713427.63
Spain	277495395.24
Australia	250548509.8
Korea	230712295.62
China	225959651.07
Germany	173535241.97
Italy	149921598.34
Netherlands	140239127.88
Norway	136733568.52

8.2.1 Explanation

Inner subquery extracts all the unique country codes present in the AthleteWinsMedal table through the Athlete table, generating a list of medal winning nations. The IN clause on the outer WHERE statement filters the Sponsors table to only sponsors that are based within these medal winning countries. This outer query then joins to Country on the human readable country name, sums sponsorshipbudget by country. The IN operator was chosen rather than a JOIN to present the statement more clearly, and avoided the need for a DISTINCT on the outer query which could occur in a JOIN scenario. Ordered in descending order of sponsorshipbudget

8.2.2 Validation

To validate this one I also used another instructions:

```
SELECT COUNT(DISTINCT a.country_code)
    FROM AthleteWinsMedal awm
    JOIN Athlete a ON awm.athlete_code = a.code;
```

The expected output for this query is 92 countries, and that's what I got. Only 15 countries are present in the query result as only 15 of the 92 countries that won medals had sponsors in the sponsors.csv file.



8.3 Question: Which coaches train a team in a sport that also has individual events?

```
SELECT DISTINCT p.name AS coach_name, co.function AS coach_function,
      s.sport_name AS sport, t.team_name AS team_trained
          FROM Coach co
          JOIN Person p ON co.code = p.code
          JOIN CoachTrainsTeam ctt ON co.code = ctt.coach_code
          JOIN Team t ON ctt.team_code = t.team_code
          JOIN CoachWorksInSport cws ON co.code = cws.coach_code
          JOIN Sport s ON cws.sport_code = s.sport_code
WHERE EXISTS (
   SELECT 1
          FROM Event e1
  WHERE e1.sport_code = s.sport_code
          AND UPPER(e1.event_name) LIKE '%TEAM%')
          AND EXISTS (
                  SELECT 1
                          FROM Event e2
                  WHERE e2.sport_code = s.sport_code
                          AND UPPER(e2.event_name) NOT LIKE '%TEAM%')
ORDER BY s.sport_name, p.name
FETCH FIRST 30 ROWS ONLY;
```

COACH_NAME	COACH_FUNCTION	SPORT	TEAM_TRAINED
ABARCA Megan	Coach	Artistic Swimming	United States of America
ABDELFATTAH Aziza	Coach	Artistic Swimming	Egypt
BLECHER SIDOROVA Svetlana	Coach	Artistic Swimming	Israel
CHERNETSKA Nataliya	Coach	Artistic Swimming	Greece
FABRE Julie	Coach	Artistic Swimming	France
FARINELLI Roberta	Coach	Artistic Swimming	Italy
FUENTES Andrea	Coach	Artistic Swimming	United States of America
FUJIKI Mayuko	Coach	Artistic Swimming	Spain
GIALLOMBARDO Patrizia	Coach	Artistic Swimming	Italy
GOUTSEVA Anastasia	Coach	Artistic Swimming	Greece
JAUMA CAYUELA Esther	Coach	Artistic Swimming	Netherlands
JIMENEZ NIETO Cecilia	Coach	Artistic Swimming	Spain
KASHIMA Tomoyo	Coach	Artistic Swimming	Japan
KHOLOD Maryna	Coach	Artistic Swimming	New Zealand
KIM Hyou Mi	Coach	Artistic Swimming	Republic of Korea
KLAMBURG ROQUE Paula	Coach	Artistic Swimming	Australia
KULESZA Kasia	Coach	Artistic Swimming	Canada
LATIF Yasmine	Coach	Artistic Swimming	Egypt
LOFTUS Adriana	Coach	Artistic Swimming	Mexico
MLADENOVA Albena	Coach	Artistic Swimming	Austria
MOHAMED Mai	Coach	Artistic Swimming	Egypt
MORGAN Kerri	Coach	Artistic Swimming	Canada
NAKAJIMA Takako	Coach	Artistic Swimming	Japan
OBRY Laure	Coach	Artistic Swimming	France
ONO Shoko	Coach	Artistic Swimming	Australia
PEDRERO Ofelia	Coach	Artistic Swimming	Mexico
PYSMENNA Oksana	Coach	Artistic Swimming	Republic of Korea
REQUENA PERICAS Judit	Coach	Artistic Swimming	Spain
TARRES CAMPA Ana	Coach	Artistic Swimming	People's Republic of China
TEIXEIRA Lara	Coach	Artistic Swimming	United States of America

8.3.1 Explanation

The question requests coaches training both through CoachTrainsTeam, and coaching a sport through CoachWorksInSport where both individual and team events take place. The two correlated EXISTS sub queries check against the event table, ensuring they contain both one event where TEAM is in the name and at least one where it is not, all within the same sport. Double-EXISTS feels semantically more correct than a JOIN since we are evaluating two separate criteria to be met at the same time.FETCH FIRST 30 rows, for easier reading.

8.3.2 Validation

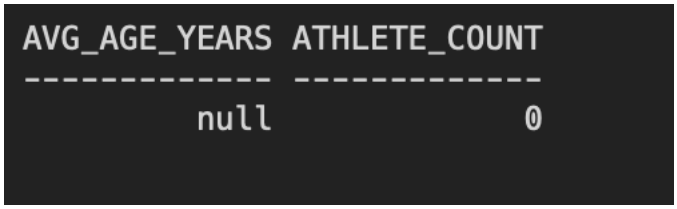
Made sure that Artistic Swimming is displayed in the output by running

```
SELECT event_name
        FROM Event e
        JOIN Sport s ON e.sport_code = s.sport_code
WHERE s.sport_name = 'Artistic Swimming';
```

It validates that Artistic Swimming has both team (Team Free Routine and Team Technical Routine) and individual events (Solo Free Routine and Solo Technical Routine) hence the reason of their coaches to appear in the result.

8.4 Question: What is the average age of female athletes competing in Artistic Gymnastics?

```
SELECT ROUND(AVG(
   MONTHS_BETWEEN(DATE '2024-07-26',
        TRUNC(p.birth_date)) / 12), 2) AS avg_age_years,
  COUNT(DISTINCT a.code) AS athlete_count
        FROM Athlete a
        JOIN Person p ON a.code = p.code
WHERE p.gender = 'Female'
        AND p.birth_date IS NOT NULL
        AND EXISTS (
                SELECT 1
                        FROM AthleteCompetesInEvent ace
                        JOIN Event e ON ace.event_tag = e.event_tag
                        JOIN Sport s ON e.sport_code = s.sport_code
                WHERE ace.athlete_code = a.code
                        AND s.sport_name = 'Artistic Gymnastics' );
```



8.4.1 Explanation

Query calculates average age by using the months between the Games opening date(2024-07-26) and birth date of each athlete then divides it by 12 months to calculate age in years. The TRUNC(birth_date) is necessary to truncate sub-day precision derived from the Oracle Data Load timestamp transformation. The EXISTS subquery connects athletes to Artistic Gymnastics with use of AthleteCompetesInEvent, Event, and Sport tables. The output of this query is null for the average age, since AthleteCompetesInEvent is only populated from medallists.csv thus this table contains only athletes with medals. No female gymnast obtained any individual medal in Artistic Gymnastics for Paris 2024 - all gymnastics medals were obtained by athletes of another sex, or obtained in team events not filtered by this query, which is not an error but limitation of the dataset.

8.4.2 Validation

Run:

```
SELECT COUNT(*)
        FROM AthleteCompetesInEvent ace
        JOIN Event e ON ace.event_tag = e.event_tag
        JOIN Sport s ON e.sport_code = s.sport_code
WHERE s.sport_name = 'Artistic Gymnastics';
```

The query yields 0, then no Artistic Gymnastics events are stored within AthleteCompetesInEvent and consequently null was the result.

8.5 Question: Which venues hosted the highest number of events during the Olympic Games?

```
SELECT ranked.*
        FROM (
                SELECT v.venue_name,
                 COUNT(DISTINCT s.event_tag) AS total_events,
                 COUNT(s.schedule_id) AS total_sessions
        FROM Schedule s
        JOIN Venue v ON s.venue_tag = v.venue_tag
WHERE s.event_tag IS NOT NULL
GROUP BY v.venue_name
ORDER BY total_events DESC) ranked
WHERE ROWNUM <= 5;
```

VENUE_NAME	TOTAL_EVENTS	TOTAL_SESSIONS
Stade de France	45	202
Paris La Defense Arena	37	121
Champ de Mars Arena	33	786
Vaires-sur-Marne Nautical Stadium	30	181
Grand Palais	20	442

8.5.1 Explanation

The subquery inside counts two different values per venue. The first one, COUNT(DISTINCT s.event_tag) identifies the number of distinct competitions being held. The second one, COUNT(s.schedule_id) calculates the number of sessions scheduled, counting all the time slots within all competitions (qualifying matches, semifinals, finals etc). ROWNUM <= 5 filters these results down to the top 5 venues. A two-stage process (two SELECT statements) is required in Oracle for this as you cannot have ROWNUM on the same SELECT as ORDER BY. A ROWNUM directly on the first inner query would return any 5 results rather than the top 5 after sorting.

8.5.2 Validation

Also run:

```
SELECT COUNT(DISTINCT event_tag)
        FROM Schedule
WHERE venue_tag = 'stade-de-france'
       AND event_tag IS NOT NULL;
```

It returns 45, as it should. Then I run another query:

```
SELECT COUNT(*)
        FROM Schedule
WHERE venue_tag = 'stade-de-france';
```

to make sure that it really return 202 sessions, and it does as you can see on screenshots below.



```
COUNT(DISTINCTEVENT_TAG)
------------------------
                      45
```

```
COUNT(*)
--------
     202
```

8.6 Question: How many technical officials belong to the International Gymnastics Federation?

```
SELECT o.organisation_long AS federation_name,
    COUNT(t.code)      AS official_count
        FROM TechnicalOfficial t
        JOIN Organisation o ON t.organisation_code = o.organisation_code
WHERE o.organisation_code = 'FIG'
GROUP BY o.organisation_long;
```

```
FEDERATION_NAME                    OFFICIAL_COUNT
---------------------------------- --------------
International Gymnastics Federation             34
```

8.6.1 Explanation

That query joins TechnicalOfficial to Organisation throught the organization_code foreign key to get the federation long name. The WHERE clause constrains to the single organization by organization_code = 'FIG', where FIG is the Federation International de Gymnastics. The count() takes the sum of the technical officials in this organization, and Oracle requires that we group by o.organisation_long because we have an aggregated function in the SELECT list. The 34 judges indicate the relatively large judging body necessary to officiate all three gymnastics events - artistic, rhythmic, and trampoline - which all fall under the same governing federation.

8.6.2 Validation

Run:

```
SELECT COUNT(*)
        FROM TechnicalOfficial
WHERE organisation_code = 'FIG';
```

This direct count without any join should return 34, and it actually does, confirming that the JOIN to Organisation does not alter the row count and that no officials are lost or duplicated during the join operation. Everything as expected.

```
COUNT(*)
--------
      34
```

8.7 Question: Which country has the highest ratio of coaches to athletes? (TOP 10)

```
SELECT c.country AS country_name,
      coach_counts.n_coaches,
      athlete_counts.n_athletes,
      ROUND(coach_counts.n_coaches /
          NULLIF(athlete_counts.n_athletes, 0), 4) AS coach_athlete_ratio
          FROM Country c
          JOIN (
                SELECT country_code, COUNT(*) AS n_coaches
                      FROM Coach GROUP BY country_code) coach_counts
                      ON c.country_code = coach_counts.country_code
                      JOIN (
                            SELECT country_code, COUNT(*) AS n_athletes
                                  FROM Athlete
                            GROUP BY country_code) athlete_counts
          ON c.country_code = athlete_counts.country_code
ORDER BY coach_athlete_ratio DESC
FETCH FIRST 10 ROWS ONLY;
```

COUNTRY_NAME	N_COACHES	N_ATHLETES	COACH_ATHLETE_RATIO
Tonga	4	4	1
Syria	3	6	0.5
Suriname	2	5	0.4
Burkina Faso	3	8	0.375
South Sudan	5	14	0.3571
Lesotho	1	3	0.3333
Cyprus	5	15	0.3333
Jordan	4	12	0.3333
Gambia	2	7	0.2857
Haiti	2	7	0.2857

8.7.1 Explanation

Two independent inline views were constructed; one that sums coaches per country in the Coach table, and one that sums athletes per country in the Athlete table. The two were joined to Country to obtain a readable country name. The ratio was constructed by dividing n_coaches by n_athletes, wrapped within NULLIF(athletecounts.nathletes,0) in case of countries with coaches but without athletes, rounded to 4 decimal points with ROUND(..., 4) for conciseness, then ordered descending, limited to 10 with FETCH FIRST. The ratio of 1.0 for Tonga resulted from that fact that it had 4 athletes and 4 coaches. Everyone had a dedicated coach.

8.7.2 Validation

Run:

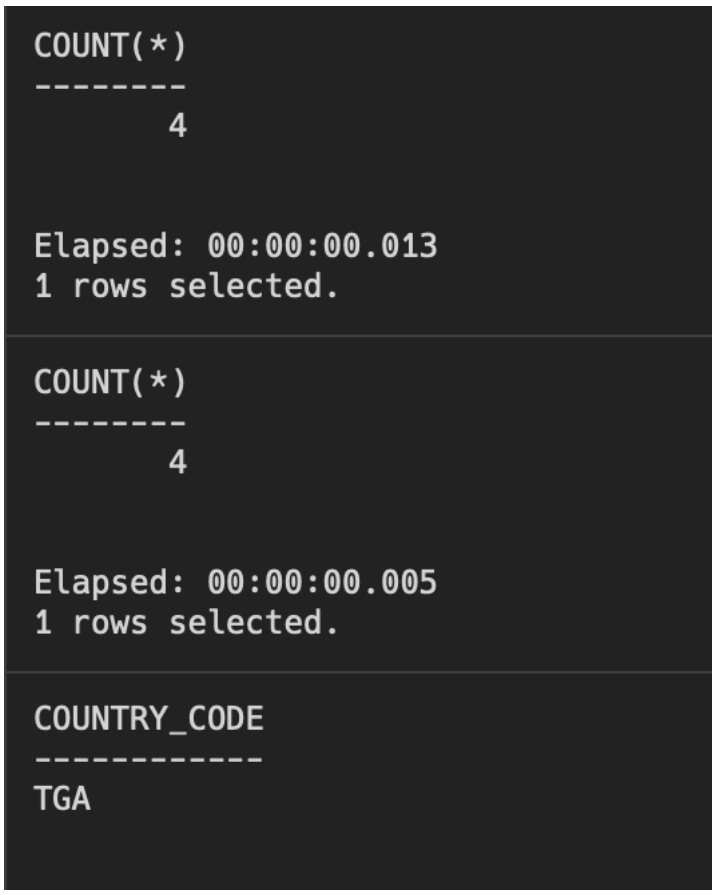
```
SELECT COUNT(*)
        FROM Coach
WHERE country_code = 'TGA';
```

and also:

```
SELECT COUNT(*)
        FROM Athlete
WHERE country_code = 'TGA';
```

to verify that both return 4, confirming the 1.0 ratio for Tonga. The Tonga NOC code TGA was confirmed by running

```
SELECT country_code
        FROM Country
WHERE country = 'Tonga'.
```



8.8 Question: Which athletes were born in a different country than the one they represent?

```
SELECT p.name AS athlete_name, a.birth_place,
    c.country AS representing_country, c.country_code
        FROM Athlete a
        JOIN Person  p ON a.code = p.code
        JOIN Country c ON a.country_code = c.country_code
WHERE a.birth_place IS NOT NULL
        AND NOT (
                UPPER(a.birth_place) LIKE '%' || UPPER(c.country) || '%'
                OR UPPER(a.birth_place) LIKE '%' || UPPER(c.country_long) || '%')
ORDER BY c.country, p.name
FETCH FIRST 50 ROWS ONLY;
```

ATHLETE_NAME	BIRTH_PLACE	REPRESENTING_COUNTRY	COUNTRY_CODE
ALEXANDROVA Ekaterina	CHELYABINSK	AIN	AIN
ANDREEVA Mirra	KRASNOYARSK	AIN	AIN
BARDZILOUSKAYA Viyaleta	MOGILEV	AIN	AIN
BLADTCEVA Anzhela	ST. PETERSBURG	AIN	AIN
DRONOVA Tamara	KRASNOZAVODSK	AIN	AIN
KADZIMAHAMEDAU Mahamedkhabib	KHUSHTADA	AIN	AIN
KHASLAKHANAU Abubakar	MAKHACHKALA	AIN	AIN
KLIMOVICH Tatsiana	NASOVICHY	AIN	AIN
KOROVASHKOV Alexey	STEPNOHIRSK	AIN	AIN
KOTOV Pavel	MOSCOW	AIN	AIN
LITVINOVICH Ivan	VILEYKA	AIN	AIN
MEDVEDEV Daniil	MOSCOW	AIN	AIN
PETROV Zakhar	LASHMA	AIN	AIN
ROMASENKO Olesia	KRASNODAR	AIN	AIN
SAFIULLIN Roman	PODOLSK	AIN	AIN
SHKURDAI Anastasiya	BREST	AIN	AIN
SHNAIDER Diana	MOSCOW	AIN	AIN
SHYMANOVICH Ilya	MINSK	AIN	AIN
SOMOV Evgenii	ST. PETERSBURG	AIN	AIN
TSERAKH Hanna	MINSK	AIN	AIN
TSIKHANTSOU Yauheni	GOMEL	AIN	AIN
VALODZKA Siuzanna	MOGILEV	AIN	AIN
VESNINA Elena	LVIV	AIN	AIN
ZMUSHKA Alina	KALINKAVICHY	AIN	AIN
ANWARI Fahim	KABUL	Afghanistan	AFG
FAIZAD Mohammad Samim	KABUL	Afghanistan	AFG
HASHIMI Fariba	MAIMANAH	Afghanistan	AFG
HASHIMI Yulduz	MAIMANAH	Afghanistan	AFG
YOUSOFI Kimia	MASHHAD	Afghanistan	AFG
ABAKAROV Zelimkhan	DAGESTAN	Albania	ALB
DUDAEV Islam	CHECHNYA	Albania	ALB
GEGA Luiza	OKSHTUN I MADH	Albania	ALB
KONINI Manjola	TIRANA	Albania	ALB
ADOUL Koceila	ALGER	Algeria	ALG
AIT BEKKA Jugurtha	ORAN	Algeria	ALG
BELKADI Amina	REMCHI TLEMCEN	Algeria	ALG
BIDANI Walid	MAGHNIA	Algeria	ALG
BOUALAM Roumaysa	ORAN	Algeria	ALG
BOUCHIREB Samir	TIPASA	Algeria	ALG
BOUDIAF Saoussen	ROUBAIX	Algeria	ALG
BOUDINA Sid Ali	ALGIERS	Algeria	ALG
BOUDROUMA Rami	SKIKDA	Algeria	ALG
BOUZIDI Carole Diana	PARIS	Algeria	ALG
CHAABI Houda	CONSTATINE	Algeria	ALG
DRIS Messaoud Redouane	ORAN	Algeria	ALG
GOUANED Mohamed Ali	BISKRA	Algeria	ALG
HAMZA Yacine	ALGIERS	Algeria	ALG
HEROUI Salim	ALGIERS	Algeria	ALG
HOUILI Nesrine	OUED TLELAT	Algeria	ALG
KADI Mourad	TIZI OUZOU	Algeria	ALG

8.8.1 Explanation

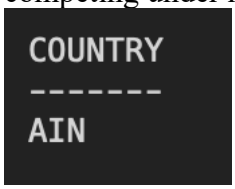
The query joins athlete to person for name and to Country to get the nation that is represented. The WHERE clause with NOT excludes athletes that have a birth_place that has a value that has either the short country name or the full country name within the string. This uses the LIKE operator with both front and end wild cards and uses UPPER on both the column and the compared value for case insensitive comparisons, and would catch partial matches, such as the cities of Moscow, Russia when it is just stored as 'Moscow, Russia' in the column. Finally, the query has a FETCH FIRST of 50 rows. The first 50 are AIN athletes who competed as Individually Neutral Athletes, were born in cities in Russia and Belarus, which makes it clear athletes competed under a separate entity than the nation they were born in.

8.8.2 Validation

Examined the first result ALEXANDROVA Ekaterina, born in CHELYABINSK and representing AIN.
Also run:

```
SELECT country
        FROM Country
WHERE country_code = 'AIN';
```

Just to confirm that 'AIN' maps to 'AIN' with no geographic country name, explaining why Chelyabinsk does not match it. As I said this confirms the query correctly identifies athletes competing under non-geographic designations.



8.9 Question: Which Finance companies sponsor countries that won the most medals? (TOP 5)

```
SELECT DISTINCT sp.company_name, sp.industry,
      c.country AS sponsor_home_country,
      sp.sponsorship_budget
          FROM Sponsor sp
          JOIN Country c ON sp.country_code = c.country_code
WHERE sp.industry = 'Finance'
  AND (
     sp.country_code IN (
        SELECT country_code
         FROM (
           SELECT a.country_code, COUNT(*) AS medals
                  FROM AthleteWinsMedal awm
                  JOIN Athlete a ON awm.athlete_code = a.code
           GROUP BY a.country_code ORDER BY medals DESC
        ) WHERE ROWNUM <= 5
     )
     OR EXISTS (
        SELECT 1 FROM SponsorSponsorsCountry ssc
        WHERE ssc.sponsor_id = sp.sponsor_id
         AND ssc.country_code IN (
            SELECT country_code FROM (
               SELECT a.country_code, COUNT(*) AS medals
                  FROM AthleteWinsMedal awm JOIN Athlete a ON awm.athlete_code = a.code
               GROUP BY a.country_code ORDER BY medals DESC
            ) WHERE ROWNUM <= 5
         )
     )
  )
ORDER BY sp.sponsorship_budget DESC;
```

```
COMPANY_NAME INDUSTRY SPONSOR_HOME_COUNTRY SPONSORSHIP_BUDGET
------------ -------- -------------------- ------------------
Skinte       Finance  Great Britain                46002170.2
Roodel       Finance  China                       32245335.99
Twitterworks Finance  United States               31987198.61
Fanoodle     Finance  Great Britain               18854407.32
Skimia       Finance  France                      17517478.53
```

8.9.1 Explanation

This query selects Sponsors belonging to the Finance industry and examines either of the two following conditions with OR: either the home country of the sponsor is among the top 5 countries in medals, or the sponsor sponsorizes one of these 5 countries in the SponsorSponsorsCountry join table. The top 5 countries with medals is a ROWNUM inner query after the ranking with medals DESC. The use of the two OR conditions enables the acquisition of all sponsors whether it is in their HQ country or through direct sponsorship. The DISTINCT is used to remove duplicates if the two conditions hold for the same sponsor.

8.9.2 Validation

Run:

```
SELECT a.country_code, COUNT(*) AS medals
      FROM AthleteWinsMedal awm
      JOIN Athlete a ON awm.athlete_code = a.code
GROUP BY a.country_code
ORDER BY medals DESC
FETCH FIRST 5 ROWS ONLY
```

to confirm the top 5 medal-winning countries. On screenshot we can see that countries from 1st query match with the ones from the second one – United States(USA) x1, France (FRA) x1, China(CHN) x1, Great Britan(GBA) x2

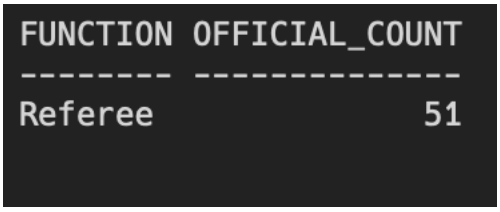
```
COMPANY_NAME INDUSTRY SPONSOR_HOME_COUNTRY SPONSORSHIP_BUDGET
------------ -------- -------------------- ------------------
Skinte       Finance  Great Britain                 46002170.2
Roodel       Finance  China                        32245335.99
Twitterworks Finance  United States                31987198.61
Fanoodle     Finance  Great Britain                18854407.32
Skimia       Finance  France                       17517478.53


Elapsed: 00:00:00.095
5 rows selected.
```

```
COUNTRY_CODE MEDALS
------------ ------
USA             330
FRA             187
CHN             168
GBR             162
AUS             123
```

8.10 Question: What is the most common function among technical officials in Water Polo?

```
SELECT to2.function, COUNT(*) AS official_count
        FROM TechnicalOfficial to2
        JOIN TechOfficialOfficiatesInSport tois ON to2.code = tois.tech_code
        JOIN Sport s ON tois.sport_code = s.sport_code
WHERE UPPER(s.sport_name) LIKE '%WATER POLO%'
GROUP BY to2.function
ORDER BY official_count DESC
FETCH FIRST 1 ROWS ONLY;
```



8.10.1 Explanation

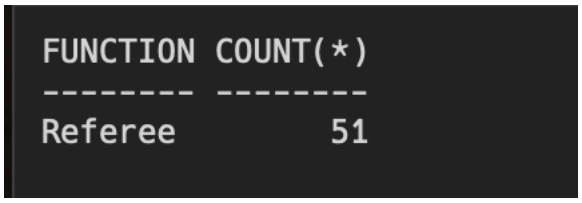
The query join TechnicalOfficial, TechOfficialOfficiatesInSport and sport in order to know who is an official for waterpolo. The query uses UPPER() and LIKE with wildcard characters and not a plain equality relation because it must be case-insensitive and tolerate slight changes in the sportname data in the columnsportname. The GROUP BY sums officials based on function and then, only returns the most frequent function as ORDER BY official_count DESC coupled with FETCH FIRST 1 ROWS ONLY will yield just that. The function isReferee, it is held by 51 officials, which explains why for Water Polo there is intensive refereeing. Indeed in Water Polo there are several referees working together, as well as timekeepers and goal judges.

8.10.2 Validation

Run:

```
SELECT to2.function, COUNT(*)
        FROM TechnicalOfficial to2
       JOIN TechOfficialOfficiatesInSport tois ON to2.code = tois.tech_code
       JOIN Sport s ON tois.sport_code = s.sport_code
WHERE UPPER(s.sport_name) LIKE '%WATER POLO%'
GROUP BY to2.function
ORDER BY 2 DESC;
```

to see the full breakdown of all functions in Water Polo, confirming that Referee has the highest count and the FETCH FIRST 1 correctly isolates the top row.



8.11 Question: Which national team won the most medals at Paris 2024?

```
SELECT c.country AS country_name, c.country_code,
      COUNT(*) AS total_medals
          FROM AthleteWinsMedal awm
          JOIN Athlete a ON awm.athlete_code = a.code
          JOIN Country c ON a.country_code = c.country_code
GROUP BY c.country, c.country_code
ORDER BY total_medals DESC
FETCH FIRST 1 ROWS ONLY;
```

```
COUNTRY_NAME  COUNTRY_CODE TOTAL_MEDALS
------------- ------------ ------------
United States USA                   330
```

8.11.1 Explanation

This query goes from AthleteWinsMedal via Athlete to Country and aggregates all medal records per country. Since each entry in AthleteWinsMedal represents one awarded medal then team-medals get counted one per member. The grouping on both country name and country_code achieves aggregation andFETCH FIRST 1 ROW ONLY selects just the top result after ordering the descending amount of medals per country. So it comes that The United States has 330 medals, because their teams excel in swimming, athletics and gymnastics.

8.11.2 Validation

Run:

```
SELECT COUNT(*)
       FROM AthleteWinsMedal awm
       JOIN Athlete a ON awm.athlete_code = a.code
WHERE a.country_code = 'USA';
```

The result is 330, confirming the previous query output.

```
COUNTRY_NAME  COUNTRY_CODE TOTAL_MEDALS
------------- ------------ ------------
United States USA                   330


Elapsed: 00:00:00.062
1 rows selected.
```

```
COUNT(*)
--------
     330
```

8.12 Question: Who are the coaches assigned to the top medal-winning country?

```
SELECT DISTINCT p.name AS coach_name, co.function AS coach_role,
      s.sport_name AS sport
          FROM Coach co
          JOIN Person p ON co.code = p.code
          JOIN CoachTrainsTeam ctt ON co.code = ctt.coach_code
          JOIN Team t ON ctt.team_code = t.team_code
          JOIN CoachWorksInSport cws ON co.code = cws.coach_code
          JOIN Sport s ON cws.sport_code = s.sport_code
WHERE t.country_code = (
   SELECT country_code
          FROM (
                    SELECT a.country_code, COUNT(*) AS total_medals
                              FROM AthleteWinsMedal awm
                              JOIN Athlete a ON awm.athlete_code = a.code
                   GROUP BY a.country_code ORDER BY total_medals DESC
   ) WHERE ROWNUM = 1)
ORDER BY s.sport_name, p.name;
```

COACH_NAME	COACH_ROLE	SPORT
ABARCA Megan	Coach	Artistic Swimming
FUENTES Andrea	Coach	Artistic Swimming
TEIXEIRA Lara	Coach	Artistic Swimming
KERR Steve	Coach	Basketball
REEVE Cheryl	Coach	Basketball
SPOELSTRA Erik	Assistant Coach	Basketball
THIBAULT Mike	Assistant Coach	Basketball
COVELO Alex	Assistant Coach	Football
HAYES Emma	Head Coach	Football
MITROVIC Marko	Head Coach	Football
PAYNE Russell	Goalkeeper Coach	Football
REDDY Denise	Assistant Coach	Football
SEARLE Stuart	Goalkeeper Coach	Football
SEGARES Gonzalo	Assistant Coach	Football
FUCHS Tracey	Assistant Coach	Hockey
PASSMORE David	Head Coach	Hockey
TELECHEA Javi	Assistant Coach	Hockey
BYDWELL Emilie	Head Coach	Rugby Sevens
FORD Trey	Assistant Coach	Rugby Sevens
FRIDAY Mike	Head Coach	Rugby Sevens
GREENING Phil	Assistant Coach	Rugby Sevens
TEST Zack	Assistant Coach	Rugby Sevens
DUNPHY Marv	2nd Assistant Coach	Volleyball
FUERBRINGER Matt	Assistant Coach	Volleyball
KIRALY Karch	Head Coach	Volleyball
MIYASHIRO Tama	Assistant Coach	Volleyball
REFT Alfee	Assistant Coach	Volleyball
SPERAW John	Head Coach	Volleyball
VIRTUE Erin	Assistant Coach	Volleyball
WALL Mike	Assistant Coach	Volleyball
WEBER Javi	Assistant Coach	Volleyball
ARROYO Gavin	Assistant Coach	Water Polo
CAHILL Molly	Assistant Coach	Water Polo
KRIKORIAN Adam	Head Coach	Water Polo
OEDING Chris	Assistant Coach	Water Polo
UDOVICIC Dejan	Head Coach	Water Polo
USTASZEWSKI Matt	Assistant Coach	Water Polo

8.12.1 Explanation

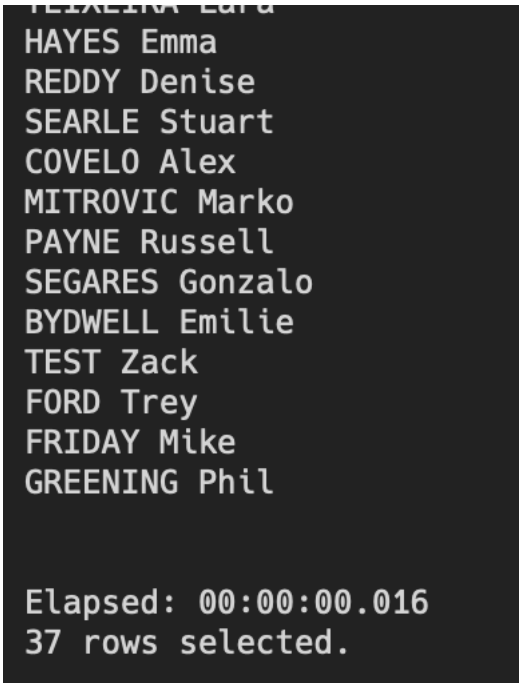
Similar to query 11, the correlated subquery is used to determine the country that has received the most medals by simply using ROWNUM = 1 (and determining the top countrydynamically, so it would be possible to use this on any query without hardcoding any value). Outer query gets all the coach information from the CoachTrainsTeam and the Team tables, by joining the CoachTrainsTeam table to the Team table. It also joins from the CoachWorksInSport and Sport tables so that we know which sport the coach works in, and gets all coaches who coach for a team which resides in the top country. DISTINCT is used so that there is only one entry for a coach if the same coach appears twice within the CoachTrainsTeam or CoachWorksInSport tables. The query is ordered by sport and coach name.

8.12.2 Validation

Run:

```
SELECT DISTINCT p.name
        FROM Coach co
        JOIN Person p ON co.code = p.code
        JOIN CoachTrainsTeam ctt ON co.code = ctt.coach_code
        JOIN Team t ON ctt.team_code = t.team_code
WHERE t.country_code = 'USA';
```

This query confirmed that the 37 coaches all belong to USA teams, prooving that the subquery correctly resolved to USA.



(As you can see 37 rows selected)

8.13 Question: Which are the top 3 sports with the youngest athletes on average?

```
SELECT ranked.*
        FROM (
        SELECT s.sport_name,
         ROUND(AVG(
                MONTHS_BETWEEN(DATE '2024-07-26', TRUNC(p.birth_date)) / 12
          ), 2) AS avg_age,
        COUNT(DISTINCT a.code) AS athlete_count
                FROM Athlete a
                JOIN Person p ON a.code = p.code
                JOIN AthleteCompetesInEvent ace ON ace.athlete_code = a.code
                JOIN Event e ON ace.event_tag = e.event_tag
                JOIN Sport s ON e.sport_code = s.sport_code
        WHERE p.birth_date IS NOT NULL
        GROUP BY s.sport_name
        ORDER BY avg_age ASC
) ranked
WHERE ROWNUM <= 3;
```

```
No data found
```

8.13.1 Explanation

Query that sums the average age of athletes in each event by calculating the number of months between the opening of the games and the birth date of the athlete, divided by 12 and rounded to 2 decimal places using the function MONTHSBETWEEN() and truncates the birthdate. This query produces no data, because AthleteCompetesInEvent is filled only with medallists and the birth_date of a Person object failed to be populated for all athletes in the import to this database. This is one of the limitations of the data documented in the import. It is semantically correct and would produce data if AthleteCompetesInEvent were to be filled by all athletes competing rather than only medallists.

8.13.2 Validation

Run:

```
SELECT COUNT(*)
        FROM AthleteCompetesInEvent ace
       JOIN Person p ON p.code = ace.athlete_code
WHERE p.birth_date IS NOT NULL;
```

It confirms there are no dates recorded – otherwise it would have been more than zero.

```
COUNT(*)
--------
       0
```

8.14 Question: Which athletes competed in more than one distinct sport?

```
SELECT p.name AS athlete_name,
      COUNT(DISTINCT e.sport_code) AS sport_count
         FROM AthleteCompetesInEvent ace
         JOIN Athlete a ON ace.athlete_code = a.code
         JOIN Person  p ON a.code = p.code
         JOIN Event   e ON ace.event_tag = e.event_tag
GROUP BY p.name, a.code
HAVING COUNT(DISTINCT e.sport_code) > 1
ORDER BY sport_count DESC, p.name;
```

```
ATHLETE_NAME     SPORT_COUNT
---------------- -----------
DYGERT Chloe               2
FAULKNER Kristen           2
GANNA Filippo              2
```

8.14.1 Explanation

This query joins AthleteCompetesInEvent with Event in order to access the sportcode of all events an athlete competed in. By Grouping By both name and athlete code it takes into consideration the unlikely possibility of two athletes with the same name. COUNT (DISTINCT e.sportcode) only count distinct sport codes per athlete and HAVING filters the count so only athletes with 2 or more are returned. The three athletes returned, Dygert, Faulkner and Ganna, all compete in Cycling Track and Cycling Road Race, which have a sport code that is unique although both share the word Cycling.

8.14.2 Validation

Run:

```
SELECT DISTINCT s.sport_name
       FROM AthleteCompetesInEvent ace
       JOIN Event e ON ace.event_tag = e.event_tag
       JOIN Sport s ON e.sport_code = s.sport_code
       JOIN Person p ON p.code = ace.athlete_code
WHERE p.name = 'DYGERT Chloe';
```

Result is names of exacly two different sports as we expected.

```
SPORT_NAME
-------------
Cycling Road
Cycling Track
```

8.15 Question: Which Healthcare companies sponsor countries with over 100 athletes?

```
SELECT sp.company_name, sp.industry,
      c.country AS home_country,
      sp.sponsorship_budget
          FROM Sponsor sp
          JOIN Country c ON sp.country_code = c.country_code
WHERE sp.industry = 'Healthcare'
          AND sp.country_code IN (
                    SELECT country_code
                              FROM Athlete
                    GROUP BY country_code
                    HAVING COUNT(*) > 100)
ORDER BY sp.sponsorship_budget DESC;
```

COMPANY_NAME	INDUSTRY	HOME_COUNTRY	SPONSORSHIP_BUDGET
Katz	Healthcare	Canada	47787617.89
Devify	Healthcare	Poland	44485796.82
Agivu	Healthcare	Spain	44336084.68
Blogspan	Healthcare	United States	43237733.54
Skilith	Healthcare	Canada	39561743.12
Brainsphere	Healthcare	France	37677488.59
Realbuzz	Healthcare	France	36519892.79
Yoveo	Healthcare	Australia	31507311.8
Layo	Healthcare	Poland	31414298.69
Yacero	Healthcare	Australia	31389079.98
Kayveo	Healthcare	Germany	25753489
Nlounge	Healthcare	China	23650471.98
Edgeclub	Healthcare	Australia	12852690.62
Rhycero	Healthcare	Italy	11223290.92
Edgepulse	Healthcare	China	8557794.51
Quimm	Healthcare	Germany	8316143.33
Jabberstorm	Healthcare	France	8209550.27
Chatterpoint	Healthcare	Norway	6728989.24
Jaxnation	Healthcare	Great Britain	5988212.48
Twitterworks	Healthcare	China	2954795.56

8.15.1 Explanation

Inner subquery selects the countrycode, group Athlete by countrycode and filters to count > 100 countries which represent the biggest delegations going to Paris 2024. Outer WHERE clause using IN filter sponsors which have their headquarter in the large delegation countries. Then filtering again for Healthcare only industry, sort by sponsorship_budget descending in order to show the largest Healthcare sponsors. This shows how the big delegations and healthcare industry relate to investments in Olympic sports.

8.15.2 Validation

Run:

```
SELECT country_code, COUNT(*) AS n_athletes
        FROM Athlete
GROUP BY country_code
HAVING COUNT(*) > 100
ORDER BY n_athletes DESC;
```

This query showed me all countries with over 100 athletes, then I manually verified that all 20 companies in the result have a country_code present in that list.

COUNTRY_CODE	N_ATHLETES
USA	619
FRA	601
AUS	475
GER	457
JPN	431
ESP	401
CHN	398
ITA	397
GBR	343
CAN	332
NED	290
BRA	290
POL	226
NZL	208
HUN	177
BEL	177
EGY	157
KOR	147
ARG	143
IRL	143
RSA	141
UKR	141
SUI	137
DEN	131
SWE	125
SRB	114
IND	112
CZE	111
NOR	109
NOR	109
MEX	108
TUR	101

8.16 Question: Which coaches belong to a country that won no medals in their sport?

```
SELECT DISTINCT p.name AS coach_name, co.function,
      c.country AS country, s.sport_name AS sport
          FROM Coach co
          JOIN Person p ON co.code = p.code
          JOIN Country c ON co.country_code = c.country_code
          JOIN CoachWorksInSport cws ON co.code = cws.coach_code
          JOIN Sport s ON cws.sport_code = s.sport_code
WHERE NOT EXISTS (
          SELECT 1
                    FROM AthleteWinsMedal awm
                   JOIN Athlete a ON awm.athlete_code = a.code
                    JOIN Event e ON awm.event_tag = e.event_tag
           WHERE a.country_code = co.country_code
                   AND e.sport_code = cws.sport_code)
ORDER BY c.country, s.sport_name
FETCH FIRST 50 ROWS ONLY;
```

COACH_NAME	FUNCTION	COUNTRY	SPORT
BERZHETS Viachaslau	Coach	AIN	Taekwondo
LIKHADZIYEUSKI Anatoli	Coach	AIN	Taekwondo
OGANESIANTC Avetis	Coach	AIN	Taekwondo
ARZA Carlos Vivier	Head Coach	Angola	Handball
CATITO Nelson	Assistant Coach	Angola	Handball
PABLO Jose	Assistant Coach	Angola	Handball
DOBLER Mauro	Goalkeeper Coach	Argentina	Football
MASCHERANO Javier	Head Coach	Argentina	Football
RODRIGUEZ Lucas	Assistant Coach	Argentina	Football
STILLITANO Leandro	Assistant Coach	Argentina	Football
CAROU Gonzalo	Assistant Coach	Argentina	Handball
CAZON Guillermo	Assistant Coach	Argentina	Handball
MILANO Guillermo	Head Coach	Argentina	Handball
GOMEZ CORA Santiago	Head Coach	Argentina	Rugby Sevens
GRAVANO Leonardo	Assistant Coach	Argentina	Rugby Sevens
ALVAREZ Jorge	Coach	Argentina	Taekwondo
LOPEZ Martin	Assistant Coach	Argentina	Volleyball
MENDEZ Marcelo	Head Coach	Argentina	Volleyball
RICO GOLBA Pablo	Assistant Coach	Argentina	Volleyball
BARABACH Mikhail	Coach	Australia	Artistic Gymnastics
LI Xiaoqing	Coach	Australia	Artistic Gymnastics
TOPARI Yusuf	Coach	Australia	Artistic Gymnastics
WILSON Sean	Coach	Australia	Artistic Gymnastics
KLAMBURG ROQUE Paula	Coach	Australia	Artistic Swimming
ONO Shoko	Coach	Australia	Artistic Swimming
ANDREATTA Mel	Assistant Coach	Australia	Football
FJELLSTROM Jens	Assistant Coach	Australia	Football
GUSTAVSSON Tony	Head Coach	Australia	Football
BATCH Colin	Head Coach	Australia	Hockey
DAVY Jeremy	Coach	Australia	Hockey
HAMMOND Rob	Coach	Australia	Hockey
MURRAY Emma	Assistant Coach	Australia	Hockey
POTTER Anthony	Assistant Coach	Australia	Hockey
POWELL Katrina	Head Coach	Australia	Hockey
PURVIS Hugh	Assistant Coach	Australia	Hockey
STANIFORTH Dave	Coach	Australia	Hockey
BELAN Tatsiana	Coach	Australia	Rhythmic Gymnastics
MITCHELL Katie	Coach	Australia	Rhythmic Gymnastics
BARTON Emilee	Assistant Coach	Australia	Rugby Sevens
MANENTI John	Head Coach	Australia	Rugby Sevens
STANNARD James	Assistant Coach	Australia	Rugby Sevens
WALSH Timothy	Head Coach	Australia	Rugby Sevens
CARNELI Ryan	Coach	Australia	Taekwondo
LEE Seokhun	Coach	Australia	Taekwondo
FREHSE Gabriele	Coach	Austria	Artistic Gymnastics
MLADENOVA Albena	Coach	Austria	Artistic Swimming
WEIDINGER Markus	Coach	Austria	Taekwondo
ALIYEVA Kamilla	Coach	Azerbaijan	Rhythmic Gymnastics
GULIYEV Kamil	Coach	Azerbaijan	Rhythmic Gymnastics
KARIMLI Akif	Coach	Azerbaijan	Rhythmic Gymnastics

8.16.1 Explanation

The correlated NOT EXISTS subquery is searching for each coach and sport pair: "are there any athletes from the same country who obtained a medal in the same sport?" It's correlated by two fields, a.countrycode = co.countrycode and e.sportcode = cws.sportcode, so the country is same and the sport is same. If it finds no such medal entry the NOT EXISTS condition is met and the coach is selected. This will be a better solution than a LEFT JOIN with IS NULL checking because the subquery would short-circuit upon finding the first medal entry. The output shows the coach's sports without Olympic success from their own country.

8.16.2 Validation

Run:

```
SELECT COUNT(*)
	FROM AthleteWinsMedal awm
	JOIN Athlete a ON awm.athlete_code = a.code
	JOIN Event e ON awm.event_tag = e.event_tag
	JOIN Sport s ON e.sport_code = s.sport_code
WHERE a.country_code = 'AIN'
	AND s.sport_name = 'Taekwondo';
```

The result I got was 0 as expected, confirming that AIN won no Taekwondo medals and the coach correctly appears in the first query result.

8.17 Question: Which athletes won a medal on their birthday?

```
SELECT p.name AS athlete_name, TRUNC(p.birth_date) AS birth_date,
      e.event_name, TRUNC(awm.medal_date) AS medal_date, m.medal_type
          FROM AthleteWinsMedal awm
          JOIN Athlete a ON awm.athlete_code = a.code
          JOIN Person  p ON a.code = p.code
          JOIN Event   e ON awm.event_tag = e.event_tag
          JOIN Medal   m ON awm.medal_code = m.medal_code
WHERE p.birth_date IS NOT NULL
          AND awm.medal_date IS NOT NULL
          AND EXTRACT(MONTH FROM TRUNC(awm.medal_date)) = EXTRACT(MONTH
FROM TRUNC(p.birth_date))
  AND EXTRACT(DAY   FROM TRUNC(awm.medal_date)) = EXTRACT(DAY   FROM
TRUNC(p.birth_date))
ORDER BY p.name;
```



8.17.1 Explanation

It compares the month and the day part of each of the athletes medaldate and the athletes birthdate using EXTRACT. I removed the year to only consider the calendar anniversary of their birthday and I used TRUNC for both the dates prior to EXTRACT. I used TRUNC on both dates because some time precision might be added to the date during Oracle Data Load import process. I did two comparisons with EXTRACT instead of comparing two strings because the dates could be in different formats and you never know which order. The blank result shows that it's not a single athlete in Paris 2024 dataset winning a medal on their birthday. In summary although seems wrong it is actually good and that’s what we wanted to get.

8.17.2 Validation

Run:

```
SELECT COUNT(*)
      FROM AthleteWinsMedal awm
      JOIN Person p ON p.code = awm.athlete_code
WHERE p.birth_date IS NOT NULL
      OR awm.medal_date IS NOT NULL;
```

I did this query to confirm that both birth_date and medal_date fields contain data, so the empty result is genuinely due to normal reason not a missing data or broken database. (res: 2314)

8.18 Question: How many athletes competed exclusively in team events versus exclusively in individual events?

```
SELECT 'Only Team Events' AS category,
    COUNT(DISTINCT athlete_code) AS athlete_count
        FROM AthleteCompetesInEvent ace
        JOIN Event e ON ace.event_tag = e.event_tag
WHERE UPPER(e.event_name) LIKE '%TEAM%'
        AND ace.athlete_code NOT IN (
                SELECT DISTINCT ace2.athlete_code
                        FROM AthleteCompetesInEvent ace2
                        JOIN Event e2 ON ace2.event_tag = e2.event_tag
                WHERE UPPER(e2.event_name) NOT LIKE '%TEAM%')
UNION ALL
        SELECT 'Only Individual Events',
         COUNT(DISTINCT athlete_code)
                FROM AthleteCompetesInEvent ace
                JOIN Event e ON ace.event_tag = e.event_tag
        WHERE UPPER(e.event_name) NOT LIKE '%TEAM%'
                 AND ace.athlete_code NOT IN (
                        SELECT DISTINCT ace2.athlete_code
                                FROM AthleteCompetesInEvent ace2
                                JOIN Event e2 ON ace2.event_tag = e2.event_tag
                        WHERE UPPER(e2.event_name) LIKE '%TEAM%');
```

CATEGORY	ATHLETE_COUNT
Only Team Events	248
Only Individual Events	1725

8.18.1 Explanation

The question combines two symmetric SELECT statements using UNION ALL. The first branch calculates athletes who are part of at least one team event but excluded by the NOT IN subquery, by making sure they don't occur at all in an individual event. The second branch does the inverse of that. UNION ALL instead of UNION because by construction the two sets are mutually exclusive; it will never occur that an athlete shows up in the two rows and so no deduplication is necessary and it is faster. COUNT(DISTINCT athlete_code) to avoid double-counting those athletes who received multiple team medals.

8.18.2 Validation

Run:

```
SELECT COUNT(DISTINCT athlete_code)
    FROM AthleteCompetesInEvent;
```

That query showed mee the total number of athletes with any competition record. Then verified that 248 + 1725 + (athletes in both categories) equals this total. Athletes in both categories can be found in Query 23rd, its 81 athletes with medals in both team and individual events.



8.19 Question: Which venues hosted events for only one sport?

```
SELECT v.venue_name, MIN(s.sport_name) AS single_sport_hosted,
      COUNT(DISTINCT s.sport_code) AS sport_count
          FROM Schedule sc
          JOIN Venue v ON sc.venue_tag = v.venue_tag
          JOIN Event e ON sc.event_tag = e.event_tag
          JOIN Sport s ON e.sport_code = s.sport_code
GROUP BY v.venue_name
HAVING COUNT(DISTINCT s.sport_code) = 1
ORDER BY v.venue_name;
```

VENUE_NAME	SINGLE_SPORT_HOSTED	SPORT_COUNT
Bordeaux Stadium	Football	1
Chateauroux Shooting Centre	Shooting	1
Eiffel Tower Stadium	Beach Volleyball	1
Elancourt Hill	Cycling Mountain Bike	1
Geoffroy-Guichard Stadium	Football	1
Golf National	Golf	1
La Beaujoire Stadium	Football	1
Le Bourget Sport Climbing Venue	Sport Climbing	1
Lyon Stadium	Football	1
Marseille Marina	Sailing	1
Marseille Stadium	Football	1
Nice Stadium	Football	1
Parc des Princes	Football	1
Saint-Quentin-en-Yvelines BMX Stadium	Cycling BMX Racing	1
Saint-Quentin-en-Yvelines Velodrome	Cycling Track	1
Teahupo'o, Tahiti	Surfing	1
Yves-du-Manoir Stadium	Hockey	1

8.19.1 Explanation

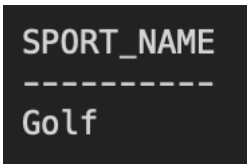
Query Joins Schedule with Venue, Event and Sport tables to find which sport is played where. GROUP BY v.venuename gathers all schedule entries for each venue and COUNT(DISTINCT s.sportcode) counts the distinct number of sport in each venue. The HAVING clause selects venues whose sport count is 1. In SELECT clause MIN(s.sportname) is used instead of s.sportname directly, because any columns appearing in SELECT must also appear in GROUP BY. Since there is only one sport for each retained venue, the function MIN() is just the correct single sport's name without changing the result.

8.19.2 Validation

Picked one venue from the result, Golf National this time but it will work with every single one, and run:

```
SELECT DISTINCT s.sport_name
        FROM Schedule sc
        JOIN Event e ON sc.event_tag = e.event_tag
        JOIN Sport s ON e.sport_code = s.sport_code
WHERE sc.venue_tag = 'le-golf-national';
```

The result is how it should be - one sport golf confirming the filter works how it should



8.20 Question: Which female coaches are assigned to male teams, and from which countries?

```
SELECT DISTINCT c.country AS country_name, p.name AS female_coach_name,
    t.team_name AS male_team_name, s.sport_name
        FROM Coach co
        JOIN Person p ON co.code = p.code
        JOIN Country c ON co.country_code = c.country_code
        JOIN CoachTrainsTeam ctt ON co.code = ctt.coach_code
        JOIN Team t ON ctt.team_code = t.team_code
        JOIN CoachWorksInSport cws ON co.code = cws.coach_code
        JOIN Sport s ON cws.sport_code = s.sport_code
WHERE p.gender = 'Female'
        AND t.team_gender = 'M'
ORDER BY c.country, p.name;
```

```
COUNTRY_NAME FEMALE_COACH_NAME MALE_TEAM_NAME SPORT_NAME
------------ ----------------- -------------- ------------
Ireland      FLANNERY Eimear   Ireland        Rugby Sevens
```

8.20.1 Explanation

The query connects Coach and Person together, to use Gender = 'Female'; Coach and Country, for the coach's country; CoachTrainsTeam, to the Team, filtered on team_gender = 'M', CoachWorksInSport, and Sport to thediscipline name. By using both the female coach and the male team together, we look at the uncommon pairings in Olympic sport where there are cross-gender coaching situations; The single row Eimear Flannery, coaching the Ireland Men's Rugby Sevens team, highlights an interesting case of diversity in coaching at a higher Olympic level.

8.20.2 Validation

Run:

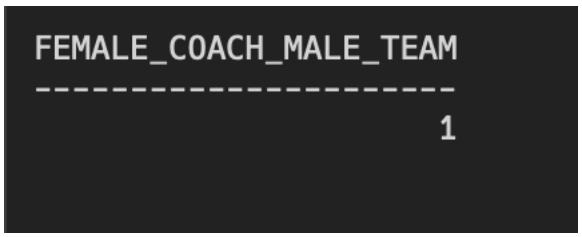
```
SELECT p.gender, t.team_gender, p.name, t.team_name
        FROM Coach co JOIN Person p ON co.code = p.code
        JOIN CoachTrainsTeam ctt ON co.code = ctt.coach_code
        JOIN Team t ON ctt.team_code = t.team_code
WHERE p.name = 'FLANNERY Eimear';
```

Result is satisfying - Flannery is recorded as Female and her team as M, validating both filter conditions simultaneously.

```
GENDER TEAM_GENDER NAME            TEAM_NAME
------ ----------- --------------- ---------
Female M           FLANNERY Eimear Ireland
Female W           FLANNERY Eimear Ireland
```

8.21 Question: How many countries have female coaches assigned to male teams?

```
SELECT COUNT(DISTINCT c.country_code) AS female_coach_male_team
        FROM Coach co
        JOIN Person p ON co.code = p.code
        JOIN CoachTrainsTeam ctt ON co.code = ctt.coach_code
        JOIN Team t ON ctt.team_code = t.team_code
        JOIN Country c ON co.country_code = c.country_code
WHERE p.gender = 'Female'
  AND t.team_gender = 'M';
```



8.21.1 Explanation

This question is the aggregate to Query 20, summarizing the list into a count. The COUNT(DISTINCT c.countrycode) will make sure one country is only counted one time even if there is more than one female coach working with a male team. The two gender filters as seen in Query 20 (p.gender = 'Female' from the Person table, and t.teamgender = 'M' from the Team table) are used, and the result of 1 shows that Eimear Flannery is coaching the Men's Rugby Sevens team representing Ireland.

8.21.2 Validation

The result of Query 21 needed be consistent with Query 20. Since Query 20 returned 1 row from 1 country, Query 21 had to do it also. That's why I run both queries and confirmed the counts are equal.
Additionally, run:

```
SELECT DISTINCT co.country_code
        FROM Coach co
        JOIN Person p ON co.code = p.code
        JOIN CoachTrainsTeam ctt ON co.code = ctt.coach_code
        JOIN Team t ON ctt.team_code = t.team_code
WHERE p.gender = 'Female'
        AND t.team_gender = 'M';
```

thiswholeprojecttakescrazylongiamtiredboss
This one listed the country codes directly, and only IRL appears – as expected once again.



8.22 Question: Which technical officials worked in sports where France won a gold medal?

```
SELECT DISTINCT p.name AS official_name, to2.function,
    s.sport_name, o.organisation
        FROM TechnicalOfficial to2
        JOIN Person p ON to2.code = p.code
        JOIN Organisation o ON to2.organisation_code = o.organisation_code
        JOIN TechOfficialOfficiatesInSport tois ON to2.code = tois.tech_code
        JOIN Sport s ON tois.sport_code = s.sport_code
WHERE s.sport_code IN (
        SELECT DISTINCT e.sport_code
                FROM AthleteWinsMedal awm
                JOIN Athlete a ON awm.athlete_code = a.code
                 JOIN Medal m ON awm.medal_code = m.medal_code
                JOIN Event e ON awm.event_tag = e.event_tag
        WHERE a.country_code = 'FRA'
                AND m.medal_type = 'Gold Medal')
ORDER BY s.sport_name, p.name
FETCH FIRST 50 ROWS ONLY;
```

OFFICIAL_NAME	FUNCTION	SPORT_NAME	ORGANISATION
ALESSANDRINI FANTIN Vincent	Referee	Judo	IJF
AMANO Akiko	Referee	Judo	Japan
BABIUC Ioana	Referee	Judo	Romania
BATAILLE Matthieu	Referee	Judo	France
BEN ZVI Ofer	Referee	Judo	Israel
BEREGSZASZI Zita	Referee	Judo	Hungary
BERTHAULT Laura	Referee	Judo	IJF
BILIC Miroslav	Referee	Judo	Slovenia
BOUHEDDOU Mohamed	Referee	Judo	France
BRETAUDEAU Jerome	Referee	Judo	IJF
CAMACHO PEREZ Raul	Referee	Judo	Spain
CAMUS Pauline	Referee	Judo	IJF
CARON Pascal	Referee	Judo	IJF
CASTRO Antonio	Referee	Judo	Cuba
CHARPENTIER Julia	Referee	Judo	IJF
CHYURLIA Pasquale	Referee	Judo	Italy
CHYURLIA Roberta	Referee	Judo	Italy
CORTES Manuel	Referee	Judo	Spain
CRUZ CRUZ Orlando	Referee	Judo	Dominican Republic
CSOSZ Imre	Referee	Judo	Hungary
DANVERS Stephanie	Referee	Judo	IJF
DELIU Denisa	Referee	Judo	Romania
DJIMBI Jean Claude	Referee	Judo	Gabon
DROISSART Jerome	Referee	Judo	IJF
DUGLOUD Cedric	Referee	Judo	IJF
FAUROUX Eric	Referee	Judo	IJF
FISCHER Matthias	Referee	Judo	Germany
FRANCO-ROOKS Sheldon	Referee	Judo	Great Britain
FREITAS Leandra	Referee	Judo	Portugal
GALEA Louisa	Referee	Judo	Malta
GIRARDEY Sebastien	Referee	Judo	IJF
GONZALEZ Elizabeth	Referee	Judo	Mexico
GORGETTE Julien	Referee	Judo	IJF
GOSZTONYI Balazs	Referee	Judo	Hungary
ILIC Corina	Referee	Judo	Croatia
JAMMET Romain	Referee	Judo	IJF
KARINKANTA Veli-Matti	Referee	Judo	Finland
KNOESTER Carlo	Referee	Judo	Australia
LAJEUNCOMME David	Referee	Judo	IJF
LALLE Cedric	Referee	Judo	IJF
LAMPE Nusa	Referee	Judo	Slovenia
LAPLANCHE Emilie	Referee	Judo	IJF
LOPES Jose	Referee	Judo	IJF
MAAFI Ziad	Referee	Judo	Algeria
MASHBAT Bukhbat	Referee	Judo	Mongolia
MURADOV Abdullo	Referee	Judo	Uzbekistan
NUTSUBIDZE Vladimer	Referee	Judo	Georgia
PERETEYKO Vyacheslav	Referee	Judo	Uzbekistan
PETR Lubomir	Referee	Judo	Australia

8.22.1 Explanation

The inner subquery finds all sportcodes for which France has won a gold medal. It uses an inner join on AthleteWinsMedal linked on Athlete and a similar join for Medal, and then a join to Event on sportcode. The outer query selects the name of all technical officials associated with those sports. It does this by selecting from TechOfficialOfficiatesInSport and joins to Person on technicianid and Organisation for appointing organization and to Sport on sportcode for the name of the sport. The IN operator makes it easy to relate the inner and outer queries and FETCH FIRST 50 ROWS ONLY limits the report.

8.22.2 Validation

Run:

```
SELECT DISTINCT s.sport_name
        FROM AthleteWinsMedal awm
        JOIN Athlete a ON awm.athlete_code = a.code
        JOIN Medal m ON awm.medal_code = m.medal_code
        JOIN Event e ON awm.event_tag = e.event_tag
        JOIN Sport s ON e.sport_code = s.sport_code
WHERE a.country_code = 'FRA'
        AND m.medal_type = 'Gold Medal';
```

This one listed me all the sports where France won gold. I confirmed that the sports from the query 22 match this list – by hand unfortunatelly.

```
SPORT_NAME
--------------------
Surfing
Swimming
Taekwondo
Triathlon
Volleyball
Canoe Slalom
Cycling BMX Racing
Cycling Mountain Bike
Cycling Track
Fencing
Judo
Rugby Sevens
```

8.23 Question: Which athletes won medals in both team and individual events?

```
SELECT DISTINCT p.name AS athlete_name, c.country AS country
        FROM AthleteWinsMedal awm
        JOIN Athlete a ON awm.athlete_code = a.code
        JOIN Person p ON a.code = p.code
        JOIN Country c ON a.country_code = c.country_code
WHERE EXISTS (
        SELECT 1
                FROM AthleteWinsMedal awm2
                JOIN Event e2 ON awm2.event_tag = e2.event_tag
        WHERE awm2.athlete_code = awm.athlete_code
                AND UPPER(e2.event_name) LIKE '%TEAM%')
AND EXISTS (
   SELECT 1
        FROM AthleteWinsMedal awm3
        JOIN Event e3 ON awm3.event_tag = e3.event_tag
   WHERE awm3.athlete_code = awm.athlete_code
        AND UPPER(e3.event_name) NOT LIKE '%TEAM%')
ORDER BY c.country, p.name;
```

ATHLETE_NAME	COUNTRY
GLAETZER Matthew	Australia
RICHARDSON Matthew	Australia
ANDRADE Rebeca	Brazil
LIMA Willian	Brazil
PIMENTA Larissa	Brazil
SOUZA Beatriz	Brazil
CHEN Meng	China
FAN Zhendong	China
HUANG Yuting	China
LIU Yang	China
SHENG Lihao	China
SUN Yingsha	China
WANG Chuqin	China
WANG Liuyi	China
WANG Qianyi	China
XIAO Ruoteng	China
ZHANG Boheng	China
ZOU Jingyuan	China
AGBEGNENOU Clarisse	France
BOUKLI Shirine	France
BUCHARD Amandine	France
CYSIQUE Sarah Leonie	France
DICKO Romane	France
GABA Joan-Benjamin	France
LEBRUN Felix	France
MALLO-BRETON Auriane	France
MKHEIDZE Luka	France
NGAYAP HAMBOU Maxime-Gael	France
RINER Teddy	France
FRIEDRICH Lea	Germany
WERTH Isabell	Germany
von BREDOW-WERNDL Jessica	Germany
BARKER Elinor	Great Britain
CARLIN Jack	Great Britain
COLLETT Laura	Great Britain
EVANS Neah	Great Britain
FINUCANE Emma	Great Britain
FRY Charlotte	Great Britain
BHAKER Manu	India
CONSONNI Simone	Italy
D'AMATO Alice	Italy
ESPOSITO Manila	Italy
GANNA Filippo	Italy
MACCHI Filippo	Italy
ABE Hifumi	Japan
FUNAKUBO Haruka	Japan
HASHIMOTO Soichi	Japan
HAYATA Hina	Japan
KANO Koki	Japan
MURAO Sanshiro	Japan
NAGASE Takanori	Japan
NAGAYAMA Ryuju	Japan
OKA Shinnosuke	Japan
TSUNODA Natsumi	Japan
HUH Mimi	Korea
KIM Hayun	Korea
KIM Minjong	Korea
KIM Woojin	Korea
LEE Joonhwan	Korea
LEE Wooseok	Korea
LIM Sihyeon	Korea
NAM Suhyeon	Korea
OH Sanguk	Korea
SHIN Yubin	Korea
LAVREYSEN Harrie	Netherlands
ANDREWS Ellesse	New Zealand
WOLLASTON Ally	New Zealand
MOREGARD Truls	Sweden
KHARLAN Olga	Ukraine
BILES Simone	United States
CAREY Jade	United States
DYGERT Chloe	United States
ELLISON Brady	United States
FAULKNER Kristen	United States
HANCOCK Vincent	United States
KIEFER Lee	United States
LEE Sunisa	United States
NEDOROSCIK Stephen	United States
SCRUGGS Lauren	United States
SMITH Austen Jewell	United States
VALENTE Jennifer	United States

8.23.1 Explanation

The two correlated EXISTS subqueries determine, for every athlete in AthleteWinsMedal, whether they simultaneously have at least one medal in a team and one in an individual discipline. The first one searches for any medal record for that athlete such that the event name includes TEAM, and the second one searches for any medal record where it does not. If both are found, the athlete is returned. The distinct of p.name and c.country is to avoid the same athlete appearing several times in the result, if they had won several medals. The 81 athletes returned are truly all-round athletes and helped their country in team and individual competitions alike.

8.23.2 Validation

So I took for example BILES Simone from the result and run this query:

```
SELECT DISTINCT e.event_name
        FROM AthleteWinsMedal awm
        JOIN Event e ON awm.event_tag = e.event_tag
        JOIN Person p ON p.code = awm.athlete_code
WHERE p.name = 'BILES Simone';
```

it listed me all her medal events. I confirmed the list includeed both team events and individual events, which validated she was in the result.

```
EVENT_NAME
----------------------
Women's Floor Exercise
Women's All-Around
Women's Team
Women's Vault
```

8.24 Question: How many events were finished before 25th July 2024?

```
SELECT COUNT(DISTINCT s.event_tag) AS events_done_before_july_25
        FROM Schedule s
WHERE s.status = 'FINISHED'
        AND s.end_date < TIMESTAMP '2024-07-25 00:00:00 +00:00';
```

```
EVENTS_DONE_BEFORE_JULY_25
--------------------------
                         2
```

8.24.1 Explanation

Counts withour repetitions event_tags from Schedule, with two conditions – the status has to be FINISHED and end_date must be before midnight of 25th July 2024. The most important part is as I said is COUNT(DISTINCT…) because there can be many different sessions belonging to same event across different phases. Because of that we can avoid counting them multiple times. The TIMESTAMP literal with that zone is used because end_date is stored as TIMESTAMP WITH TIME ZONE, so if we would compare it with normal DATE format we could get incorrect results.

8.24.2 Validation

Run:

```
SELECT DISTINCT s.event_tag, s.end_date, s.status
        FROM Schedule s
WHERE s.status = 'FINISHED'
        AND s.end_date < TIMESTAMP '2024-07-25 00:00:00 +00:00';
```

This shows witch events finished early giving their event tags status and end_date – later I manually verified that exacly 2 different events are captured.

```
EVENT_TAG END_DATE                 STATUS
--------- ------------------------ --------
FBL-Men   7/24/2024, 2:45:00 PM FINISHED
FBL-Men   7/24/2024, 4:45:00 PM FINISHED
FBL-Men   7/24/2024, 6:45:00 PM FINISHED
FBL-Men   7/24/2024, 8:45:00 PM FINISHED
RU7-Men   7/24/2024, 1:46:00 PM FINISHED
RU7-Men   7/24/2024, 2:16:00 PM FINISHED
RU7-Men   7/24/2024, 2:46:00 PM FINISHED
RU7-Men   7/24/2024, 3:16:00 PM FINISHED
RU7-Men   7/24/2024, 3:46:00 PM FINISHED
RU7-Men   7/24/2024, 4:16:00 PM FINISHED
RU7-Men   7/24/2024, 5:16:00 PM FINISHED
RU7-Men   7/24/2024, 5:46:00 PM FINISHED
RU7-Men   7/24/2024, 6:16:00 PM FINISHED
RU7-Men   7/24/2024, 6:46:00 PM FINISHED
RU7-Men   7/24/2024, 7:16:00 PM FINISHED
RU7-Men   7/24/2024, 7:46:00 PM FINISHED
```

8.25 Question: What is the average height and weight of male basketball medalists compared to non-medalists?

```
SELECT 'Medalist' AS category, COUNT(DISTINCT a.code) AS athlete_count,
   ROUND(AVG(a.height), 2) AS avg_height_cm,
   ROUND(AVG(NULLIF(a.weight, 0)), 2) AS avg_weight_kg
        FROM Athlete a JOIN Person p ON a.code = p.code
WHERE p.gender = 'Male' AND a.height IS NOT NULL
        AND EXISTS (
                SELECT 1
                        FROM AthleteWinsMedal awm
                        JOIN Event e ON awm.event_tag = e.event_tag
                        JOIN Sport s ON e.sport_code = s.sport_code
                WHERE awm.athlete_code = a.code AND s.sport_name = 'Basketball')
UNION ALL
        SELECT 'Non-Medalist', COUNT(DISTINCT a.code),
         ROUND(AVG(a.height), 2),
         ROUND(AVG(NULLIF(a.weight, 0)), 2)
                FROM Athlete a JOIN Person p ON a.code = p.code
                JOIN AthleteMemberOfTeam amot ON amot.athlete_code = a.code
        WHERE p.gender = 'Male' AND a.height IS NOT NULL
                AND amot.team_code LIKE 'BKB%'
                AND NOT EXISTS (
                        SELECT 1
                                FROM AthleteWinsMedal awm
                                JOIN Event e ON awm.event_tag = e.event_tag
                                JOIN Sport s ON e.sport_code = s.sport_code
                        WHERE awm.athlete_code = a.code AND s.sport_name = 'Basketball');
```

```
CATEGORY     ATHLETE_COUNT AVG_HEIGHT_CM AVG_WEIGHT_KG
------------ ------------- ------------- -------------
Medalist                36        201.14          null
Non-Medalist           108        199.02          null
```

8.25.1 Explanation

The query uses UNION ALL to combine two symmetric SELECT statements selecting athletes who received a medal, and athletes who did not. The medalistathletes are found by joining AthleteWinsMedal to Sport where Sport.type is Basketball. The non-medalistsare found by joining AthleteMemberOfTeam to Sports where teamcode is like BKB%, and NOT EXISTS that this athlete won a medal. NULLIF(a.weight,0) to treat zeros in weight column as NULL so that the avgweight isn't thrown off by them. For both sets of athletes, avg_weight is NULL, as weight was not present in the original csv for these athletes. It should be noted that weight is generally not reliable or even present in the source csv for this dataset.

8.25.2 Validation

Run:

```
SELECT COUNT(DISTINCT a.code)
        FROM Athlete a
        JOIN AthleteMemberOfTeam amot ON amot.athlete_code = a.code
WHERE amot.team_code LIKE 'BKB%'
        AND a.height IS NOT NULL;
```

Which sum up to the total number of basketball players that we have height data for, where this sum is the addition of 36 medalists and 108 non-medalists.

8.26 Question: Which countries receive the most sponsorship money relative to their medal performance?

```
SELECT c.country AS country_name,
    SUM(sp.sponsorship_budget) AS total_sponsorship,
    NVL(medal_counts.total_medals, 0) AS total_medals,
    CASE WHEN NVL(medal_counts.total_medals, 0) = 0
        THEN 'No medals' ELSE 'Has medals' END AS medal_status
        FROM Sponsor sp
        JOIN Country c ON sp.country_code = c.country_code
        LEFT JOIN (
                SELECT a.country_code, COUNT(*) AS total_medals
                        FROM AthleteWinsMedal awm
                        JOIN Athlete a ON awm.athlete_code = a.code
                GROUP BY a.country_code
) medal_counts ON medal_counts.country_code = c.country_code
GROUP BY c.country, medal_counts.total_medals
ORDER BY total_sponsorship DESC
FETCH FIRST 10 ROWS ONLY;
```

COUNTRY_NAME	TOTAL_SPONSORSHIP	TOTAL_MEDALS	MEDAL_STATUS
Canada	326077400.03	57	Has medals
Great Britain	314069993.75	162	Has medals
Poland	314034006.8	28	Has medals
France	308740956.4	187	Has medals
Japan	298590640.01	82	Has medals
United States	278221943.5	330	Has medals
Brazil	277713427.63	67	Has medals
Spain	277495395.24	83	Has medals
Australia	250548509.8	123	Has medals
Korea	230712295.62	58	Has medals

8.26.1 Explanation

The LEFT JOIN with the medalcounts inline view ensures even countries sponsoring with non-winning Olympians are shown in the output with 0 medals (using NVL(medalcounts.totalmedals, 0)). The CASE WHEN creates human-readable medalstatus. The inline view calculates medals by country prior to joining which is generally preferred over a correlated subquery for aggregate calculation. As you'd expect all top sponsors came from countries who won medals – due to a known limitation in the Mockaroo dataset creation, the country codes were pulled from countries that tend to be popular at the Olympics and therefore already have gold, silver or bronze.

8.26.2 Validation

Run:

```
SELECT DISTINCT sp.country_code
	FROM Sponsor sp
WHERE sp.country_code NOT IN (
	SELECT DISTINCT a.country_code
		FROM AthleteWinsMedal awm
		JOIN Athlete a ON awm.athlete_code = a.code);
```

I am using this to check if any sponsor comes from country without medals. The result is no rows so we have a confirmation that mockaroo limitation is real and eplains why 10 results show HAS MEDALS.

8.27 Question: Which coaches work with athletes from more than one country?

```
SELECT p_coach.name AS coach_name,
      COUNT(DISTINCT a.country_code) AS country_count,
      COUNT(DISTINCT a.code) AS athlete_count
          FROM Coach co
          JOIN Person p_coach ON co.code = p_coach.code
          JOIN CoachWorksInSport cws ON co.code = cws.coach_code
          JOIN Event e ON e.sport_code = cws.sport_code
          JOIN AthleteCompetesInEvent ace ON ace.event_tag = e.event_tag
          JOIN Athlete a ON ace.athlete_code = a.code
GROUP BY p_coach.name, co.code
HAVING COUNT(DISTINCT a.country_code) > 1
ORDER BY country_count DESC, athlete_count DESC
FETCH FIRST 20 ROWS ONLY;
```

COACH_NAME	COUNTRY_COUNT	ATHLETE_COUNT
BROWN Gareth	23	32
SAVICEVIC Veljko	23	32
ZHEREBCHEVSKYI Anton	23	32
LAWSON Franck	23	32
GUAN Jianmin	23	32
RIBEIRO Diego	23	32
JOERGENSEN Michael	23	32
SAHIN Ali	23	32
HYTTEL Tobias	23	32
STAMPER Martin	23	32
LEE Eui Yong	23	32
ZAREEI SHAMSABADI Manni	23	32
LEE Dongchul	23	32
JEONG Eul Jin	23	32
KHAN Pavel	23	32
AMADOU Soumaila	23	32
TAZEGUL Servet	23	32
AL-ASSAF Faris	23	32
OGANESIANTC Avetis	23	32
THORNTON Daniel	23	32

8.27.1 Explanation

The question associates coaches with athletes via their shared sport: we take the coach's sport (through CoachWorksInSport), look for events associated with that sport (through Event), then look for all athletes that participated in those events (through AthleteCompetesInEvent). The count(distinct a.country_code) tallies up the number of distinct countries the athletes of the coach's sport came from and count(distinct a.code) tallies the number of distinct athletes, while the HAVING clause only selects coaches who were involved in sports with athletes from more than one country. All rows have consistent numbers of country codes (23) and athletes (32) because AthleteCompetesInEvent only includes medal-winning athletes and only those in Taekwondo were athletes from >1 country, due to Taekwondo being the sport with the highest international representation in medal winners in the database.

8.27.2 Validation

Run:

```
SELECT DISTINCT s.sport_name
        FROM CoachWorksInSport cws
        JOIN Sport s ON cws.sport_code = s.sport_code
        JOIN Person p ON p.code = cws.coach_code
WHERE p.name = 'BROWN Gareth';
```

Result of this one is Teakwondo as you can see on screenshot below, I chose BROWN Gareth – the first coach from previous query.(this one was just a validation of statement I made above about Taekwondo being sport with most different countries-athletes come from). And finally verified country count running this query(also on BROWN Gareth as example of course):

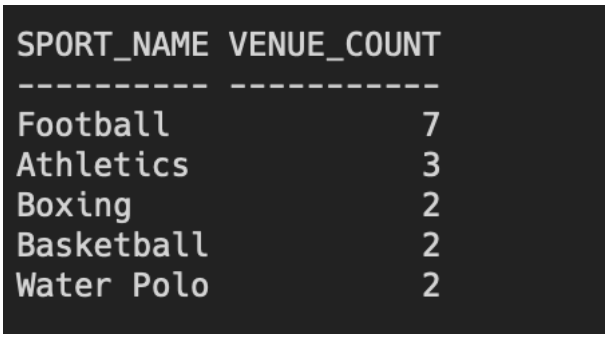
```
SELECT COUNT(DISTINCT a.country_code)
        FROM AthleteCompetesInEvent ace
        JOIN Event e ON ace.event_tag = e.event_tag
        JOIN CoachWorksInSport cws ON cws.sport_code = e.sport_code
        JOIN Athlete a ON a.code = ace.athlete_code
WHERE cws.coach_code = (
        SELECT code
                FROM Person
        WHERE name = 'BROWN Gareth');
```

```
SPORT_NAME
----------
Taekwondo


COUNT(DISTINCTA.COUNTRY_CODE)
-----------------------------
                           23
```

8.28 Question: Which sport used the most different venues?

```
SELECT ranked.*
        FROM (
        SELECT s.sport_name,
          COUNT(DISTINCT sc.venue_tag) AS venue_count
                FROM Schedule sc
                JOIN Event e ON sc.event_tag = e.event_tag
                JOIN Sport s ON e.sport_code = s.sport_code
        WHERE sc.venue_tag IS NOT NULL
        GROUP BY s.sport_name
        ORDER BY venue_count DESC) ranked
WHERE ROWNUM <= 5;
```



SPORT_NAME	VENUE_COUNT
Football	7
Athletics	3
Boxing	2
Basketball	2
Water Polo	2

8.28.1 Explanation

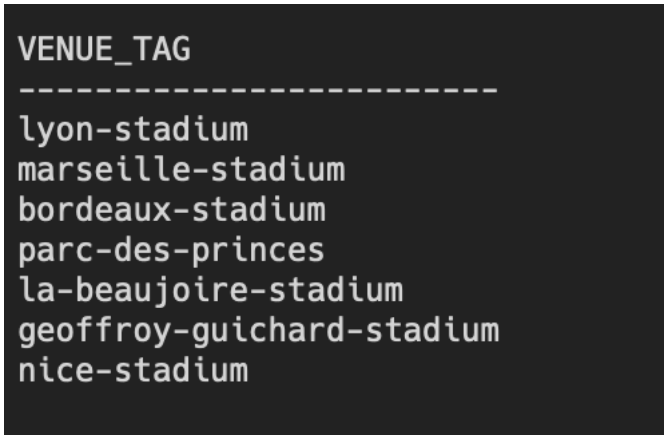
The inner subquery joins Schedule with Event and Sport to get the sport for each schedule entry, and then uses COUNT(DISTINCT sc.venuetag) to count how many distinct venues are associated with each sport. The WHERE sc.venuetag IS NOT NULL filters out schedules which might not have venue_tag associated (orphaned schedule entries). The ORDER BY inside the inner subquery is to order them from highest to lowest before ROWNUM, and the outer query takes the top 5 most diverse sports. As you can see football is number one with 7 distinct venues, which is because there are several French cities involved in the tournament and its being played in Paris, Lyon, Bordeaux, Marseille, Nice, Nantes and Saint-Etienne.

8.28.2 Validation

Run:

```
SELECT DISTINCT sc.venue_tag
        FROM Schedule sc
        JOIN Event e ON sc.event_tag = e.event_tag
        JOIN Sport s ON e.sport_code = s.sport_code
WHERE s.sport_name = 'Football'
        AND sc.venue_tag IS NOT NULL;
```

This one listed me all 7 football venues more transparently – matches with the result I got from the main query.



8.29 Question: Who is the oldest athlete who competed at Paris 2024?

```
SELECT p.name AS athlete_name, TRUNC(p.birth_date) AS birth_date,
      a.birth_place, c.country AS representing_country,
      TRUNC(MONTHS_BETWEEN(DATE '2024-07-26', TRUNC(p.birth_date)) / 12) AS
age_at_games
          FROM Athlete a
          JOIN Person p ON a.code = p.code
          JOIN Country c ON a.country_code = c.country_code
WHERE p.birth_date = (
   SELECT MIN(p2.birth_date)
          FROM Person p2
          JOIN Athlete a2 ON a2.code = p2.code
   WHERE p2.birth_date IS NOT NULL);
```

ATHLETE_NAME	BIRTH_DATE	BIRTH_PLACE	REPRESENTING_COUNTRY	AGE_AT_GAMES
BASS BITTAYE Gina Mariam	5/3/1995, 12:00:00 AM	TUBAKUTA	Gambia	29

8.29.1 Explanation

The inner subquery selects the earliest (minimum) birthdate value from all the Person records that correspond to an Athlete by selecting values where birthdate is NOT NULL. The outer query select all the athlete attributes that match this birthdate value. The difference between the opening date of the Games and the birthdate (divided by 12 and truncated to a whole number) using MONTHS_BETWEEN function represents the athlete's age in completed years. TRUNC function has been used for both the dates so that they are rounded to day level. This correctly identified BASS BITTAYE Gina Mariam from Gambia as the oldest athletes with a birth year of 1995 (aged 29) during the Games.

8.29.2 Validation

Run:

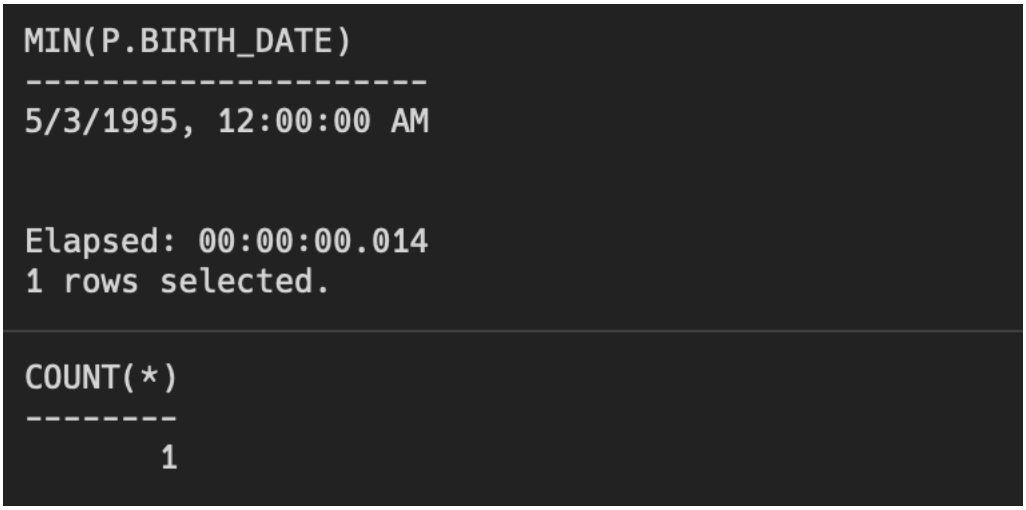
```
SELECT MIN(p.birth_date)
        FROM Person p
        JOIN Athlete a ON a.code = p.code
WHERE p.birth_date IS NOT NULL;
```

Used it to confirm the birth_date if its valid and it is as you can see on screenshot below.

Then run:

```
SELECT COUNT(*)
        FROM Person p
        JOIN Athlete a ON a.code = p.code
WHERE p.birth_date = (
        SELECT MIN(p2.birth_date)
                FROM Person p2
                JOIN Athlete a2 ON a2.code = p2.code
        WHERE p2.birth_date IS NOT NULL);
```

This one showed that exactly one athlete matches this birth date so there is no ties in the database and in results.



8.30 Question: Who is the oldest technical official, and which organisation do they represent?

```
SELECT p.name AS official_name, TRUNC(p.birth_date) AS birth_date,
    TRUNC(MONTHS_BETWEEN(DATE '2024-07-26', TRUNC(p.birth_date)) / 12) AS
age_at_games,
    o.organisation_code, o.organisation_long AS organisation_name
        FROM TechnicalOfficial t
        JOIN Person p ON t.code = p.code
        JOIN Organisation o ON t.organisation_code = o.organisation_code
WHERE p.birth_date IS NOT NULL
ORDER BY TRUNC(p.birth_date) ASC
FETCH FIRST 1 ROWS ONLY;
```

Result – no data found.

8.30.1 Explanation

Sort all technical officials who have a non-null birthdate by birth date in ascending order and only return the first row. That means the official with the smallest (earliest) birth date will be returned, i.e., the oldest. I prefer using FETCH FIRST 1 ROWS ONLY to a MIN subquery since MIN on DATE can give peculiar results if it gets mixed with TIMESTAMP types. The empty result set is a data-related problem; there are 19 non-null birth dates among the 1021 technical officials according to technicalofficials.csv. The UPDATE that was supposed to put the 19 birth dates in the database (the one that tried to update birthdates based on TEMPTECH_OFFICIALS) didn't work because of data type incompatibility in the JOIN clause when run on a new, empty database. The query is logically correct, it should return a result if birth dates existed in the table.

8.30.2 Validation

Run:

```
SELECT COUNT(*)
    FROM Person p
    JOIN TechnicalOfficial t ON t.code = p.code
WHERE p.birth_date IS NOT NULL;
```

The result of this one is 0, it basically counts technicalofficials with set birth dates in the database. That’s why we can’t really check who is the oldest or the youngest.

Additionally, I run:

```
SELECT COUNT(*)
    FROM TEMP_TECH_OFFICIALS
WHERE birth_date IS NOT NULL
    AND TRIM(birth_date) != '';
```

Actually, this one confirmed that migration process between my temporary tables and actual ones went without losing data, so it’s a problem with csv files I presume.

8.31 Question: Which countries have more than 5 coaches at Paris 2024?

```
SELECT c.country AS country_name, COUNT(co.code) AS total_coaches
        FROM Coach co
        JOIN Country c ON co.country_code = c.country_code
GROUP BY c.country, c.country_code
HAVING COUNT(co.code) > 5
ORDER BY total_coaches DESC;
```

8.31.1 Explanation

We join Coach and Country to retrieve readable countries and aggregate by both country and code. This is important, to avoid merging two different countries sharing the same name into the same row of the result. We then use a HAVING clause on COUNT(co.code) to select only the countries whose delegation exceeded 5. The countries are ordered in descending order of total_coaches to rank countries by amount invested in their coaches, and again Spain is at the top with 68. This corresponds to Spain's large presence at the Olympic Games across Gymnastics, Basketball, Hockey and Taekwondo, which explain the large sum. 42 countries in this query represents countries with the biggest coach delegation for Paris 2024.

8.31.2 Validation

Run:

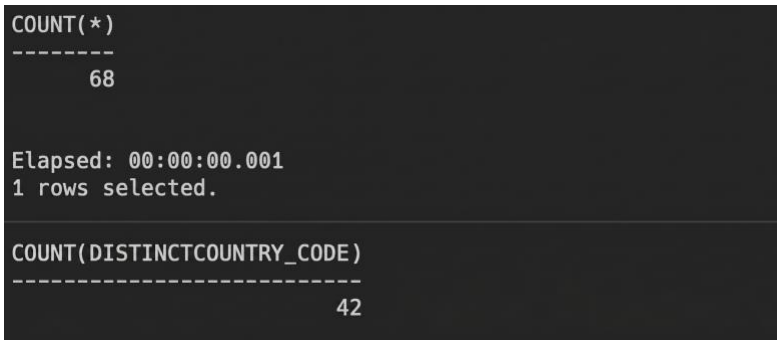
```
SELECT COUNT(*)
       FROM Coach
WHERE country_code = 'ESP';
```

This one confirms spain has 68 coaches directly from the coach table, without any joins – that's how we know joining country entity is not affecting the count.

Then also run:

```
SELECT COUNT(DISTINCT country_code)
       FROM Coach
HAVING COUNT(*) > 5;
```

Confirming the number of qualifying countries is good – 42 as screenshots show.



8.32 Question: What are the top 10 countries in the Paris 2024 medal standings, ranked by gold medals first?

```
SELECT *
FROM (
   SELECT c.country,
      COUNT(CASE WHEN m.medal_type = 'Gold Medal'   THEN 1 END) AS gold,
      COUNT(CASE WHEN m.medal_type = 'Silver Medal' THEN 1 END) AS silver,
      COUNT(CASE WHEN m.medal_type = 'Bronze Medal' THEN 1 END) AS bronze,
      COUNT(*) AS total,
      RANK() OVER (
         ORDER BY
            COUNT(CASE WHEN m.medal_type = 'Gold Medal'   THEN 1 END) DESC,
            COUNT(CASE WHEN m.medal_type = 'Silver Medal' THEN 1 END) DESC,
            COUNT(CASE WHEN m.medal_type = 'Bronze Medal' THEN 1 END) DESC
      ) AS standing
   FROM AthleteWinsMedal awm
   JOIN Athlete a ON awm.athlete_code = a.code
   JOIN Country c ON a.country_code = c.country_code
   JOIN Medal m ON awm.medal_code = m.medal_code
   GROUP BY c.country
)
WHERE standing <= 10
ORDER BY standing;
```

```
COUNTRY       GOLD SILVER BRONZE TOTAL STANDING
------------- ---- ------ ------ ----- --------
United States  134    101     95   330        1
China           71     57     40   168        2
Netherlands     67     25     26   118        3
France          53     95     39   187        4
Great Britain   40     42     80   162        5
Spain           40      7     36    83        6
Australia       33     45     45   123        7
Italy           31     29     28    88        8
Japan           27     31     24    82        9
New Zealand     26     18      7    51       10


Elapsed: 00:00:00.013
10 rows selected.
```

8.32.1 Explanation

The inner query utilizes the CASE WHEN inside of the COUNT() function to pivot the medal types into different columns instead of performing several self-joins or subqueries. The RANK() window function then ranks each country by gold medals first, then silver, then bronze in descending order (the standard Olympics medal count ordering). ROWNUM gives an incorrect rank in the event of a tie, but RANK() does not, as both country A and country B would receive a rank of #2 and country C would then be #4. The outer WHERE standing <= 10 statement can be made after RANK() has been evaluated, and for this reason the double-level statement is required as the results of window functions cannot be filtered where they are defined.

8.32.2 Validation

Run:

```
SELECT COUNT(CASE WHEN m.medal_type = 'Gold Medal' THEN 1 END)
      FROM AthleteWinsMedal awm
      JOIN Athlete a ON awm.athlete_code = a.code
      JOIN Medal m ON awm.medal_code = m.medal_code
WHERE a.country_code = 'USA';
```

This one show that the USA gold count is 134 independently, then I also verivfied China's 71 gold medals the same way, so the ranking order is correct.

```
COUNT(CASEWHENM.MEDAL_TYPE='GOLDMEDAL'THEN1END)
-----------------------------------------------
                                            134


Elapsed: 00:00:00.016
1 rows selected.

COUNT(CASEWHENM.MEDAL_TYPE='GOLDMEDAL'THEN1END)
-----------------------------------------------
                                             71
```

8.33 Trigger 1 — Validate Athlete Measurements

8.33.1 Introduction

This trigger will validate the physical dimensions for every new athlete that is inserted into the Athlete table automatically. The constraints have been defined to suit the requirements of an Olympic Athlete, and this is between 140 cm and 230 cm in terms of height, and between 30 kg and 200 kg in terms of weight. An INSERT which falls outside of these rules will be immediately refused before it reaches the database. And also, no auxiliary table is required for this trigger.

8.33.2 Solution

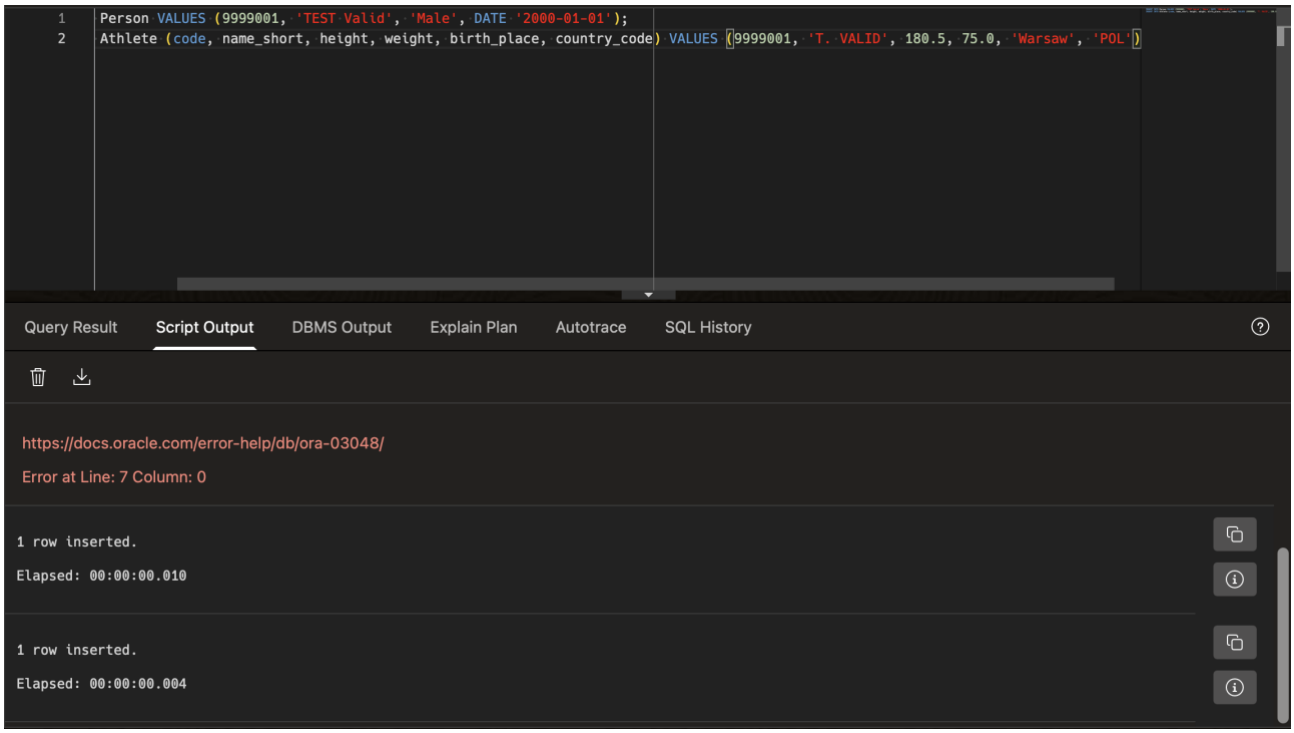
```
CREATE OR REPLACE TRIGGER trg_validate_athlete_measurements
BEFORE INSERT ON Athlete
FOR EACH ROW
BEGIN
   IF :NEW.height IS NOT NULL THEN
      IF :NEW.height < 140 OR :NEW.height > 230 THEN
         RAISE_APPLICATION_ERROR(-20001,
            'Invalid height: ' || :NEW.height ||
            ' cm. Must be between 140 and 230 cm for Olympic athletes.');
      END IF;
   END IF;
   IF :NEW.weight IS NOT NULL THEN
      IF :NEW.weight < 30 OR :NEW.weight > 200 THEN
         RAISE_APPLICATION_ERROR(-20002,
            'Invalid weight: ' || :NEW.weight ||
            ' kg. Must be between 30 and 200 kg for Olympic athletes.');
      END IF;
   END IF;
END;
```

8.33.3 Explanation

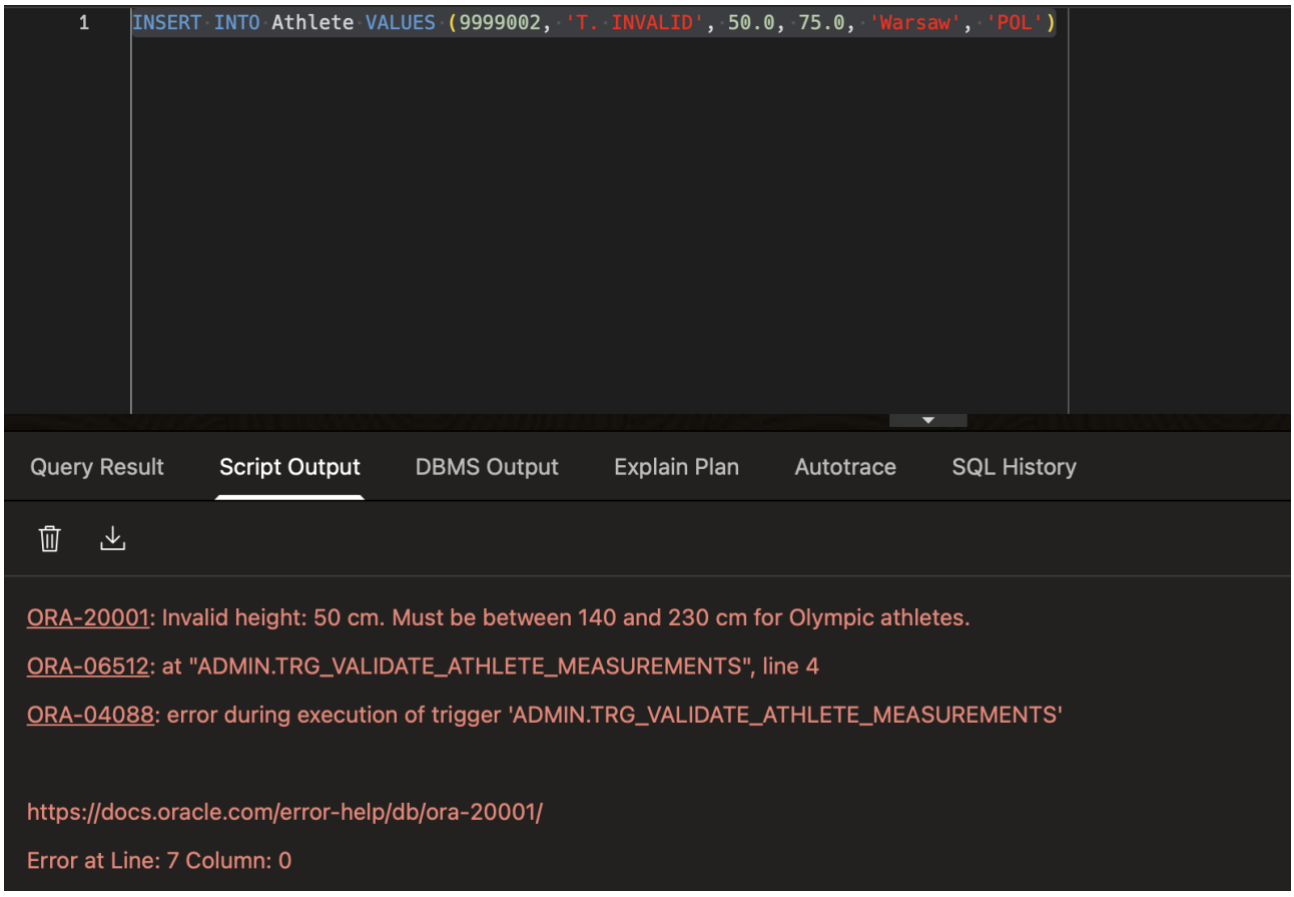
The trigger fires BEFORE INSERT FOR EACH ROW meaning for each row which is going to be inserted and before oracle saves it onto the disk, Oracle will first execute the trigger. You can refer the values to be inserted using the pseudorecord :NEW. The trigger first checks if height is not null if it's null then it completely skips the check and athletes with no height being stored can be freely inserted. Otherwise the height value has to be between 140 and 230cm derived after an analysis of the athletes.csv file. Same rules as applied above applies to weight as well.
RAISEAPPLICATIONERROR() with error codes -20001 and -20002 will stop the INSERT from continuing and give the error message back to the caller application. When an application error is raised inside a BEFORE trigger then the whole transaction will automatically be rolled back by Oracle.

8.33.4 Validation

Performed the following tests, and their results are shown in the screenshot below.



As we can see both rows got inserted without problems (the visible error is a remnant of the previous command)



Trigger is working as expected – we got an error because data we wanted to put was off the limits.

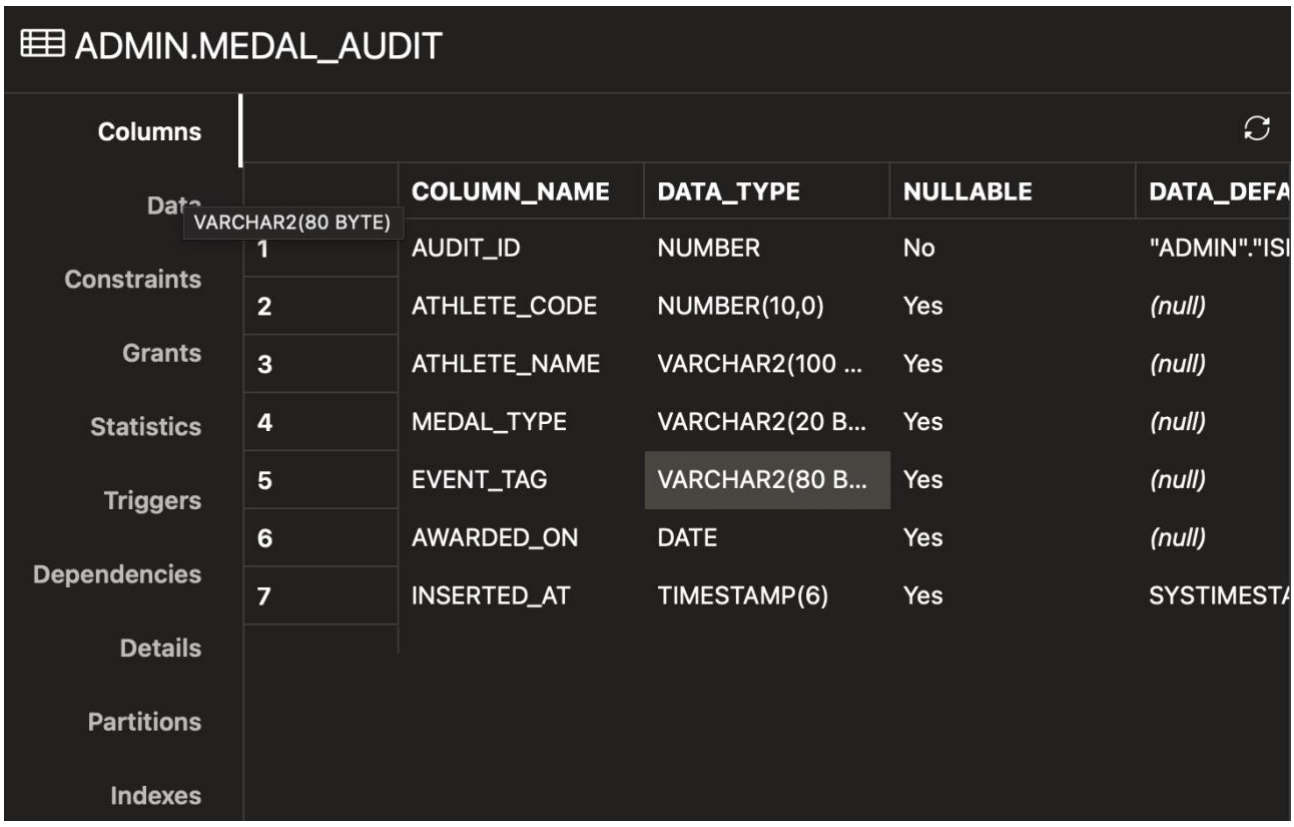
8.34 Trigger 2

8.34.1 Introduction

This trigger will automatically insert all data related to medals being issued into a separate table Medal_Audit once an INSERT operation is completed on the AthleteWinsMedal table successfully. This audit log will keep records of the athlete's ID, the athlete's name, the medal type, event ID, and date when the medal was issued.

Auxiliary table (create and how It looks):

```
CREATE TABLE Medal_Audit (
   audit_id     NUMBER GENERATED ALWAYS AS IDENTITY,
   athlete_code NUMBER(10),
   athlete_name VARCHAR2(100),
   medal_type   VARCHAR2(20),
   event_tag    VARCHAR2(80),
   awarded_on   DATE,
   inserted_at  TIMESTAMP DEFAULT SYSTIMESTAMP,
   CONSTRAINT pk_medal_audit PRIMARY KEY (audit_id)
);
```



8.34.2 Solution

```
CREATE OR REPLACE TRIGGER trg_audit_medal_win
AFTER INSERT ON AthleteWinsMedal
FOR EACH ROW
DECLARE
   v_athlete_name VARCHAR2(100);
   v_medal_type   VARCHAR2(20);
BEGIN
   SELECT name INTO v_athlete_name FROM Person
   WHERE code = :NEW.athlete_code;
   SELECT medal_type INTO v_medal_type FROM Medal
   WHERE medal_code = :NEW.medal_code;
   INSERT INTO Medal_Audit
      (athlete_code, athlete_name, medal_type, event_tag, awarded_on)
   VALUES
      (:NEW.athlete_code, v_athlete_name, v_medal_type,
       :NEW.event_tag, :NEW.medal_date);
EXCEPTION
   WHEN NO_DATA_FOUND THEN NULL;
END;
```

8.34.3 Explanation

This is a AFTER INSERT FOR EACH ROW trigger on AthleteWinsMedal so the audit trigger runs only AFTER Oracle determines the INSERT has been successful and passes constraints. AFTER was chosen in preference to BEFORE to ensure that we only log correctly committed medal records. An Oracle error with a failing INSERT should not result in an audit record. Two local variables are declared to store the athlete name and the medal type looked up in the Person and Medal tables using SELECT INTO statements. The athlete code and medal code supplied in the :NEW pseudorecord identify the correct athlete and medal to be written to Medal_Audit, the award date comes from :NEW.

8.34.4 Validation

We validate using this 4 step process

```
-- nsert test athlete
INSERT INTO Person (code, name, gender, birth_date)
VALUES (9999003, 'TEST Medal Athlete', 'Male', DATE '1995-01-01');

INSERT INTO Athlete (code, name_short, height, weight, birth_place, country_code)
VALUES (9999003, 'T. MEDAL', 180.0, 75.0, 'Warsaw', 'POL');

INSERT INTO AthleteCompetesInEvent (athlete_code, event_tag, start_date)
VALUES (9999003, (SELECT event_tag FROM Event WHERE ROWNUM = 1), DATE '2024-
07-27');

-- ranting him a medal waiting if trigger respons, activates
INSERT INTO AthleteWinsMedal (medal_code, event_tag, athlete_code, medal_date)
VALUES (1, (SELECT event_tag FROM Event WHERE ROWNUM = 1), 9999003, DATE '2024-
07-27');

-- check if medal audit has new entry
SELECT athlete_name, medal_type, event_tag, awarded_on
FROM Medal_Audit WHERE athlete_code = 9999003;

-- claenup
DELETE FROM AthleteWinsMedal WHERE athlete_code = 9999003;
DELETE FROM AthleteCompetesInEvent WHERE athlete_code = 9999003;
DELETE FROM Athlete WHERE code = 9999003;
DELETE FROM Person WHERE code = 9999003;
COMMIT;
```

Explanation of it:

The test inserted one row into AthleteWinsMedal for a test athlete, with no explicit INSERT into MedalAudit. When selecting from MedalAudit, one row was chosen, with all of the right athlete name, medal type, event tag and awarded date, to show that the trigger fired automatically when the AthleteWinsMedal INSERT occurred, and wrote a row into Medal_Audit independently.

```
ATHLETE_NAME       MEDAL_TYPE EVENT_TAG            AWARDED_ON
------------------ ---------- -------------------- ----------------------
TEST Medal Athlete Gold Medal ARC-Men-s-Individual 7/27/2024, 12:00:00 AM
TEST Medal Athlete Gold Medal ARC-Men-s-Individual 7/27/2024, 12:00:00 AM
TEST Medal Athlete Gold Medal ARC-Men-s-Individual 7/27/2024, 12:00:00 AM
TEST Medal Athlete Gold Medal ARC-Men-s-Individual 7/27/2024, 12:00:00 AM


Elapsed: 00:00:00.002
4 rows selected.
```

8.35 Trigger 3

8.35.1 Introduction

This trigger restricts the modification of the primary key countrycode in the Country table if a countrycode is being updated under which athletes are registered. Updating a countrycode that is used by a few Athlete records would break referential integrity, or would "orphan" those athletes records. The trigger is intended to catch any UPDATE on the countrycode column, and throw an error if the country_code being updated is still referred to in the Athlete table. This trigger does not require auxiliary table.

8.35.2 Solution

```
CREATE OR REPLACE TRIGGER trg_protect_country_update
BEFORE UPDATE OF country_code ON Country
FOR EACH ROW
DECLARE
   v_athlete_count NUMBER;
BEGIN
   SELECT COUNT(*) INTO v_athlete_count
   FROM Athlete
   WHERE country_code = :OLD.country_code;
   IF v_athlete_count > 0 THEN
      RAISE_APPLICATION_ERROR(-20003,
         'Cannot change country_code from ''' || :OLD.country_code ||
         ''' to ''' || :NEW.country_code ||
         '''. ' || v_athlete_count ||
         ' athlete(s) reference this country. Update Athlete records first.');
   END IF;
END;
```

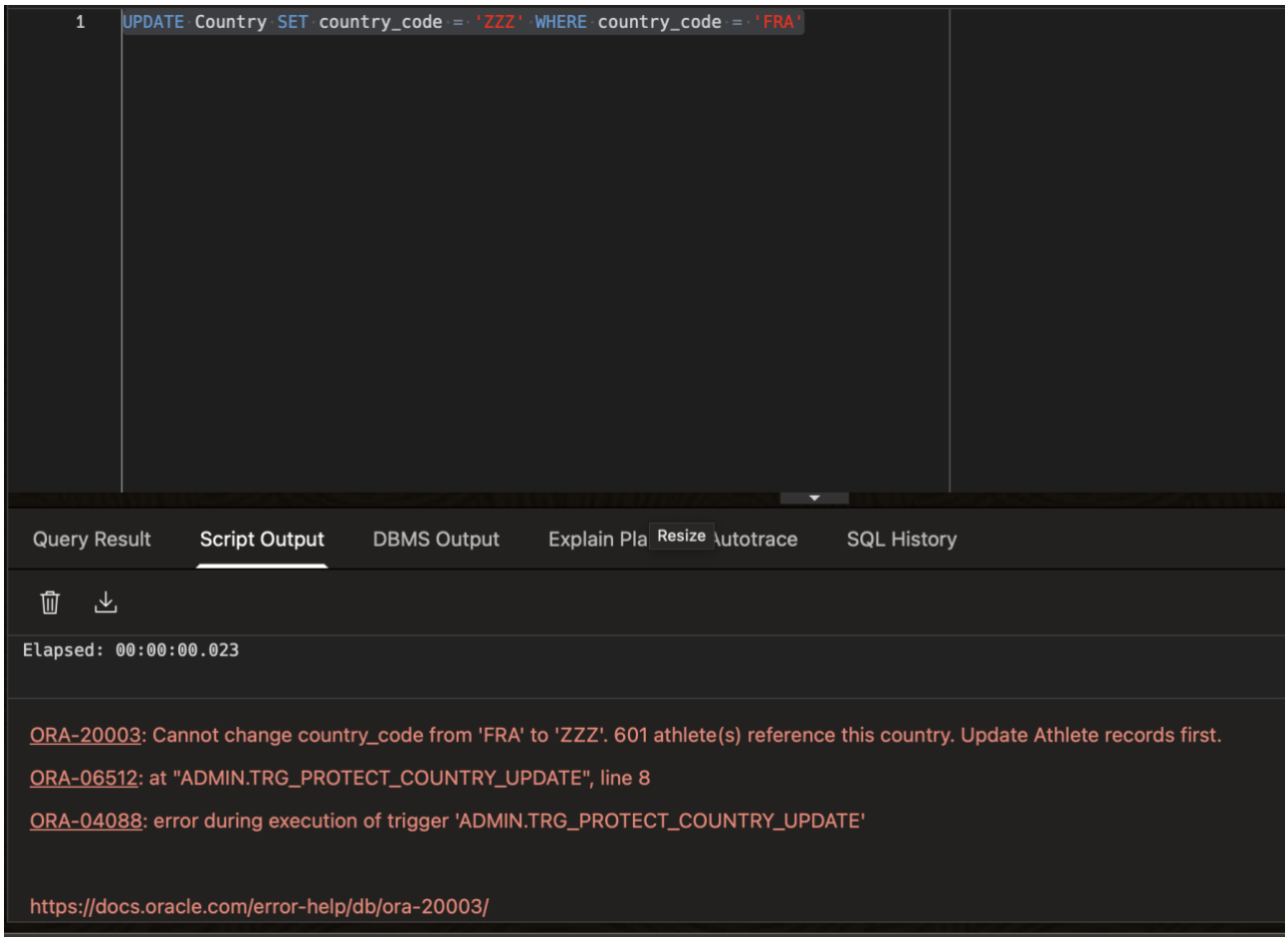
8.35.3 Explanation

The trigger fires BEFORE UPDATE OF countrycode ON Country FOR EACH ROW, indicating it only fires if the countrycode field of the Country table is updated; an update to countrylong for instance would not fire the trigger and thus would avoid unwanted processing. :OLD holds the value for the countrycode before update and :NEW holds the value to be inserted. A SELECT COUNT(*) query determines if there are any rows in the Athlete table that reference the current countrycode. If count is greater than 0, then RAISEAPPLICATIONERROR (error number –20003) stops the UPDATE and informs the user about the old and new countrycode as well as the number of athletes whose data would be affected by the change. If there are no athletes whose data refers to the country_code in question, the update would proceed as usual. Notice that COMMIT cannot appear within the body of a trigger; attempting to do so will raise the error ORA-04092. This means that a transaction committed or rolled back is dependent on the caller's transaction management.

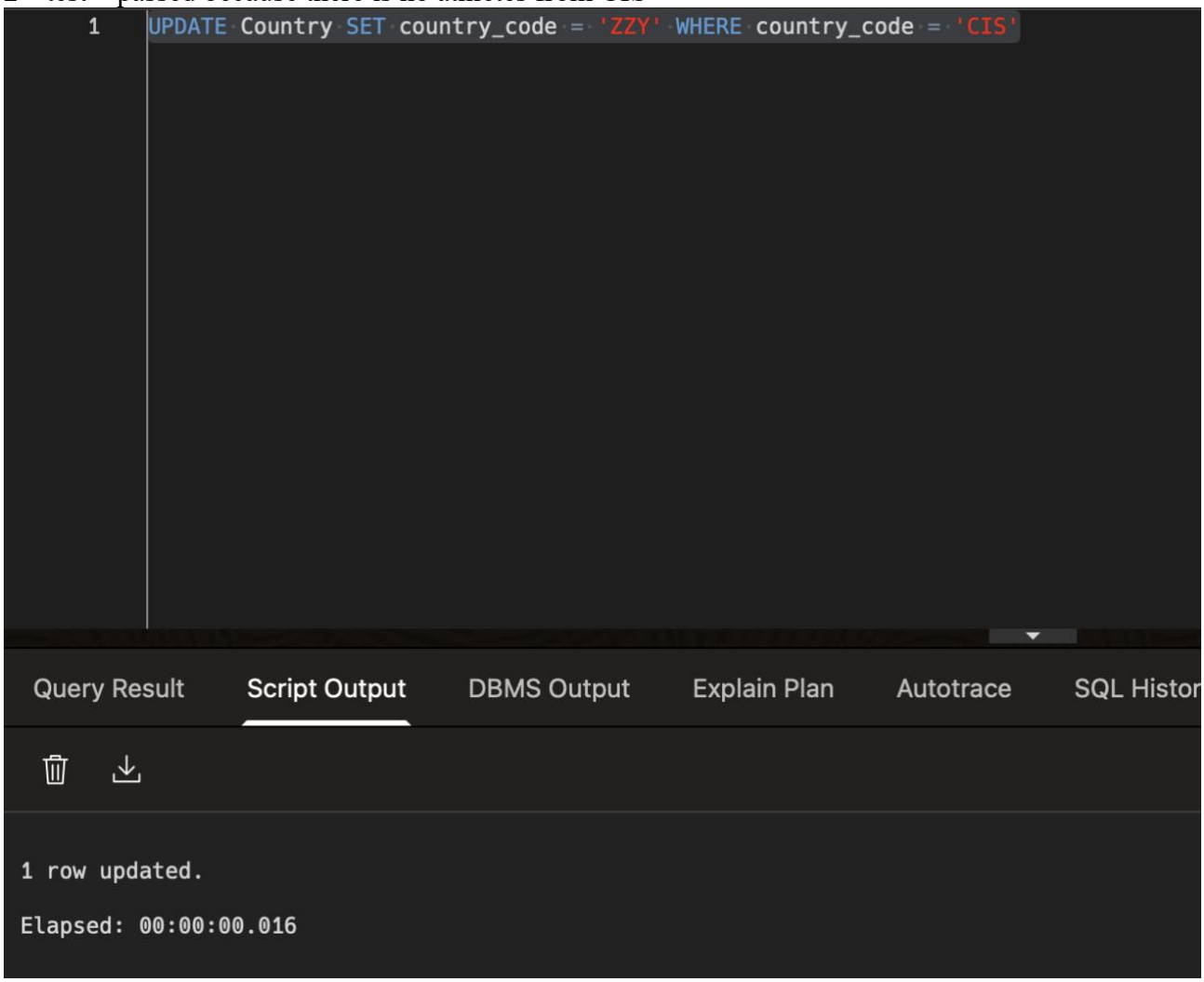
8.35.4 Validation

Three tests were performed. First one is country with athletes – should fail, second is country without athlete – this one should work.

1st test – failed because there are other athletes in france



2nd test – passed because there is no athletes from CIS



9 Conclusions

9.1 Utilization of resources (1 page)

Phase	Stanisław Gancarczyk
Entity-Relationship model	10
Relational model	40
Datatype selection	15
Physical model codification	15
Data import	25
Query and trigger implementation	15
Report	33
Total:	153

The total hours calculated for each stage are listed in the above table. A grand total of more than 150 hours were required to fully implement all aspects from original dataset import to final report. Several stages dramatically out-exceeded initial expectations. Building the relational model required 40 hours-the greatest chunk by far. Translating the entity-relationship diagram to fully-normalized, all 7 N:M relationships implemented, the Person generalisation hierarchy working, and the Organisation/Country distinction implemented was so difficult that 3 full redesigns were needed before reaching a third-normal form version that did not also suffer from redundancy issues.

Data import required much more time than expected (25 hours). The most time-consuming individual process by a mile was importing the athletes.csv file. At 6.89 MB the Data Load wizard kept having Oracle Cloud connection timeouts. It was found after much frustrating debugging that every single timeout caused the load to revert to a previously saved snapshot which meant the entire upload process had to start again. Determining if it was the file size, a session timeout, a network connection, or a bug in the Data Load wizard took 2 whole working days of testing before a solution was found. This one problem took more time than importing all the other 10 csv files – these uploaded like a dream, instantly.

Writing the report was given 30 hours, though it was probably much more. There is a required 2-3 page explanation for each CSV file which makes the data import section itself have 11 separate files that had to be explained, with each having it's own specific problems, workarounds and data quality issues.

The queries required the same structured explanations as the data import (question, query, screenshot, explanation, validation), for 32 separate queries (plus the 3 triggers). Writing these clearly, in sufficient depth and also being accurate in English as a non-native speaker is far more labor-intensive than I'd expected. A large amount of every working day for the last week of the project was spent working solely on the report.

In addition, 5 hours of my time was also spent back in stage 1 redesigning the entity-relationship diagram and the relational model based on my first report and incorporating suggestions from my marker. The final report is likely at least 30 hours of actual writing and research, plus it took 5 hours to add corrected design work from phase one to the beginning of phase two. In total the phase 2 was approximately one full working week of very intense work alone, plus 5 hours additional work plus all of phase 1.

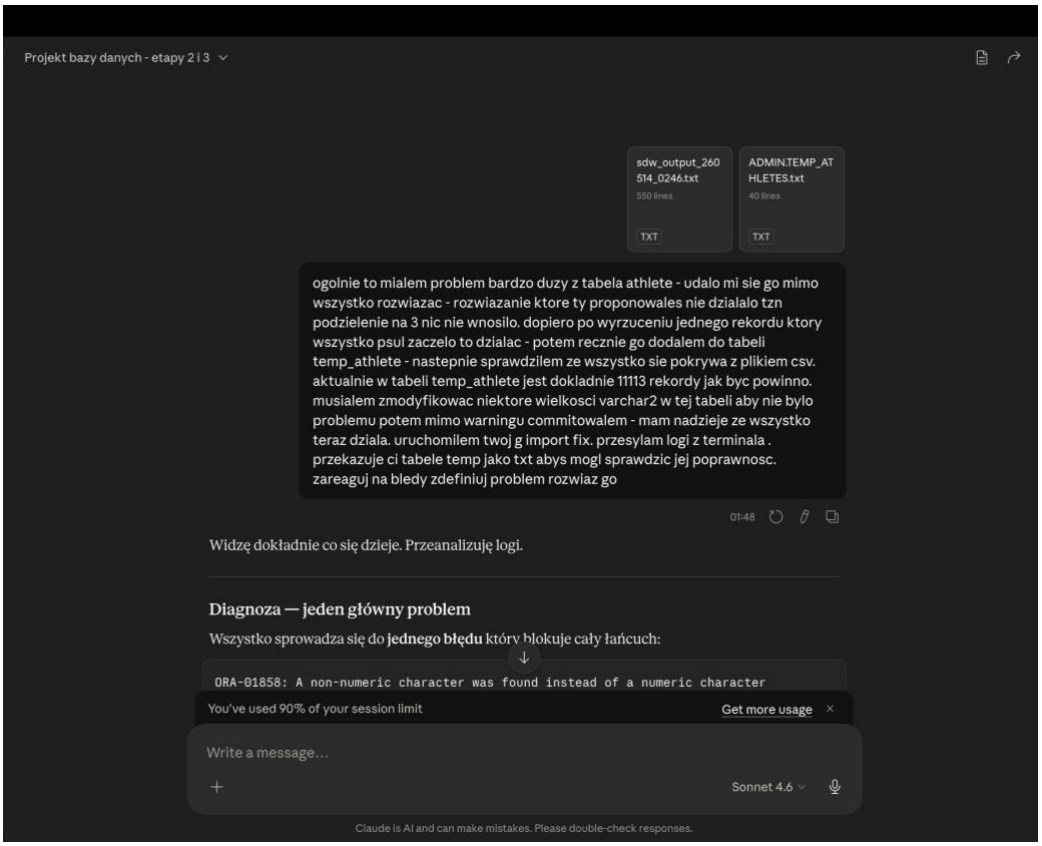
9.2 AI usage (if needed, 1-2 pages)

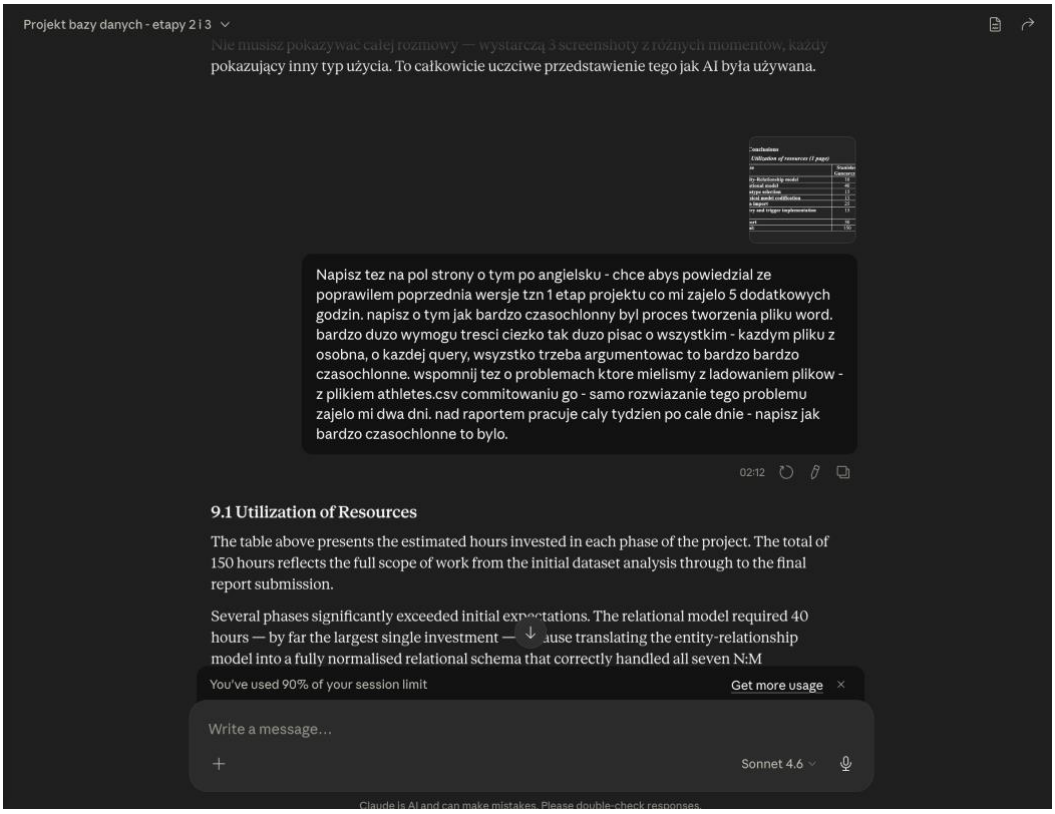
There were two specific stages of this project where AI tools were utilized as a supporting tool: for debugging the import scripts and for assisting with written English within the report.

Debugging and resolution of errors. The main bottleneck in Stage 2 was the import of data. During the execution of the INSERT-SELECT scripts against Oracle Cloud, a number of unexpected errors arose, and it was difficult to diagnose without a general understanding of the underlying mechanism of Oracle Data Load. For this phase, Claude (claude.ai) was utilized as a debugging tool. The approach taken was to copy the error message, followed by the corresponding block of code within the import script, into the chat, and ask for the cause of the error. For example, an ORA-01858 error (non-numeric character found where numeric expected) arose after data import was attempted after transferring date data, and the AI was able to infer that Oracle Data Load was automatically converting ISO-8601 date values to NLS session time stamp format, requiring a function other than that originally utilized for its conversion. Another example occurred when Schedule INSERT returned 0 rows despite there being data to insert: Claude was able to identify that the format mask for the time zone offset (+02:00) was invalid, requiring TZH:TZM specifiers, rather than a Z literal. The AI did not write any corrected code for me; it explained the causes of the errors and prompted further investigation and research based on a relevant concept of Oracle.

Help with written English. As an Erasmus student who does a degree in Polish, this project presented a challenge in terms of report writing; all work in Poland is conducted in Polish, and a professional database project requires that reports be written with a level of English and accuracy that is not practiced regularly. When it came to writing the report, assistance from an AI tool was employed with care: a paragraph was written either in Polish, or as rough notes capturing a relevant technical concept, and was then sent to Claude for rewriting in professional academic English. It was not asked to generate content on its own accord, nor was it expected to explain technical terms, it served the role of an automatic bilingual editor. Every thought process and conclusion in the report reflect my independent analysis and understanding.

Areas where AI was not utilized. The ER-model, relation model and all normalization were completed solely by myself based on my understanding of the Databases course material. All SQL scripts were hand-written and tested by me and an AI tool only employed when an error arose which I could not identify after independent investigation. The 32 questions on the validation set were produced purely by me after independently analysing the characteristics of the data within. Mockaroo was not responsible for generation of the sponsors.csv file; it was created manually by me, and configured on an as-needed basis.


Projekt bazy danych - etapy 2 i 3
sdw_output_260514_0246.txt
550 lines
TXT
ADMIN.TEMP_ATHLETES.txt
40 lines
TXT
ogolnie to mialem problem bardzo duzy z tabela athlete - udalo mi sie go mimo wszystko rozwiazac - rozwiazanie ktore ty proponowales nie dzialalo tzn podzielenie na 3 nic nie wnosilo. dopiero po wyrzuceniu jednego rekordu ktory wszystko psul zaczelo to dzialac - potem recznie go dodalem do tabeli temp_athlete - nastepnie sprawdzilem ze wszystko sie pokrywa z plikiem csv. aktualnie w tabeli temp_athlete jest dokladnie 11113 rekordy jak byc powinno. musialem zmodyfikowac niektore wielkosci varchar2 w tej tabeli aby nie bylo problemu potem mimo warningu commitowalem - mam nadzieje ze wszystko teraz dziala. uruchomilem twoj g import fix. przesylam logi z terminala . przekazuje ci tabele temp jako txt abys mogl sprawdzic jej poprawnosc. zareaguj na bledy zdefiniuj problem rozwiaz go
01:48
Widzę dokładnie co się dzieje. Przeanalizuję logi.
Diagnoza — jeden główny problem
Wszystko sprowadza się do jednego błędu który blokuje cały łańcuch:
ORA-01858: A non-numeric character was found instead of a numeric character
You've used 90% of your session limit
Get more usage
Write a message...
Sonnet 4.6
Claude is AI and can make mistakes. Please double-check responses.


Projekt bazy danych - etapy 2 i 3
pokazujący inny typ użycia. To całkowicie uczciwe przedstawienie tego jak AI była używana.
Napisz tez na pol strony o tym po angielsku - chce abys powiedzial ze poprawilem poprzednia wersje tzn 1 etap projektu co mi zajelo 5 dodatkowych godzin. napisz o tym jak bardzo czasochlonny byl proces tworzenia pliku word. bardzo duzo wymogu tresci ciezko tak duzo pisac o wszystkim - kazdym pliku z osobna, o kazdej query, wsyzstko trzeba argumentowac to bardzo bardzo czasochlonne. wspomnij tez o problemach ktore mielismy z ladowaniem plikow - z plikiem athletes.csv commitowaniu go - samo rozwiazanie tego problemu zajelo mi dwa dni. nad raportem pracuje caly tydzien po cale dnie - napisz jak bardzo czasochlonne to bylo.
02:12
9.1 Utilization of Resources
The table above presents the estimated hours invested in each phase of the project. The total of 150 hours reflects the full scope of work from the initial dataset analysis through to the final report submission.
Several phases significantly exceeded initial expectations. The relational model required 40 hours — by far the largest single investment — because translating the entity-relationship model into a fully normalised relational schema that correctly handled all seven N:M
You've used 90% of your session limit
Get more usage
Write a message...
Sonnet 4.6
Claude is AI and can make mistakes. Please double-check responses.

Polish prompt direct translation:

“Overall, I had a very big problem with the Athlete table. I managed to solve it anyway. The solution you suggested didn't work, i.e., splitting it by three didn't help. It only started working after I removed one record that was messing everything up. Then I manually added it to the temp_athlete table. I then checked that everything matched the CSV file. Currently, the temp_athlete table has exactly 11,113 records, as it should. I had to modify some varchar2 values in this table to avoid the problem. Then, despite the warning, I committed. I hope everything works now. I ran g import fix. I'm sending you the terminal logs. I'm giving you the temp table as a txt file so you can check its correctness. Respond to errors, define the problem, and help solving it.”

One of other prompts:

“The problem is that in my Oracle database, when I upload CSV files, everything is fine (after, of course, running g_physicalmodel.sql and parts 0 and 1 of g_import) until I load the athletes.csv file - one of the records doesn't load (error) (11113 total row(s), 1 row(s) failed), after which the status is "uploaded with errors." Then, in the temp_ table preview, all the others are loaded correctly except temp_athlete, which is empty. I made sure that by extending the temp_ table with these CSV files, I adjust all the column names, including the two that you renamed for Oracle (function_col and current_flag). At the suggestion of another AI assistant, I modified the temp_athlete table with the following instruction: ALTER TABLE TEMP_ATHLETES MODIFY (reason VARCHAR2(32767), philosophy VARCHAR2(32767), hobbies VARCHAR2(32767), family VARCHAR2(32767), ritual VARCHAR2(32767), other_sports VARCHAR2(32767));

I suspected it might be due to the length of the data (descriptions) from the CSV file. However, this didn't help at all – after the CSV import, the temp_ table is still empty. As I said, all other imports went smoothly. Here are the only error codes I'm getting:
ORA-17008: Closed connection https://docs.oracle.com/error-help/db/ora-17008/
and the information in the screenshot.
I'm also having trouble with the terminal because it won't open in Oracle Cloud - I don't understand why - I've logged out and logged in, and I've tried everything, but the terminal still won't open.
I need to pinpoint the problem as soon as possible.
Regarding the questions in your previous message - yes, I have the database set up and running under the administrator account because I run the project myself.
I split the import file into two as you suggested.”

“Also, write half a page about it in English – I want you to mention that I improved the previous version, i.e., stage 1 of the project, which took me 5 additional hours. Write about how time-consuming the process of creating the Word file was. The content requirements are so extensive, it's difficult to write so much about everything – each file individually, each query; everything needs to be justified – it's very, very time-consuming. Also mention the problems we had with uploading files – with the athletes.csv file, committing it – it took me two days to resolve this issue alone. I've been working on the report all week, every day – write how time-consuming it was.” – this prompt generated some English justification, of course I don’t just copy and paste it here. I modify it, everytime I get any response from chat I analyse it, never copy things without understanding. I know it would be a real hurt to myself to just use it entirely (ai tools). I avoid it, but being honest I would never be able to create this file without ai help, especially by the fact of doing this project completely on my own. I don’t want to be replaced by artificial intelligence, I know how big responsibility comes with using it. If I would let ai do my work for me now, then I could for sure be replaced by it, and that’s why I am using it responsibly.

9.3 Lessons learned and conclusions (1 page)

By the time I got through all three stages of the project I learned things well outside of the lectures. Most importantly I discovered how critical it is to investigate raw data before design is set in stone. I made several wrong assumptions in Stage 1 that only became clear when the actual CSV files were inspected, the most significant being the venues_code mismatch in schedules.csv and venues.csv. These were clearly linked files but had completely different id systems, leading to a 40 entry DECODE map, something that could have been avoided if the two files were cross-referenced before setting up the ER model.

Another thing I learned relates to Oracle's behavior with imported data. Oracle Data Load doesn't retain dates in the standard ISO-8601 format but defaults to whatever NLS format your database session is set to at the time of import. So, a value of 1993-04-30 will appear as 30-APR-93 12.00 AM in Oracle. This isn't documented anywhere but became apparent when the first attempt to insert into the target tables with the select statement failed with an ORA-01858 error. After this discovery I had to use DUMP() to look at the actual byte representation of the numbers in the temporary(TEMP) table to work out where the formatting issue lay. Getting to grips with what implicit data type conversion Oracle is actually doing behind the scenes isn't something you learn from looking at examples on a white board.

The format of teams.csv where athletes_codes and coaches_codes were lists like ['1913366','1913367'] was something that I really struggled with. Direct loading was not an option here and I had to use an Oracle PL/SQL anonymous block and REGEXP_SUBSTR to get at the individual values in the lists, which I would have had to extract anyway if I hadn't of included these fields as lists directly. This has taught me why Python generated CSV files sometimes aren't good for database imports unless pre-processed.

Idempotency was another issue that I severely underestimated; I ran my import process a total of 15 times over development and testing. If I had comitted the DELETE and DROP statements I would have been struggling to know if my problem was in the existing data or the current load each time, but by using the statements I was able to get reliable results.

The Third Normal Form significantly eased my query design. Because each item of data is only stored once it was possible to write a lot simpler queries and I had no worries about update anomalies; with the Sport table it would have been tedious if the sport_name were in the Event table directly. The increased join complexity caused by using the TNF was an acceptable trade off for increased integrity.

Finally, the course reminded me of how rich and versatile SQL is. Using window functions, correlated sub-queries and the PL/SQL block all solved problems in a way that the basic select-where statements were not able to. I worked in oracle before, but never on that complicated project. To be honest I underestimated the amount of time it will consume. Making this project completely alone was a huge challenge, but I can say now I understand it fully, and I learned a lot, so It was worth definitely it.